

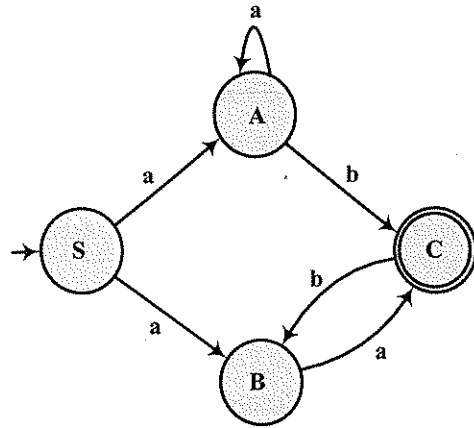
S.3.25. $L_{S.3.25}$ dili, $\{a, b, c\}$ alfabesinde, içinde **abc** aldizgisi bulunan dizgiler kümesi olarak tanımlanıyor.

- $L_{S.3.25}$ dilini tanıyan NFA'nın geçiş çizeneğini oluşturunuz. Oluşturacağınız geçiş çizeneği λ -geçişi içermesin.
- $L_{S.3.25}$ dilini tanıyan DFA'nın geçiş çizelgesini bulunuz. DFA'nın durumlarını $S_0, S_1, \dots$ diye adlandırınız.
- Bulduğunuz DFA'yı ingirgeyerek, $L_{S.3.25}$ dilini tanıyan en az durumlu DFA'nın geçiş çizeneğini bulunuz (durumları $S, T, U, V, \dots$ diye adlandırınız).
- $L'_{S.3.25}$ dili, $\{a, b, c\}$ alfabesinde, içinde **abc** aldizgisi bulunmayan dizgiler kümesi olarak tanımlanıyor. $L'_{S.3.25}$ dilini tanıyan en az durumlu DFA'nın geçiş çizeneğini bulunuz.

Not : c'de bulduklarınızdan yararlanınız.

- $L'_{S.3.25}$ dilini türeten düzgün bir dilbilgisi tanımlayınız. Tanımladığınız dilbilgisinin yeniden yazma kuralları arasında, yararsız olduğu açıkça görülenler var mı? Varsa hangileri?

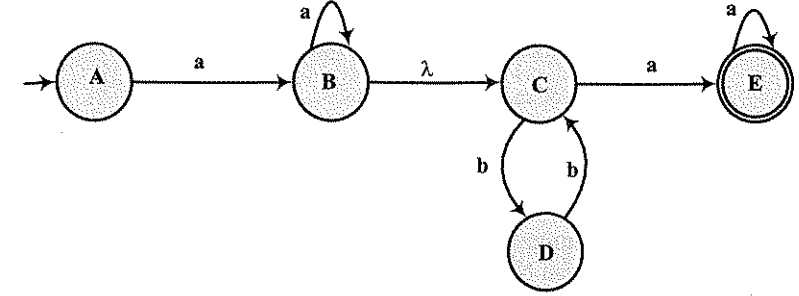
S.3.26.



Yukarıda geçiş çizeneği verilen NFA için:

- Makinenin tanıdığı dili türeten bir dilbilgisi tanımlayınız.
- Makinenin tanıdığı düzgün kümeyi tanımlayan bir düzgün deyim yazınız (denklemleri yazıp çözmeden, düzgün deyimi doğrudan yazınız).
- NFA'ya eşdeğer DFA'nın geçiş çizelgesi ile geçiş çizeneğini, durumları $q_0, q_1, q_2, \dots$ diye adlandırarak bulunuz.

Soru 3.27.



- Sezgisel olarak, geçiş çizeneği verilen NFA'nın tanıdığı dili tanımlayan bir düzgün deyim oluşturunuz (düzgün deyimi denklem sistemi ile sistematik olarak bulmanız istenmiyor).
- Bulduğunuz düzgün deyimden bakarak ve yalnız 3 sözdizim değişkeni kullanarak, sezgisel olarak makinenin tanıdığı dili türeten sol-doğrusal bir dilbilgisi oluşturunuz (dilbilgisinin geçiş çizeneğinden sistematik olarak oluşturmamız istenmiyor).
- Geçiş çizeneği verilen NFA'ya denk DFA'yı bulmanız isteniyor. Bunun için, önce λ -geçişini yok ederek λ -geçişsiz çizeneği bulunuz. Sonra da λ -geçişsiz çizeneğe bir ya da birkaç durum ekleyerek sezgisel olarak denk DFA'nın çizeneğini bulunuz (denk DFA'yı sistematik yöntemle bulmanız istenmiyor). Bulduğunuz DFA için geçiş çizelgesini oluşturunuz.

S.3.28. $L_{S.3.28}$ dili $\{a, b\}$ alfabesinde içindeki a'ların sayısı b'lerin sayısına eşit olan, ve de tüm örneklerinde a'ların sayısı ile b'lerin sayısı arasındaki fark en çok iki (± 2) olan dizgiler kümesi olarak tanımlanıyor. Buna göre örneğin **abaabaabbb** dizgisi $L_{S.3.28}$ 'de yer almıyor çünkü bu dizginin öneki olan **abaabaa**'da a'ların sayısı b'lerin sayısından 3 fazladır. Benzer nedenle **abbbbbaaa** dizgisi de $L_{S.3.28}$ 'de yer almıyor. Buna karşılık **abbbbaa** ve **babbabaa** dizgileri $L_{S.3.28}$ 'de yer alıyor.

- Durumların anlamlarını belirterek, $L_{S.3.28}$ dilini tanıyan FA'nın geçiş çizeneğini oluşturunuz. Oluşturduğunuz geçiş çizeneği λ -geçişi içermesin.
- $L_{S.3.28}$ dilini tanıyan FA'nın geçiş çizeneğinden, $L_{S.3.28}$ dilini türeten bir tür-3 dilbilgisinin ($G_{S.3.28}$) tanımını oluşturunuz.
- Bulduğunuz tür-3 dilbisinden, en çok 3 değişkenli sağ doğrusal bir dilbilgisi oluşturunuz.

S.3.29. $G_{S.3.29} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b\}$$

$$P : S \Rightarrow abbA \mid bB$$

$$A \Rightarrow bA \mid \lambda$$

$$B \Rightarrow aB \mid \lambda$$

- Yukarıda tanımı verilen sağ-doğrusal dilbilgisi tarafından türetilen dili ($L_{S.3.29}$) tanıyan NFA'nın geçiş çizeneğini oluşturunuz.
- Oluşturduğunuz geçiş çizeneği λ -geçişleri içeriyorsa, λ -geçiş içermeyen eşdeğer geçiş çizeneğini bulunuz.
- Denklemleri yazıp çözmeden, $L_{S.3.29}$ 'u tanımlayan bir düzgün deyim yazınız.

S.3.30. $L_{S.3.30}$ dili, $\{a, b, c\}$ alfabesinde, içinde aaa ve ccc alt dizgileri bulunmayan dizgiler kümesi olarak tanımlanıyor. Başka bir deyişle $L_{S.3.30}$ dilindeki her dizgide, eğer aa dizgi sonu değilse aa 'dan sonra mutlaka bir b ya da c bulunması; benzer biçimde eğer cc dizgi sonu değilse cc 'den sonra mutlaka bir a ya da b bulunması gerekiyor. $L_{S.3.30}$ dili λ 'yı içeriyor.

- Durumları $S, A, B, \dots$ diye adlandırarak $L_{S.3.30}$ 'u tanıyan NFA'nın (λ -geçişsiz) geçiş çizeneğini oluşturunuz.
- $L_{S.3.30}$ dilini türeten bir tür-3 dilbilgisi yazınız.

S.3.31. $L_{S.3.31}$ dili, $\{a, b, c\}$ alfabesinde, içindeki her b 'den önce en az bir a ; içindeki her c 'den sonra da en az iki a bulunan dizgiler kümesi olarak tanımlanıyor ($L_{S.3.31}$ dili λ 'yı içeriyor).

- $L_{S.3.31}$ 'i tanımlayan bir düzgün deyim yazınız ve $L_{S.3.31}$ 'i tanıyan NFA'nın (λ -geçişsiz) geçiş çizeneğini oluşturunuz.
- $L_{S.3.31}$ dilini türeten bir tür-3 dilbilgisi yazınız.

S.3.32. $L_{S.3.32} = (abcd)^*e$

- $L_{S.3.32}$ dilini tanıyan NFA'nın geçiş çizeneğini oluşturunuz. Oluşturduğunuz geçiş çizeneği λ -geçişleri içerebilir.
- Eğer geçiş çizeneği λ -geçişleri içeriyorsa, bu geçişleri yok ederek λ -geçiş içermeyen eşdeğer geçiş çizeneğini bulunuz.
- $L_{S.3.32}$ 'yi tanımlayan düzgün deyim bakarak, $L_{S.3.32}$ 'yi türeten sağ doğrusal bir dilbilgisi oluşturunuz.

S.3.33. $L_{S.3.33}$ dili $\{0, 1\}$ alfabesinde içinde tam üç tane 1 bulunan ve iki 1 arasında en az bir tane 0 bulunan dizgiler kümesi olarak tanımlanıyor. $L_{S.3.33}$ 'ü türeten bağlamdan bağımsız bir dilbilgisi tanımlayınız.

S.3.34. $L_{S.3.34}$ dili $\{0, 1\}$ alfabesinde içindeki her 1 'den önce en az iki tane 0 bulunan ve boş olmayan (en az bir uzunluğundaki) dizgiler kümesi olarak tanımlanıyor.

- $L_{S.3.34}$ 'ü türeten sağ doğrusal (*right linear*) bir dilbilgisi tanımlayınız.
- $L_{S.3.34}$ 'ü tanıyan bir sonlu özdevinirin (NFA) geçiş çizeneğini oluşturunuz.

S.3.35. $L_{S.3.35}$ dili $\{0, 1\}$ alfabesinde içindeki her 1 'den önce ve sonra en az birer tane 0 bulunan, iki 1 arasında ise en az iki tane 0 bulunan dizgiler kümesi olarak tanımlanıyor. $L_{S.3.35}$ dili λ yı içermiyor.

- $L_{S.3.35}$ dilini türeten, tek değişkenli, sağ doğrusal bir dilbilgisi ($G_{S.3.35}$) tanımlayınız. ($G_{S.3.35}$ 'in tek sözdizim değişkeni S olacak).
- $G_{S.3.35}$ 'i en az sayıda değişkeni olan bir tür-3 dilbilgisine dönüştürünüz ($G'_{S.3.35}$).
- $L_{S.3.35}$ 'i tanıyan sonlu durumlu makinenin geçiş çizeneğini en az durumla oluşturunuz.

Bölüm 4

Bağlamdan-Bağımsız Dilbilgisi ve Diller

4.1. Bağlamdan-Bağımsız Dilbilgisi

Programlama dilleri, derleyiciler, yorumlayıcılar, sözdizim çözümleyiciler, aritmetik deyimler, ... vb. birçok yazılım bileşeninin bünyesinde yer aldığı için bağlamdan-bağımsız dilbilgisi ve diller ile bu dilleri tanıyan makine modeli bilgisayar bilimleri ve mühendisliği açısından önem taşır.

3. bölümde tanımlandığı gibi bağlamdan-bağımsız ya da tür-2 dilbilgisi bir dördüldür:

$$G = \langle V_N, V_T, P, S \rangle$$

V_N : Sözdizim değişkenleri kümesi (sonlu bir küme)

V_T : Uç simgeler kümesi (sonlu bir küme) : $V_N \cap V_T = \Phi$

S : Başlangıç simgesi : $S \in V_N$

P : Bağlamdan-bağımsız dilbilgisinin yeniden yazma kuralları aşağıdaki gibi tanımlanmaktadır:

$$A \Rightarrow \beta \quad A \in V_N \quad \beta \in V^*$$

Daha önce örnek olarak verilen dilbilgilerinden $G_{3.1}$ ve $G_{3.4}$ bağlamdan-bağımsız dilbilgileridir. Aşağıda bağlamdan-bağımsız dilbilgileri için birkaç örnek daha verilmektedir.

Örnek 4.1.

$$G_{4.1} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S\}$$

$$V_T = \{+, -, *, /, (,), v, c\}$$

$$P: S \Rightarrow S+S \mid S-S \mid S*S \mid S/S \mid (S) \mid v \mid c$$

$G_{4.1}$ tarafından türetilen tümcelerden birkaçını bulalım.

$$\begin{aligned} S &\Rightarrow S*S \Rightarrow S*(S) \Rightarrow S*(S-S) \Rightarrow v*(S-S) \\ &\Rightarrow v*(v-S) \Rightarrow v*(v-c) \end{aligned}$$

$$\begin{aligned} S &\Rightarrow S*S \Rightarrow (S)*S \Rightarrow (S)*(S) \Rightarrow (S+S)*(S) \\ &\Rightarrow (S+S)*(S-S) \Rightarrow (S+S)*(S-S/S) \\ &\Rightarrow (v+S)*(S-S/S) \Rightarrow (v+c)*(S-S/S) \\ &\Rightarrow (v+c)*(v-S/S) \Rightarrow (v+c)*(v-v/S) \\ &\Rightarrow (v+c)*(v-v/c) \end{aligned}$$

$$\begin{aligned} S &\Rightarrow S+S \Rightarrow S*S+S \Rightarrow (S)*S+S \Rightarrow (S/S)*S+S \\ &\Rightarrow (S/(S))*S+S \Rightarrow (S/(S-S))*S+S \\ &\Rightarrow (v/(S-S))*S+S \Rightarrow (v/(v-S))*S+S \\ &\Rightarrow (v/(v-c))*S+S \Rightarrow (v/(v-c))*v+S \\ &\Rightarrow (v/(v-c))*v+c \end{aligned}$$

Yukarıdaki örneklerden, bu bağlamdan-bağımsız dilbilgisi tarafından türetilen $L(G_{4.1})$ dilinin temel aritmetik işlemler, değişkenler (v), değişmezler (c) ve parantezlerden oluşan geçerli aritmetik ifadeleri içeren dil olduğu görülmektedir.

Örnek 4.2.

$$G_{4.2} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b, c, d\}$$

$$P: S \Rightarrow aAdd$$

$$A \Rightarrow aAd \mid Ad \mid bBcc$$

$$B \Rightarrow bBc \mid Bc \mid \lambda$$

$G_{4.2}$ tarafından türetilen tümcelerden birkaçını bulalım.

$$S \Rightarrow aAdd \Rightarrow abBccdd \Rightarrow abccdd$$

$$S \Rightarrow aAdd \Rightarrow aaAddd \Rightarrow aabBccddd \Rightarrow aabccddd$$

$$\begin{aligned} S &\Rightarrow aAdd \Rightarrow aaAddd \Rightarrow aaAdddd \Rightarrow aaaAddddd \\ &\Rightarrow aaabBccddddd \Rightarrow aaabBccddddd \\ &\Rightarrow aaabbBccccddddd \Rightarrow aaabbccccddddd \end{aligned}$$

Yukarıdaki örneklerden, bu bağlamdan-bağımsız dilbilgisi tarafından türetilen $L(G_{4.2})$ dilinin aşağıdaki gibi tanımlanabileceği görülmektedir:

$$L(G_{4.2}) = \{a^n b^m c^k d^p \mid n, m, p, k \geq 1, p > n, k > m\}$$

Örnek 4.3.

$$G_{4.3} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, X, Y, Z\}$$

$$V_T = \{a, b, c\}$$

$$P: S \Rightarrow XY$$

$$X \Rightarrow aXbb \mid aZbb \mid abb$$

$$Y \Rightarrow cY \mid c$$

$$Z \Rightarrow Zb \mid Xb$$

$G_{4.3}$ tarafından türetilen tümcelerden birkaçını bulalım.

$$S \Rightarrow XY \Rightarrow abbY \Rightarrow abbcY \Rightarrow abbcc$$

$$S \Rightarrow XY \Rightarrow aXbbY \Rightarrow aaZbbbbY \Rightarrow aaZbbbbbbY$$

$$\Rightarrow aaXbbbbbbY \Rightarrow aaabbbbbbbY \Rightarrow aaabbbbbbbbc$$

Yukarıdaki örneklerden, bu bağlamdan-bağımsız dilbilgisi tarafından türetilen $L(G_{4.3})$ dilinin aşağıdaki gibi tanımlanabileceği görülmektedir:

$$L(G_{4.3}) = \{a^n b^m c^k \mid n \geq 1, k \geq 1, m \geq 2n\}$$

Örnek 4.4.

$$G_{4.4} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A\}$$

$$V_T = \{0, 1\}$$

$$P: S \Rightarrow 0S0 \mid 1S1 \mid 0A0 \mid 1A1$$

$$A \Rightarrow 0A1 \mid 1A0 \mid 01 \mid 10$$

$G_{4.4}$ tarafından türetilen tümcelerden birkaçını bulalım:

$$S \Rightarrow 0S0 \Rightarrow 01A10 \Rightarrow 010A110 \Rightarrow 01010110$$

$$S \Rightarrow 0S0 \Rightarrow 01S10 \Rightarrow 011S110 \Rightarrow 0110A1110 \\ \Rightarrow 01100A11110 \Rightarrow 011001011110$$

$$S \Rightarrow 0S0 \Rightarrow 00S00 \Rightarrow 000S000 \Rightarrow 0001A1000 \\ \Rightarrow 00010A11000 \Rightarrow 000100A111000 \\ \Rightarrow 0001000A1111000 \Rightarrow 0001000101111000$$

Yukarıdaki örneklerden, bu bağlamdan-bağımsız dilbilgisi tarafından türetilen $L(G_{4.4})$ dilinin aşağıdaki gibi tanımlanabileceği görülmektedir:

$$L(G_{4.4}) = \{ u v (v')^R u^R \mid u, v \in (0+1)^*, |u| \geq 1, |v| \geq 1 \}$$

Yukarıdaki gösterimde u^R u 'nun tersine, $(v')^R$ ise v 'nin tümlerinin tersine eşittir.

4.2. Türetme Ağacı

Bir dilbilgisi tarafından türetilen tümcesel yapı ve tümceler aşağıdaki gibi gösterilebilir:

$$S \Rightarrow \beta_1 \Rightarrow \beta_2 \Rightarrow \beta_3 \dots \dots \dots \Rightarrow \beta_{n-1} \Rightarrow \beta_n \Rightarrow w \quad \beta_i \in V^*, w \in V_T^*$$

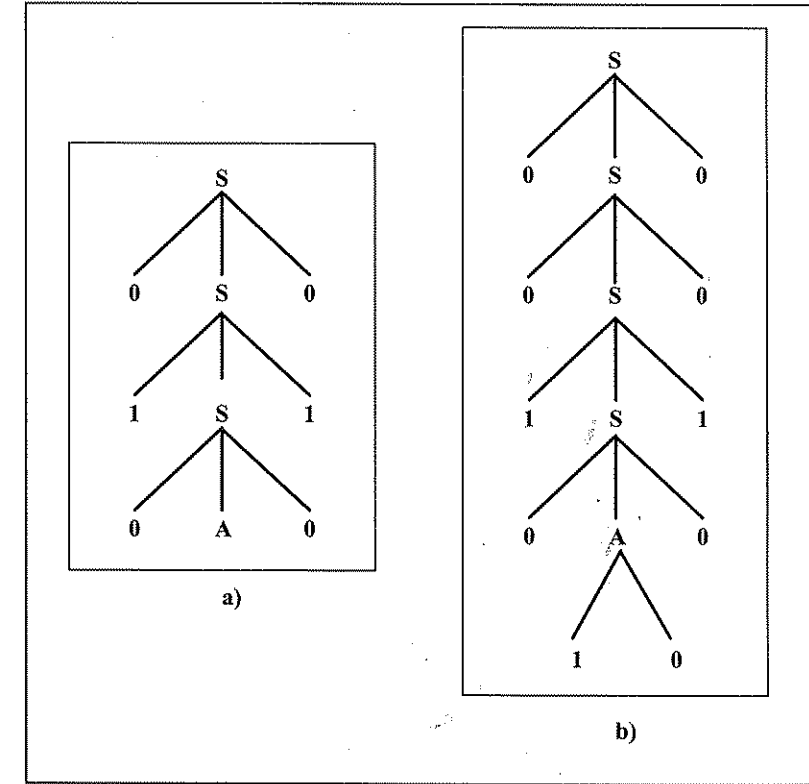
Tümcesel yapılar
Tümce

Bağlamdan-bağımsız dillerin her tümcesel yapısına ve her tümcesine bir türetme ya da ayrıştırma ağacı (*derivation or parsing tree*) karşı gelir. Örneğin $G_{4.4}$ tarafından türetilen:

$$\beta = 010A010 \quad \text{tümcesel yapısı ile}$$

$$w = 0010100100 \quad \text{tümcesine}$$

karşı gelen türetme ağaçları Çizim 4.1'de görülmektedir.



Çizim 4.1. $G_{4.4}$ İçin Türetme Ağacı Örnekleri

Türetme ya da Ayrıştırma Ağacının Tanımı

- Ağacın kökünün etiketi S 'dir.
- Kök dışındaki ara düğümlerin etiketleri sözdizim değişkenleridir ($A \in V_N$)
- Eğer ağaç bir tümcesel yapıya karşı geliyorsa, yaprakların etiketleri sözdizim değişkenleri ya da uç simgeler olabilir ($X \in V$). Eğer ağaç bir tümceye karşı geliyorsa yaprakların etiketleri yalnız uç simgeler ($a \in V_T$) olabilir.
- Eğer bir ara düğümün etiketi A , bu ara düğümün hemen altındaki düğümlerin etiketleri ise soldan sağa $X_1, X_2, X_3, \dots, X_k$ ise, dilbilgisinin yeniden yazma kuralları arasında

$$A \Rightarrow X_1 X_2 X_3 \dots X_k \quad X_1, X_2, X_3, \dots, X_k \in V$$

kuralı yer almalıdır.

- Eğer bir düğümün etiketi λ ise, bu düğüm bir uç düğüm (yaprak) olmalı ve bu düğümün kardeşi bulunmamalıdır.

Soldan ve Sağdan Türetme

Yapraklarının etiketleri uç simgeler olan her türetme ağacına dilin bir tümcesi karşı gelir. Eğer dil belirgin (*unambiguous*) bir dil ise de, dilin her tümcesine bir türetme ağacı karşı gelir. Eğer dilin bir tümcesine birden çok türetme ağacı karşı geliyorsa, dil belirgin olmayan (*ambiguous*) bir dildir. Belirginlik dilbilgisinin ve dilin en temel özelliklerinden biridir. Tümceleri birden çok anlam taşıyan belirgin olmayan dillerin uygulamada hiçbir değeri yoktur. Bu nedenle, kullanılan dillerin tümünün belirginlik özelliğini sağladığı varsayılacaktır. Buna göre dilin her tümcesine bir türetme ağacı, yapraklarının etiketleri uç simgeler olan her türetme ağacına da bir tümce karşı geldiğini söyleyebiliriz.

Dilin her tümcesine bir türetme ağacı karşı geldiği gibi, bir soldan türetme, bir de sağdan türetme karşı gelir.

Eğer bir tümce türetilirken, her adımda en soldaki değişkene bir türetme uygulanıyorsa, yapılan türetmeye soldan türetme (*leftmost derivation*) denir. Soldan türetmenin her ara adımında, eğer tümcesel yapı birden çok değişken içeriyorsa, öncelik en soldaki değişkene verilir.

Eğer bir tümce türetilirken, her adımda en sağdaki değişkene bir türetme uygulanıyorsa, yapılan türetmeye sağdan türetme (*rightmost derivation*) denir. Sağdan türetmenin her ara adımında, eğer tümcesel yapı birden çok değişken içeriyorsa, öncelik en sağdaki değişkene verilir.

Çizim 4.2'de, $L(G_{4.1})$ dilinin iki tümcesine karşı gelen türetme ağacı görülmektedir. Çizimdeki türetme ağaçlarına karşı gelen tümceler sırasıyla aşağıdakilerdir.

$$w_1 = (v+c)^*(v-c)$$

$$w_2 = v/(v-c)+v^*(v+c)$$

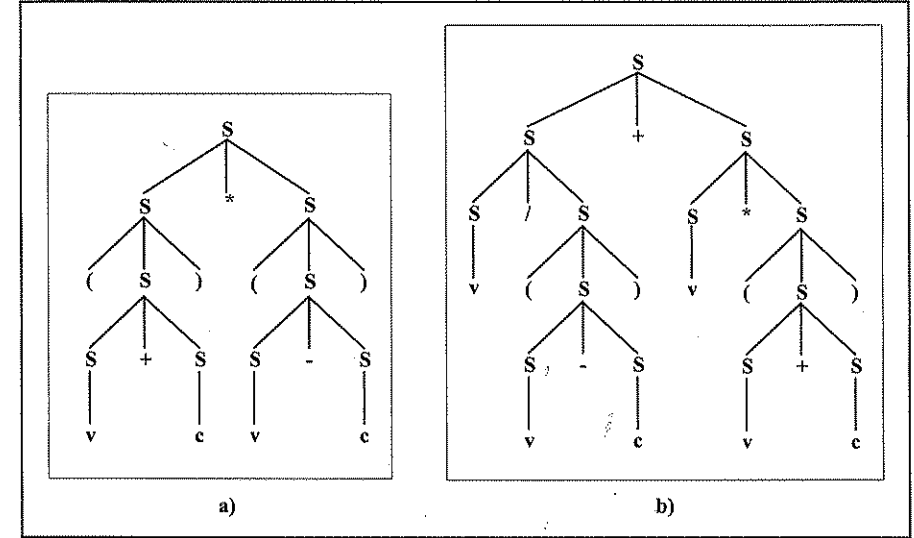
Bu iki tümceye karşı gelen soldan ve sağdan türetmeler aşağıdaki gibidir.

a) w_1 'in soldan türetilmesi:

$$\begin{aligned} S &\Rightarrow S^*S \Rightarrow (S)^*S \Rightarrow (S+S)^*S \Rightarrow (v+S)^*S \\ &\Rightarrow (v+c)^*S \Rightarrow (v+c)^*(S) \Rightarrow (v+c)^*(S-S) \\ &\Rightarrow (v+c)^*(v-S) \Rightarrow (v+c)^*(v-c) \end{aligned}$$

b) w_1 'in sağdan türetilmesi:

$$\begin{aligned} S &\Rightarrow S^*S \Rightarrow S^*(S) \Rightarrow S^*(S-S) \Rightarrow S^*(S-c) \\ &\Rightarrow S^*(v-c) \Rightarrow (S)^*(v-c) \Rightarrow (S+S)^*(v-c) \\ &\Rightarrow (S+c)^*(v-c) \Rightarrow (v+c)^*(v-c) \end{aligned}$$



Çizim 4.2. $G_{4.1}$ için Türetme Ağacı Örnekleri

c) w_2 'in soldan türetilmesi:

$$\begin{aligned} S &\Rightarrow S+S \Rightarrow S/S+S \Rightarrow v/S+S \Rightarrow v/(S)+S \\ &\Rightarrow v/(S-S)+S \Rightarrow v/(v-S)+S \Rightarrow v/(v-c)+S \\ &\Rightarrow v/(v-c)+S^*S \Rightarrow v/(v-c)+v^*S \\ &\Rightarrow v/(v-c)+v^*(S) \Rightarrow v/(v-c)+v^*(S+S) \\ &\Rightarrow v/(v-c)+v^*(v+S) \Rightarrow v/(v-c)+v^*(v+c) \end{aligned}$$

d) w_2 'in sağdan türetilmesi:

$$\begin{aligned} S &\Rightarrow S+S \Rightarrow S+S^*S \Rightarrow S+S^*(S) \Rightarrow S+^*(S+S) \\ &\Rightarrow S+S^*(S+c) \Rightarrow S+S^*(v+c) \Rightarrow S+v^*(v+c) \\ &\Rightarrow S/S+v^*(v+c) \Rightarrow S/(S)+v^*(v+c) \\ &\Rightarrow S/(S-S)+v^*(v+c) \Rightarrow S/(S-c)+v^*(v+c) \\ &\Rightarrow S/(v-c)+v^*(v+c) \Rightarrow v/(v-c)+v^*(v+c) \end{aligned}$$

4.3. Dilbilgisinin Yalınlaştırılması

Bağlamdan-bağımsız bir dilbilgisi verildiğinde, bu dilbilgisinin daha az sayıda değişken ve kural içeren, ve belirli biçimdeki kuralları içermeyen bir dilbilgisine dönüştürülmesine dilbilgisinin yalınlaştırması denilir. Aşağıda dilbilgisinin yalınlaştırılması ile ilgili bir dizi tanım ve algoritma verilmektedir.

4.3.1. Özyineli Kural, Özyineli Değişken

Bir yeniden yazma kuralının solundaki değişken, kuralın sağında da yer alıyorsa, başka bir deyişle kural $(A \Rightarrow \alpha_1 A \alpha_2)$ biçiminde ise, bu kurala **özyineli kural** (*recursive rule*), A değişkenine de **özyineli değişken** (*recursive variable*) denir.

Eğer bir yeniden yazma kuralının solundaki değişken, aynı zamanda kuralın sağındaki ilk simge ise, başka bir deyişle kural $(A \Rightarrow A \alpha_2)$ biçiminde ise, bu kurala **doğrudan özyineli kural** (*direct recursive rule*) denir.

Bazı işlemlerde dilbilgisinin başlangıç değişkeninin (S) özyineli olması istenmez. Dilbilgisine yeni bir değişken (S') ve yeni bir kural ($S' \Rightarrow S$) ekleyip, yeni eklenen değişkeni başlangıç değişkeni yaparak dilbilgisinin bu özelliği taşıması sağlanabilir.

4.3.2. Yok Edilebilir Değişken

Eğer bir değişkenin yerine bir ya da birkaç türetme sonunda λ konulabiliyorsa,

$$A \Rightarrow \lambda \quad \text{ya da} \quad A^* \Rightarrow \lambda$$

bu değişkene yok edilebilir (*nullable*) değişken denir. Bazı işlemlerde, başlangıç değişkeni dışında dilbilgisinde yok edilebilir değişken bulunmaması; başka bir deyişle $(S \Rightarrow \lambda)$ dışında λ -kuralının bulunmaması istenir. Buna göre, eğer dil λ 'yı içermiyorsa ($\lambda \notin L$), dilbilgisinde hiç λ -kuralı bulunmayacaktır. Eğer dil λ 'yı içeriyece de ($\lambda \in L$), dilbilgisindeki tek λ -kuralı $(S \Rightarrow \lambda)$ olacaktır.

Bir algoritmayla, dilbilgisinin yok edilebilir değişkenlerinin hangileri olduğu kolaylıkla bulunabilir. Bir başka algoritmayla da dilbilgisi, başlangıç değişkeni dışında yok edilebilir değişkeni bulunmayan eşdeğer bir dilbilgisine dönüştürülebilir. Bu algoritmalar aşağıda verilmektedir.

Algoritma 4.1. Yok edilebilir değişkenlerin bulunması

1. $T_L = \{ A \mid (A \Rightarrow \lambda) \in P \}$;
 2. $T_{Eski} = \Phi$;
 3. $(T_L = T_{Eski})$ oluncaya kadar aşağıdaki işlemleri tekrarla:
 - 3.1. $T_{Eski} = T_L$;
 - 3.2. V_N 'deki her değişken (A) için aşağıdaki işlemleri tekrarla:

eğer en az bir $(A \Rightarrow \alpha)$ kuralı için $\alpha \in (T_L)^*$ ise

A 'yı T_L 'ye ekle;

son 3.2;
- son 3;

Algoritma 4.2. Başlangıç değişkeni dışında yok edilebilir değişkeni bulunmayan eşdeğer dilbilgisinin bulunması

1. P 'deki tüm $(A \Rightarrow \lambda)$ kurallarını çıkar;
 2. Eğer $\lambda \in L(G)$ ise P 'ye $(S \Rightarrow \lambda)$ kuralını ekle;
 3. P 'deki her $(A \Rightarrow \alpha)$ kuralı için aşağıdaki işlemleri tekrarla:
 - 3.1. Eğer α yok edilebilir değişkenler içeriyorsa, bu değişkenlerin bir ya da birkaçını α 'dan çıkararak oluşturulabilen her α_i için aşağıdaki işlemleri tekrarla:

P 'ye $(A \Rightarrow \alpha_i)$ kuralını ekle;

son 3.1;
- son 3;

Örnek 4. 5.

$$G_{4.5} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{ S, A, B, C \}$$

$$V_T = \{ a, b \}$$

$$P: S \Rightarrow ACA$$

$$A \Rightarrow B \mid C \mid aAa$$

$$B \Rightarrow bB \mid b$$

$$C \Rightarrow cC \mid \lambda$$

$G_{4.5}$ 'e eşdeğer, başlangıç değişkeni dışında yok edilebilir değişken içermeyen bir dilbilgisi bulmak için önce dilbilgisinin yok edilebilir değişkenlerini bulalım. Algoritma kullanılarak bu değişkenler aşağıdaki gibi bulunur:

$$T_L = \{C, A, S\}$$

Dilbilgisinin başlangıç değişkeni yok edilebilir bir değişken olduğuna göre dilbilgisinin türettiği dil λ 'yı içermektedir. Bu nedenle eşdeğer dilbilgisinde ($S \Rightarrow \lambda$) kuralı yer alacaktır. Algoritma 4.2 kullanılarak, $G_{4.5}$ 'e eşdeğer, başlangıç değişkeni dışında yok edilebilir değişken içermeyen dilbilgisi aşağıdaki gibi bulunur:

$$G'_{4.5} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B, C\}$$

$$V_T = \{a, b\}$$

$$P: S \Rightarrow ACA \mid AC \mid CA \mid AA \mid A \mid C \mid \lambda$$

$$A \Rightarrow B \mid C \mid aAa \mid aa$$

$$B \Rightarrow bB \mid b$$

$$C \Rightarrow cC \mid c$$

4.3.3. Birim Türetme Kuralları

Dilbilgisindeki yeniden yazma kurallarından ($A \Rightarrow B$) biçiminde olanlara "birim türetme kuralı" (*unit production rule*) ya da "zincir kuralı" (*chain rule*) denir. Birim türetme kuralları tümcelerini türettilmesini geciktiren kurallardır. Dilbilgisinin kullanım amacı da dilin tümcelerini türetmek olduğuna göre, birim türetme kuralları içermeyen dilbilgilerinin kullanılması daha uygundur.

Bağlamdan-bağımsız bir dilbilgisi verildiğinde, bu dilbilgisine eşdeğer, birim türetme kuralları içermeyen bir dilbilgisi bulunabilir. Bu amaçla önce dilbilgisindeki her değişken (A) için, bu değişkenden türetilen değişkenler kümesinin (T_A) bulunması gerekir.

$$\forall A \in V_N \Rightarrow T_A = \{B \mid B \in V_N, A \Rightarrow^* B\}$$

Her değişkenden (A) türetililecek değişkenler kümesi Algoritma 4.3 ile bulunabilir.

Algoritma 4.3. Bir değişkenden (A) türetilen değişkenler kümesinin bulunması

```

1.   $T_A = \{A\}$ ;
2.   $T_{Eski} = \Phi$ ;
3.  ( $T_A = T_{Eski}$ ) oluncaya kadar aşağıdaki işlemleri tekrarla:
    3.1.  $T_{Yeni} = T_A - T_{Eski}$ ;
    3.2.  $T_{Eski} = T_A$ ;
    3.3.  $T_{Yeni}$ 'deki her değişken ( $B$ ) için aşağıdaki işlemleri tekrarla:
        3.3.1. her ( $B \Rightarrow C$ ) kuralı için aşağıdaki işlemleri tekrarla:
             $T_A = T_A \cup \{C\}$ ;
        son 3.3.1;
    son 3.3;
son 3;
```

Her değişken için, bu değişkenden türetilen değişkenler kümesi bulunduktan sonra, Algoritma 4.4'ü kullanarak birim türetme kuralı içermeyen eşdeğer dilbilgisi bulunabilir.

Algoritma 4.4. Birim türetme kuralı içermeyen dilbilgisinin bulunması

```

1.   $P = P - \{A \Rightarrow B \mid A, B \in V_N\}$ ;
2.   $V_N$ 'deki her değişken ( $A$ ) için, eğer  $T_A$ 'da  $A$ 'dan başka en az bir değişken varsa aşağıdaki işlemleri tekrarla:
    2.1.  $T_A = T_A - \{A\}$ ;
    2.2.  $T_A$ 'daki her değişken ( $B$ ) için aşağıdaki işlemleri tekrarla:
        B kurallarının tümü ( $B \Rightarrow \beta_1 \mid \beta_2 \mid \beta_3 \mid \dots \mid \beta_k$ ) olmak üzere,
         $P = P \cup \{A \Rightarrow \beta_1 \mid \beta_2 \mid \beta_3 \mid \dots \mid \beta_k\}$ ;
    Son 2.2;
Son 2;
```


Örnek 4. 6.

$$G_{4.6} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B, C\}$$

$$V_T = \{a, b, c\}$$

$$P: S \Rightarrow A \mid AA \mid AC \mid CA \mid ACA \mid \lambda$$

$$A \Rightarrow B \mid aAa \mid aa$$

$$B \Rightarrow C \mid bB \mid b$$

$$C \Rightarrow cC \mid c$$

$G_{4.6}$ 'ya eşdeğer, birim türetme kuralı içermeyen bir dilbilgisi bulmak için önce S, A, B ve C değişkenlerinden türetilen değişken kümelerinin bulunması gerekir. Algoritma 4.3 kullanılarak bu kümeler aşağıdaki gibi bulunur.

$$T_S = \{S, A, B, C\}$$

$$T_A = \{A, B, C\}$$

$$T_B = \{B, C\}$$

$$T_C = \{C\}$$

Daha sonra Algoritma 4.4 kullanılarak $G_{4.6}$ 'ya eşdeğer aşağıdaki dilbilgisi bulunur.

$$G'_{4.6} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B, C\}$$

$$V_T = \{a, b, c\}$$

$$P: S \Rightarrow AA \mid AC \mid CA \mid ACA \mid aAa \mid aa \mid bB \mid b \mid$$

$$cC \mid c \mid \lambda$$

$$A \Rightarrow aAa \mid aa \mid bB \mid b \mid cC \mid c$$

$$B \Rightarrow bB \mid b \mid cC \mid c$$

$$C \Rightarrow cC \mid c$$

4.3.4. Yararsız Değişken, Simge ve Kurallar

Eğer bir dilbilginin uç simgelerinden biri, bu dilbilgisi tarafından türetilen tümcelerden hiçbirinde yer almıyorsa, bu uç simgeye yararsız simge (*useless symbol*) denilir.

Eğer bir dilbilgisinin değişkenlerinden biri (A), en az bir tümce türetilirken elde edilen tümcesel yapıların en az birinde yer alıyorsa:

$$S \Rightarrow \alpha_1 A \alpha_2 \Rightarrow w$$

bu değişken yararlı (*usefull*) bir değişkendir. Bu özelliği taşımayan değişkenlere ise yararsız değişken (*useless variable*) denilir. Yararsız değişken hiçbir tümcenin türetilmesinde kullanılmayan; kullanıldığı türetmelerin hiçbirinin sonunda ise bir tümce elde edilemeyen değişkendir.

Benzer biçimde, eğer bir dilbilgisinin yeniden yazma kurallarından biri ($A \Rightarrow \beta$), en az bir tümcenin türetilmesi sırasında kullanılıyorsa, bu kural yararlı bir kuraldır (*usefull rule*). Hiçbir tümcenin türetilmesinde kullanılmayan kurallara ise yararsız kurallar (*useless rules*) adı verilir.

Eğer bağlamdan-bağımsız bir dilbilgisi yararsız değişken, uç simge ya da kural içeriyorsa, bu dilbilgisine eşdeğer olan ve yararsız uç simge, değişken ve kural içermeyen bir dilbilgisi bulunabilir. Bunun için de öncelikle değişkenlerden hangilerinin yararlı olduğunun bulunması gerekir. Bir değişkenin (A) yararlı bir değişken olabilmesi için:

1. Bu değişkenden başlanarak, en az bir uç simge dizgisinin türetilmesi:

$$A \Rightarrow u \quad u \in V_T^* \quad (u : \text{dilin bir tümcesi olması şart değil})$$

2. Başlangıç değişkeninden bu değişkene ulaşmanın mümkün olması:

$$S \Rightarrow \alpha_1 A \alpha_2$$

gereklidir. Buna göre, bağlamdan-bağımsız bir dilbilgisi (G) verildiğinde, yararsız değişken ve kurallar atılarak eşdeğer bir dilbilgisinin bulunması iki adımda gerçekleştirilir.

Birinci adımda, Algoritma 4.5 kullanılarak uç simge dizgisi türeten değişkenler kümesi bulunur. Bu kümede yer almayan değişkenler yararsız değişkenlerdir. Dilbilgisinden bu değişkenler ile bu değişkenlerin yer aldığı kurallar çıkarılarak eşdeğer bir dilbilgisi (G') bulunur.

İkinci adımda, Algoritma 4.6 kullanılarak, birinci adımda bulunan dilbilgisindeki (G') değişkenlerden hangilerine başlangıç değişkeninden ulaşılabilirdiği bulunur. Başlangıç değişkeninden ulaşamayan değişkenler de yararsız değişkenlerdir. Dilbilgisinden (G') bu değişkenler ile bu değişkenlerin yer aldığı kurallar çıkarılarak eşdeğer dilbilgisi (G'') bulunur.

Aşağıda uç simge dizgisi türeten değişkenler kümesi (T_D) ile başlangıç değişkeninden ulaşılabilen değişkenler kümesini (T_U) bulmayı sağlayan algoritmalar (sırasıyla Algoritma 4.5 ve Algoritma 4.6) sunulmaktadır.

Algoritma 4.5. Uç simge dizgisi türeten değişkenler kümesinin bulunması.

1. $T_D = \{A \mid (A \Rightarrow w) \in P \text{ ve } w \in V_T^*\};$
 2. $(T = T_{Eski})$ oluncaya kadar aşağıdaki işlemleri tekrarla:
 - 2.1. $T_{Eski} = T_D;$
 - 2.2. Dilbilgisindeki her değişken (A) için aşağıdaki işlemleri tekrarla:
 - 2.2.1. Her $(A \Rightarrow \beta)$ kuralı için aşağıdaki işlemleri tekrarla:

eğer $\beta \in (T_{Eski} \cup V_T)^*$ ise

A'yı T_D 'ye ekle;
- Son 2.2.1 ;
Son 2.2 ;
Son 2 ;

Algoritma 4.6. S'den ulaşılabilen değişkenler kümesinin bulunması

1. $T_U = \{S\};$
 2. $T_{Eski} = \Phi;$
 3. $(T_U = T_{Eski})$ oluncaya kadar aşağıdaki işlemleri tekrarla:
 - 3.1. $T_{Yeni} = T_U - T_{Eski};$
 - 3.2. $T_{Eski} = T_U;$
 - 3.3. T_{Yeni} 'deki her değişken için aşağıdaki işlemleri tekrarla:
 - 3.3.1. Her $(A \Rightarrow \beta)$ yeniden yazma kuralı için aşağıdaki işlemleri tekrarla:

β 'daki tüm değişkenleri T_U 'ya ekle;
- Son 3.3.1 ;
Son 3.3 ;
Son 3 ;

Örnek 4.7.

$$G_{4.7} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B, C, D, E, F\}$$

$$V_T = \{a, b\}$$

$$P: S \Rightarrow B \mid AC \mid BS$$

$$A \Rightarrow aA \mid aF$$

$$B \Rightarrow CF \mid b$$

$$C \Rightarrow cC \mid D$$

$$D \Rightarrow C \mid aD \mid BD$$

$$E \Rightarrow aA \mid BSA$$

$$F \Rightarrow bB \mid b$$

$G_{4.7}$ 'ye eşdeğer; yararsız değişken, simge ve kural içermeyen bir dilbilgisi bulmak için önce $G_{4.7}$ 'deki değişkenlerden hangilerinin uç simge dizgisi türeten değişkenler olduğunu bulmak gerekir. Uç simge dizgisi türeten değişkenler kümesi Algoritma 4.5 ile aşağıdaki gibi bulunur:

$$T_D = \{S, A, B, E, F\}$$

Yukarıdaki kümede yer almayan **C** ve **D** değişkenleri yararsız değişkenlerdir. Dilbilgisinden bu değişkenler ile bu değişkenlerin yer aldığı kuralları çıkaralım:

$$G'_{4.7} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B, E, F\}$$

$$V_T = \{a, b\}$$

$$P: S \Rightarrow B \mid BS$$

$$A \Rightarrow aA \mid aF$$

$$B \Rightarrow b$$

$$E \Rightarrow aA \mid BSA$$

$$F \Rightarrow bB \mid b$$

Şimdi de $G'_{4.7}$ 'deki değişkenlerden hangilerinin S'den ulaşılabilen değişkenler olduğunu bulmak gerekir. S'den ulaşılabilen değişkenler kümesi Algoritma 4.6 kullanılarak aşağıdaki gibi bulunur:

$$T_U = \{S, B\}$$

Yukarıdaki kümede yer almayan **A**, **E** ve **F** değişkenleri yararsız değişkenlerdir. Dilbilgisinden bu üç değişken ile bu değişkenlerin yer aldığı

kurallar çıkarıldığında a simgesinin de yararsız bir simge olduğu görülür. Bu simge de çıkarıldığında $G_{4.7}$ 'ye eşdeğer; yararsız simge, değişken ve kural içermeyen aşağıdaki dilbilgisi bulunur:

$$G_{4.7} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, B\}$$

$$V_T = \{b\}$$

$$P: S \Rightarrow B \mid BS$$

$$B \Rightarrow b$$

4.4. Normal Biçimler

Bağlamdan-bağımsız dilbilgileri için Chomsky Normal Biçimi (CNF : *Chomsky Normal Form*) ve Greibach Normal Biçimi (GNF : *Greibach Normal Form*) olarak adlandırılan iki normal biçim vardır. Bu normal biçimlerin tanımları ve bağlamdan-bağımsız bir dilbilgisini CNF ve GNF'e dönüştürmek için kullanılan algoritmalar aşağıda verilmektedir.

4.4.1. Chomsky Normal Biçimi

Tanım 4.1. Eğer bağlamdan-bağımsız bir dilbilgisinin yeniden yazma kurallarının tümü

$$S \Rightarrow \lambda$$

$$A \Rightarrow BC$$

$$A \Rightarrow a \quad : \quad A, B, C \in V_N, \quad a \in V_T$$

biçiminde ise, dilbilgisi Chomsky normal biçimindedir.

Önerme 4.1. Bağlamdan-bağımsız her dil için, bu dili türeten Chomsky normal biçiminde bir dilbilgisi bulunabilir.

Bağlamdan-Bağımsız Dilbilgisinin Chomsky Normal Biçimine Dönüştürülmesi

Bağlamdan-bağımsız bir dilbilgisi verildiğinde, bu dilbilgisine eşdeğer (aynı dili türeten) CNF dilbilgisini bulmak için, dilbilgisinin ($A \Rightarrow BC$) ve ($A \Rightarrow a$) biçimindeki yeniden yazma kuralları korunur. Diğer yeniden yazma kuralları atılarak yerlerine aşağıdaki gibi türetilen yeni kurallar konulur.

1. CNF'e uygun olmayan kuralın (K) sağ tarafının ilk simgesi bir değişken ya da bir uç simge olabilir.

1.1. Eğer ilk simge bir değişken ise kural (K) :

$$A \Rightarrow BX_2 X_3 \dots X_k \quad A, B \in V_N \quad X_2, X_3, \dots, X_k \in V$$

biçimindedir. Bu durumda değişkenler kümesine bir yeni değişken (örneğin D_1) eklenir ve (K) yerine aşağıdaki 2 kural konulur.

$$A \Rightarrow BD_1 \quad (K_1)$$

$$D_1 \Rightarrow X_2 X_3 \dots X_k \quad (K_2)$$

1.2. Eğer ilk simge bir uç simge ise, kural (K) :

$$A \Rightarrow aX_2 X_3 \dots X_k \quad A \in V_N \quad a \in V_T \quad X_2, X_3, \dots, X_k \in V$$

biçimindedir. Bu durumda değişkenler kümesine iki yeni değişken (örneğin C_1 ve D_1) eklenir ve (K) yerine aşağıdaki 3 kural konulur.

$$A \Rightarrow C_1 D_1 \quad (K_1)$$

$$D_1 \Rightarrow X_2 X_3 \dots X_k \quad (K_2)$$

$$C_1 \Rightarrow a \quad (K_3)$$

2. Eğer K 'nın yerine konulan kurallardan biri (K_2) CNF'e uygun değilse, K için yapılan işlemler K_2 için tekrarlanır ve tüm kurallar CNF'e uygun oluncaya kadar dönüştürme işlemleri sürdürülür.

Örnek 4.8.

$$G_{4.8} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b\}$$

$$P: S \Rightarrow aB \quad (1)$$

$$S \Rightarrow bA \quad (2)$$

$$A \Rightarrow aS \quad (3)$$

$$A \Rightarrow bAA \quad (4)$$

$$A \Rightarrow a \quad (5)$$

$$B \Rightarrow bS \quad (6)$$

$$B \Rightarrow aBB \quad (7)$$

$$B \Rightarrow b \quad (8)$$

Dilbilgisinin 8 yeniden yazma kuralı vardır. Bu kurallardan yalnız ikisi (5 ve 8) CNF'e uygundur. Diğer 6 kuralın yerine aşağıdaki kurallar konulur:

$$\begin{array}{llll}
 S \Rightarrow aB & \text{yerine} & S \Rightarrow C_a B & \text{ve} & C_a \Rightarrow a \\
 S \Rightarrow bA & \text{yerine} & S \Rightarrow C_b A & \text{ve} & C_b \Rightarrow b \\
 A \Rightarrow aS & \text{yerine} & A \Rightarrow C_a S & & \\
 A \Rightarrow bAA & \text{yerine} & A \Rightarrow C_b D_1 & \text{ve} & D_1 \Rightarrow AA \\
 B \Rightarrow bS & \text{yerine} & B \Rightarrow C_b S & & \\
 B \Rightarrow aBB & \text{yerine} & B \Rightarrow C_a D_2 & \text{ve} & D_2 \Rightarrow BB
 \end{array}$$

Sonuç olarak, $G_{4.8}$ 'e eşdeğer Chomsky normal biçimindeki dilbilgisi aşağıdaki gibi oluşur.

$$G'_{4.8} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B, C_a, C_b, D_1, D_2\}$$

$$V_T = \{a, b\}$$

$$P: S \Rightarrow C_a B \mid C_b A$$

$$A \Rightarrow C_a S \mid C_b D_1 \mid a$$

$$B \Rightarrow C_b S \mid C_a D_2 \mid b$$

$$C_a \Rightarrow a$$

$$C_b \Rightarrow b$$

$$D_1 \Rightarrow AA$$

$$D_2 \Rightarrow BB$$

4.4.2. Greibach Normal Biçimi

Tanım 4.2. Eğer bağlamdan-bağımsız bir dilbilgisinin yeniden yazma kurallarının tümü

$$S \Rightarrow \lambda$$

$$A \Rightarrow a\alpha \quad : \quad A \in V_N, \quad a \in V_T, \quad \alpha \in V_N^*$$

biçiminde ise, dilbilgisi Greibach normal biçimindedir.

Önerme 4.2. Bağlamdan-bağımsız her dil için, bu dili türeten Greibach normal biçiminde bir dilbilgisi bulunabilir.

Bağlamdan-bağımsız bir dilbilgisi GNF'e dönüştürülürken iki özellik (*lemma*) kullanılır. Dönüştürmenin nasıl yapıldığını vermeden önce bu iki özelliğin verilmesi gereklidir.

Lemma 4.1. Bağlamdan-bağımsız bir dilbilgisinin yeniden yazma kurallarından biri:

$$A \Rightarrow \alpha_1 B \alpha_2 \quad (k_1)$$

olsun. Eğer dilbilgisinin tüm B kuralları

$$B \Rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

ise, dilbilgisinden k_1 kuralını çıkarıp yerine

$$A \Rightarrow \alpha_1 \beta_1 \alpha_2 \mid \alpha_1 \beta_2 \alpha_2 \mid \dots \mid \alpha_1 \beta_n \alpha_2 \quad (k_2)$$

kuralları konulduğunda eşdeğer (aynı dili türeten) bir dilbilgisi elde edilir.

Lemma 4.2. Eğer bağlamdan-bağımsız bir dilbilgisi doğrudan özyineli (*direct recursive*) yeniden yazma kuralları içeriyorsa, bu dilbilgisine eşdeğer, doğrudan özyineli kural içermeyen bir dilbilgisi bulunabilir. Bunun için doğrudan özyineli kuralların yerine konulacak yeni kurallar aşağıdaki gibi bulunur.

Dilbilgisinin A kurallarının bir kesimi doğrudan özyineli ise, A kuralları iki gruba ayrılır:

$$A \Rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_r \quad (g_1)$$

$$A \Rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_s \quad (g_2)$$

A kurallarından doğrudan özyineli olanların (g_1) yerine aşağıdaki kurallar konulur:

$$A \Rightarrow \beta_i B \quad i = 1, 2, \dots, s \quad B : \text{yeni bir değişken}$$

$$B \Rightarrow \alpha_j \quad j = 1, 2, \dots, r$$

$$B \Rightarrow \alpha_j B \quad j = 1, 2, \dots, r$$

Bağlamdan-Bağımsız Dilbilgisinin Greibach Normal Biçimine Dönüştürülmesi

Chomsky normal biçiminde (CNF) bağlamdan-bağımsız bir dilbilgisi verildiğinde, aşağıdaki algoritmayı kullanarak bu dilbilgisine eşdeğer (aynı dili türeten) Greibach normal biçiminde (GNF) bir dilbilgisi bulunabilir. Buna göre, GNF'e dönüştürülecek dilbilgisinin önce CNF'e dönüştürülmesi gerekmektedir. GNF'e dönüştürme algoritması 3 adımdan oluşur.

1. Adım

1.1. Dilbilgisinin sözdizim değişkenleri $A_1, A_2, A_3, \dots, A_k$ gibi dizinli değişkenlerle değiştirilir. Bu değişiklik yapılırken S 'nin yerine dizin değeri en küçük olan (A_1) değişken konulur.

1.2. Lemma 4.1 kullanılarak yeniden yazma kurallarının tümü

$$A_i \Rightarrow A_j \gamma \quad j \geq i$$

koşulunu sağlayacak biçime dönüştürülür.

2. Adım

2.1. Lemma 4.2 kullanılarak doğrudan özyineli kuralların yerine yeni kurallar konulur. Bu adımın sonunda, dizin değeri en büyük (A_k) değişkenle başlayan tüm kurallar GNF'e uygun biçime dönüşmüş olur. Bu adımda dilbilgisine $B_1, B_2, \dots$ gibi yeni değişkenler de eklenir.

3. Adım.

3.1. Lemma 4.1 kullanılarak, $A_{k-1}, A_{k-2}, \dots, A_2, A_1$ sırasında tüm A_i kuralları GNF'e uygun biçime dönüştürülür.

3.2. Lemma 4.1 kullanılarak tüm B_i kuralları (sol tarafında 2. adımda eklenen değişkenlerin yer aldığı kurallar) GNF'e uygun biçime dönüştürülür.

3. adımın sonunda Greibach normal biçimindeki dilbilgisi elde edilmiş olur. Bundan sonra, istenirse değişkenler yeniden adlandırılabilir.

Örnek 4.9.

$$G_{4.9} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{A_1, A_2, A_3\} \quad A_1 : \text{Başlangıç değişkeni}$$

$$V_T = \{a, b\}$$

$$P : A_1 \Rightarrow A_2 A_3 \quad (1)$$

$$A_2 \Rightarrow A_3 A_1 \quad (2)$$

$$A_2 \Rightarrow b \quad (3)$$

$$A_3 \Rightarrow A_1 A_2 \quad (4)$$

$$A_3 \Rightarrow a \quad (5)$$

Dilbilgisi CNF biçimindedir. Ancak dilbilgisinin 1, 2 ve 4 numaralı yeniden yazma kuralları GNF'e uygun değildir. Dönüştürme algoritmasını kullanarak eşdeğer GNF dilbilgisini bulalım.

1. Kurallarda ($A_i \Rightarrow A_j \gamma$, $j \geq i$) koşulunun sağlanması.

Bu koşul 1 ve 2 numaralı kurallarda sağlanıyor; 4 numaralı kuralda sağlanmıyor. Lemma 4.1 kullanılarak bu kuralın yerine aşağıdaki kurallar konulur.

Önce $A_3 \Rightarrow A_1 A_2$ yerine $A_3 \Rightarrow A_2 A_3 A_2$ konulur.

Koşul sağlanmadığı için de

$$A_3 \Rightarrow A_2 A_3 A_2 \quad \text{yerine} \quad \left\{ \begin{array}{l} A_3 \Rightarrow A_3 A_1 A_3 A_2 \\ A_3 \Rightarrow b A_3 A_2 \end{array} \right\} \text{ konulur.}$$

Bu adımın sonunda oluşan yeniden yazma kuralları aşağıdaki gibidir:

$$A_1 \Rightarrow A_2 A_3$$

$$A_2 \Rightarrow A_3 A_1$$

$$A_2 \Rightarrow b$$

$$A_3 \Rightarrow A_3 A_1 A_3 A_2$$

$$A_3 \Rightarrow b A_3 A_2$$

$$A_3 \Rightarrow a$$

2. Doğrudan özyineli kuralların yok edilmesi.

Birinci adımın sonunda elde edilen kurallardan yalnız birisi özyinelidir. Lemma 4.2 kullanılarak:

$$A_3 \Rightarrow A_3 A_2 A_3 A_2 \quad \text{yerine} \quad \left\{ \begin{array}{l} A_3 \Rightarrow b A_3 A_2 B_3 \\ A_3 \Rightarrow a B_3 \\ B_3 \Rightarrow A_1 A_3 A_2 \\ B_3 \Rightarrow A_1 A_3 A_2 B_3 \end{array} \right\} \text{ konulur.}$$

B_3 : yeni bir değişkendir.

Bu adımın sonunda oluşan yeniden yazma kuralları aşağıdaki gibidir:

$$A_1 \Rightarrow A_2 A_3$$

$$A_2 \Rightarrow A_3 A_1$$

$$A_2 \Rightarrow b$$

$$A_3 \Rightarrow b A_3 A_2 B_3$$

$$A_3 \Rightarrow a B_3$$

$$A_3 \Rightarrow b A_3 A_2$$

$$A_3 \Rightarrow a$$

$$B_3 \Rightarrow A_1 A_3 A_2$$

$$B_3 \Rightarrow A_1 A_3 A_2 B_3$$

3. Tüm kuralların GNF'e uygun biçime getirilmesi.

2. adım sonunda A_3 kurallarının GNF'e uygun biçime geldiği görülür.
3. adımda da Lemma 4.1 kullanılarak A_2 , A_1 ve B_3 kuralları GNF'e uygun biçime dönüştürülür. Bunun için de:

$$\left\{ \begin{array}{l} A_2 \Rightarrow A_3 A_1 \\ A_2 \Rightarrow b \end{array} \right\} \text{ yerine } \left\{ \begin{array}{l} A_2 \Rightarrow b A_3 A_2 A_1 \\ A_2 \Rightarrow a A_1 \\ A_2 \Rightarrow b A_3 A_2 B_3 A_1 \\ A_2 \Rightarrow a B_3 A_1 \\ A_2 \Rightarrow b \end{array} \right\} \text{ konulur.}$$

$$A_1 \Rightarrow A_2 A_3 \text{ yerine } \left\{ \begin{array}{l} A_1 \Rightarrow b A_3 A_2 A_1 A_3 \\ A_1 \Rightarrow a A_1 A_3 \\ A_1 \Rightarrow b A_3 A_2 B_3 A_1 A_3 \\ A_1 \Rightarrow a B_3 A_1 A_3 \\ A_1 \Rightarrow b A_3 \end{array} \right\} \text{ konulur.}$$

$$B_3 \Rightarrow A_1 A_3 A_2 \text{ yerine } \left\{ \begin{array}{l} B_3 \Rightarrow b A_3 A_2 A_1 A_3 A_3 A_2 \\ B_3 \Rightarrow a A_1 A_3 A_3 A_2 \\ B_3 \Rightarrow b A_3 A_2 B_3 A_1 A_3 A_3 A_2 \\ B_3 \Rightarrow a B_3 A_1 A_3 A_3 A_2 \\ B_3 \Rightarrow b A_3 A_3 A_2 \end{array} \right\} \text{ konulur.}$$

$$B_3 \Rightarrow A_1 A_3 A_2 B_3 \text{ yerine } \left\{ \begin{array}{l} B_3 \Rightarrow b A_3 A_2 A_1 A_3 A_3 A_2 B_3 \\ B_3 \Rightarrow a A_1 A_3 A_3 A_2 B_3 \\ B_3 \Rightarrow b A_3 A_2 B_3 A_1 A_3 A_3 A_2 B_3 \\ B_3 \Rightarrow a B_3 A_1 A_3 A_3 A_2 B_3 \\ B_3 \Rightarrow b A_3 A_3 A_2 B_3 \end{array} \right\} \text{ konulur.}$$

Sonuç olarak, $G_{4.9}$ 'a eşdeğer Greibach normal biçimindeki dilbilgisi aşağıdaki gibi oluşur.

$$G'_{4.9} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A_1, A_2, A_3, B_3\} \quad A_1 : \text{Başlangıç değişkeni}$$

$$V_T = \{a, b\}$$

$$P : A_1 \Rightarrow b A_3 A_2 A_1 A_3 \mid a A_1 A_3 \mid b A_3 A_2 B_3 A_1 A_3 \mid a B_3 A_1 A_3 \mid b A_3$$

$$A_2 \Rightarrow b A_3 A_2 A_1 \mid a A_1 \mid b A_3 A_2 B_3 A_1 \mid a B_3 A_1 \mid b$$

$$A_3 \Rightarrow b A_3 A_2 B_3 \mid a B_3 \mid b A_3 A_2 \mid a$$

$$B_3 \Rightarrow b A_3 A_2 A_1 A_3 A_3 A_2 \mid a A_1 A_3 A_3 A_2 \mid b A_3 A_2 B_3 A_1 A_3 A_3 A_2 \mid a B_3 A_1 A_3 A_3 A_2 \mid b A_3 A_3 A_2 \mid b A_3 A_2 A_1 A_3 A_3 A_2 B_3 \mid a A_1 A_3 A_3 A_2 B_3 \mid b A_3 A_2 B_3 A_1 A_3 A_3 A_2 B_3 \mid a B_3 A_1 A_3 A_3 A_2 B_3 \mid b A_3 A_3 A_2 B_3$$

Örnek 4.10.

$$G_{4.10} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S\}$$

$$V_T = \{a, b\}$$

$$P : S \Rightarrow a S b \mid a b$$

$G_{4.10}$ bağlamdan-bağımsız bir dilbilgisi. Önce bu dilbilgisini Chomsky normal biçimine dönüştürelim:

$$P : S \Rightarrow C_a B$$

$$B \Rightarrow S C_b$$

$$S \Rightarrow C_a C_b$$

$$C_a \Rightarrow a$$

$$C_b \Rightarrow b$$

Sonra S 'nin yerine A_1 , C_a 'nın yerine A_2 , C_b 'nin yerine A_3 , B 'nin yerine de A_4 koyarak değişkenleri dizimli değişkenler yapalım.

$$P : A_1 \Rightarrow A_2 A_3$$

$$A_1 \Rightarrow A_2 A_4$$

$$A_2 \Rightarrow a$$

$$A_3 \Rightarrow b$$

$$A_4 \Rightarrow A_1 A_3$$

Sonra $(A_i \Rightarrow A_j, j \geq i)$ koşulunu sağlamayan $(A_4 \Rightarrow A_1 A_3)$ kuralının yerine yeni kurallar koyalım.

$$A_4 \Rightarrow A_1 A_3 \text{ yerine } \left\{ \begin{array}{l} A_4 \Rightarrow A_2 A_3 A_3 \\ A_4 \Rightarrow A_2 A_4 A_3 \end{array} \right\} \text{ yerine } \left\{ \begin{array}{l} A_4 \Rightarrow a A_3 A_3 \\ A_4 \Rightarrow a A_4 A_3 \end{array} \right\}$$

Yeniden yazma kurallarının son durumu:

$$\begin{aligned} P: & A_1 \Rightarrow A_2 A_3 \\ & A_1 \Rightarrow A_2 A_4 \\ & A_2 \Rightarrow a \\ & A_3 \Rightarrow b \\ & A_4 \Rightarrow a A_3 A_3 \\ & A_4 \Rightarrow a A_4 A_3 \end{aligned}$$

Yeniden yazma kurallarının hiçbirisi doğrudan özyineli değildir. Kuralların son dördü de GNF'e uygun biçime gelmiştir. Son olarak A_1 kurallarını GNF'e uygun biçime sokalım. Bunun için

$$\begin{aligned} A_1 \Rightarrow A_2 A_3 \text{ yerine } & A_1 \Rightarrow a A_3 \\ A_1 \Rightarrow A_2 A_4 \text{ yerine de } & A_1 \Rightarrow a A_4 \end{aligned}$$

konulur ve Greibach normal biçimindeki dilbilgisi elde edilir.

$$\begin{aligned} P: & A_1 \Rightarrow a A_3 \mid a A_4 \\ & A_2 \Rightarrow a \\ & A_3 \Rightarrow b \\ & A_4 \Rightarrow a A_3 A_3 \mid a A_4 A_3 \end{aligned}$$

Elde edilen dilbilgisinde A_2 'nin yararsız değişken, $(A_2 \Rightarrow a)$ 'nın da yararsız kural olduğu görülmektedir. Yararsız değişken ve kuralı atıp, kalan değişkenleri S, B ve C olarak adlandırarak, $G_{4.10}$ 'a denk, Greibach normal biçimindeki aşağıdaki dilbilgisi bulunur.

$$\begin{aligned} G'_{4.10} = & \langle V_N, V_T, P, S \rangle \\ V_N = & \{S, B, C\} \\ V_T = & \{a, b\} \\ P: & S \Rightarrow aB \mid aC \\ & B \Rightarrow b \\ & C \Rightarrow aBB \mid aCB \end{aligned}$$

Sorular

S.4.1. Aşağıda tanımlanan dillerin her birini türeten bir dilbilgisi tanımlayınız.

- $L_{S.4.1.1} = \{0^n 1^m 2^k \mid n \geq 1, m \geq 1, k \geq 1, (m = n) \text{ ya da } (m = k)\}$
- $L_{S.4.1.2} : \{a, b\}$ alfabesinde, "içerdiği b 'lerin sayısı a 'ların sayısından daha büyük ya da eşit olan" dizgiler kümesi.
(Not: bu dil λ 'yı içermeyecek).
- $L_{S.4.1.3} : \{a, b, c, d\}$ alfabesinde, "içindeki her a 'dan hemen sonra en az bir b ; her c 'den hemen sonra da en az iki d bulunan" dizgiler kümesi.
(Not: bu dil λ 'yı içermeyecek).

S.4.2. Bağlamdan-bağımsız $G_{S.4.2}$ dilbilgisi aşağıdaki gibi tanımlanıyor.

$$\begin{aligned} G_{S.4.2} = & \langle V_N, V_T, P, S \rangle \\ V_N = & \{S, D, E\} \\ V_T = & \{a, b, c\} \\ P: & S \Rightarrow aaS \mid aaD \\ & D \Rightarrow ccD \mid bEb \\ & E \Rightarrow bEb \mid c \end{aligned}$$

- $G_{S.4.2}$ tarafından tanımlanan bağlamdan-bağımsız dilin $(L_{S.4.2})$ küme tanımını oluşturunuz.
- $G_{S.4.2}$ 'yi Chomsky normal biçimine (CNF) dönüştürünüz. Dönüştürme sırasında kullanacağınız yeni değişkenleri A, B, C , ve $X_1, X_2, X_3, \dots$ diye adlandırınız. Oluşturduğunuz CNF dilbilgisinin tanımını eksiksiz ve düzenli biçimde yazınız.

S.4.3. Aşağıdaki her dilbilgisi için:

- Dilbilgisinin türü nedir? Nedeniyle birlikte belirtiniz.
- Dilbilgisinin türettiği dili $(L(G))$ bulunuz ve matematiksel bir ifadeyle gösteriniz.
- Eğer mümkünse, $L(G)$ 'yi türeten bağlamdan-bağımsız bir dilbilgisi tanımlayınız.

$$\begin{aligned} \text{a) } P: S &\Rightarrow aAbc \mid abc \\ A &\Rightarrow aAbC \mid aBc \\ Cb &\Rightarrow bC \\ Cc &\Rightarrow cc \end{aligned}$$

$$\begin{aligned} \text{b) } P: S &\Rightarrow SBA \mid a \\ BA &\Rightarrow AB \\ aA &\Rightarrow aaB \\ B &\Rightarrow b \end{aligned}$$

$$\text{S.4.4. } G_{S.4.4} = \langle V_N, V_T, P, S \rangle$$

$$\begin{aligned} V_N &= \{S, C\} \\ V_T &= \{a, b, d, e\} \\ P: S &\Rightarrow abC \mid babS \mid de \\ C &\Rightarrow aCa \mid b \end{aligned}$$

$G_{S.4.4}$ 'ü Chomsky normal biçimine (CNF) dönüştürünüz. Dönüştürme işlemlerinde kullanacağınız yeni değişkenleri A, B, D, E ile $T, U, V, W, ..$ olarak adlandırınız.

S.4.5. Aşağıda tanımlanan bağlamdan-bağımsız dillerin her birini türeten bağlamdan-bağımsız bir dilbilgisi tanımlayınız.

$$\begin{aligned} \text{a) } L_{S.4.5.1} &= \{a^{2k} b^n c^{2n} d^k \mid k \geq 1, n \geq 1\} \\ \text{b) } L_{S.4.5.2} &= \{a^{n+m} b^k c^{m+k} d^{2n} \mid n, m, k \geq 0\} \end{aligned}$$

S.4.6. Aşağıdaki bağlamdan-bağımsız her dilbilgisinin türettiği dilin küme tanımını veriniz.

$$\begin{aligned} \text{a) } G_{S.4.6.1} &= \langle V_N, V_T, P, S \rangle \\ V_N &= \{S, A, B, C\} \\ V_T &= \{a, b\} \\ P: S &\Rightarrow aS \mid aA \\ A &\Rightarrow aA \mid bB \\ B &\Rightarrow aB \mid bC \\ C &\Rightarrow aC \mid a \end{aligned}$$

$$\begin{aligned} \text{b) } G_{S.4.6.2} &= \langle V_N, V_T, P, S \rangle \\ V_N &= \{S, X, Y\} \\ V_T &= \{a, b, c, d, e\} \\ P: S &\Rightarrow aSe \mid XY \\ X &\Rightarrow aXc \mid b \\ Y &\Rightarrow cYe \mid d \end{aligned}$$

S.4.7. Aşağıda tanımlanan bağlamdan-bağımsız dillerin her birini türeten bağlamdan-bağımsız bir dilbilgisi tanımlayınız.

$$\begin{aligned} \text{a) } L_{S.4.7.1} &= \{a^n b^m c^k \mid m > 0, k > 0, n > m + k\} \\ \text{b) } L_{S.4.7.2} &= \{a^n b^m c^k \mid n > 0, m > 0, k = 2n + m\} \end{aligned}$$

S.4.8. Aşağıdaki bağlamdan-bağımsız her dilbilgisinin türettiği dilin küme tanımını veriniz.

$$\begin{aligned} \text{a) } G_{S.4.8.1} &= \langle V_N, V_T, P, S \rangle \\ V_N &= \{S, B, C\} \\ V_T &= \{a, b, d\} \\ P: S &\Rightarrow aSaa \mid B \\ B &\Rightarrow bbBdd \mid C \\ C &\Rightarrow bd \end{aligned}$$

$$\begin{aligned} \text{b) } G_{S.4.8.2} &= \langle V_N, V_T, P, S \rangle \\ V_N &= \{S, A\} \\ V_T &= \{a, b, c, d\} \\ P: S &\Rightarrow abSdc \mid A \\ A &\Rightarrow cdAba \mid \lambda \end{aligned}$$

- S.4.9. Aşağıdaki bağlamdan-bağımsız dilbilgisindeki yararsız uç simge, değişken ve kuralları bulunuz. Dilbilgisini yararsız simge, değişken ve kurallardan arındırarak denk bir dilbilgisi oluşturunuz. $G_{S.4.9}$ tarafından türetilen dili bir düzgün deyimle gösteriniz.

$$G_{S.4.9} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B, C, D, E, F, G\}$$

$$V_T = \{a, b, c, d, e\}$$

$$P: S \Rightarrow dA \mid BD$$

$$A \Rightarrow dA \mid dAB \mid dD$$

$$B \Rightarrow eB \mid cC \mid BF$$

$$C \Rightarrow Bc \mid dAC \mid E$$

$$D \Rightarrow aD \mid aF \mid a$$

$$E \Rightarrow dB \mid aC$$

$$F \Rightarrow dF \mid dG \mid b$$

$$G \Rightarrow eC \mid aE$$

- S.4.10. Aşağıda tanımlanan bağlamdan-bağımsız dillerin her birini türeten bağlamdan-bağımsız bir dilbilgisi tanımlayınız.

$$a) L_{S.4.10.1} = \{a^n b^k c^k d^n \mid n \geq 1, k \geq 1\}$$

$$b) L_{S.4.10.2} = \{a^{n+k} b^n c^k \mid n \geq 1, k \geq 0\}$$

$$c) L_{S.4.10.3} = \{b^n c^k \mid 1 \leq k \leq n \leq 2k\}$$

$$d) L_{S.4.10.4} = \{a^i b^n c^n d^j \mid n \geq 1, i \geq 0, j \geq 0\}$$

- S.4.11. $G_{S.4.11} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S\}$$

$$V_T = \{\sim, \supset, [,], p, q\}$$

$$P: S \Rightarrow \sim S \mid [S \supset S] \mid p \mid q$$

$G_{S.4.11}$ 'i Chomsky normal biçimine (CNF) dönüştürünüz. CNF dilbilgisini tanımladıktan sonra, aşağıdaki tümce için türetme ağacını oluşturunuz.

$$w = \sim[\sim p \supset [q \supset \sim p]]$$

- S.4.12. $G_{S.4.12} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, T, U, V\}$$

$$V_T = \{a, b, c\}$$

$$P: S \Rightarrow VU \mid c$$

$$T \Rightarrow a$$

$$U \Rightarrow b$$

$$V \Rightarrow TS$$

- Yukarıda tanımlanan dilbilgisinin türü nedir? Dilbilgisi belirli bir normal biçimde midir?
- Dilbilgisinin türettiği dilin küme tanımını veriniz.
- Dilbilgisinin türettiği 5 uzunluğunda bir tümce bulunuz ve bu tümce için türetme ağacını oluşturunuz.

- S.4.13. Aşağıda tanımlanan dillerin her birinin türünü belirtiniz. Eğer dil düzgün ya da bağlamdan-bağımsız bir dil ise, dili türeten bir dilbilgisi tanımlayınız.

$$a) L_{S.4.13.1} = \{a^n b^n \mid n \geq 1\}$$

$$b) L_{S.4.13.2} = \{a^n b^m \mid n \geq 1, m \geq 1\}$$

$$c) L_{S.4.13.3} = \{\alpha \alpha^R \beta \beta^R \mid \alpha, \beta \in (0+1)^*\}$$

$$d) L_{S.4.13.4} = \{\alpha \beta \beta^R \alpha^R \mid \alpha, \beta \in (0+1)^*\}$$

- S.4.14. $G_{S.4.14} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, A\}$$

$$V_T = \{0, 1\}$$

$$P: S \Rightarrow AA \mid 0$$

$$A \Rightarrow AAS \mid 0S \mid 1$$

$G_{S.4.14}$ 'ü Greibach normal biçimine (GNF) dönüştürünüz.

S.4.15. $L_{S.4.15} = \{a^m b^m c^n \mid m \geq 1, n \geq 1\}$

Yukarıda küme tanımı verilen bağlamdan-bağımsız dili türeten bir dilbilgisi oluşturunuz.

S.4.16. $G_{S.4.16} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, T, U, V, W, A, B\}$$

$$V_T = \{a, b, c\}$$

$$P : S \Rightarrow TU$$

$$T \Rightarrow VB \mid c$$

$$U \Rightarrow WA \mid c$$

$$V \Rightarrow AT$$

$$W \Rightarrow BU$$

$$A \Rightarrow a$$

$$B \Rightarrow b$$

- Yukarıda tanımlanan dilbilgisinin türü ve biçimi nedir?
- Dilbilgisinin türettiği dilin küme tanımını veriniz.
- Dilbilgisinin türettiği en az 5 uzunluğunda bir tümce bulup bu tümce için türetme ağacını oluşturunuz.

S.4.17.

- Bağlamdan-bağımsız $G_{S.4.17.1}$ dilbilgisi aşağıdaki gibi tanımlanıyor.

$$G_{S.4.17.1} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B, C\}$$

$$V_T = \{a, b, c\}$$

$$P : S \Rightarrow AC \mid CA \mid B$$

$$A \Rightarrow a$$

$$B \Rightarrow BC \mid ABc$$

$$C \Rightarrow aB \mid b$$

$G_{S.4.17.1}$ 'deki yararsız değişken, kural ve uç simgeleri bulunuz ve dilbilgisini yalınlaştırınız.

b) $G_{S.4.17.2} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, A, B, C\}$$

$$V_T = \{a, b, c\}$$

$$P : S \Rightarrow A \mid aB$$

$$A \Rightarrow BC \mid a$$

$$B \Rightarrow Bb \mid C$$

$$C \Rightarrow Cc \mid c$$

$G_{S.4.17.2}$ dilbilgisinin türü nedir? Dilbilgisi hangi normal biçimdedir?

$G_{S.4.17.2}$ 'nin türettiği dil bir düzgün dildir. Bu dili tanımlayan bir düzgün deyim oluşturunuz.

- S.4.18. $L_{S.4.18}$ dili aşağıdaki gibi tanımlanıyor:

$$L_{S.4.18} = \{a^k b^m a^n c^p a^q \mid k, m, p, k \geq 1, n \geq k+q\}$$

- $L_{S.4.18}$ dilini türeten bağlamdan-bağımsız bir dilbilgisi oluşturunuz.
- Oluşturduğunuz dilbilgisi ile sol öncelikli olarak 8-10 uzunluğunda bir tümce türetiniz ve bu tümce için türetme ağacını oluşturunuz.

- S.4.19. $\{a, b, c, d, e\}$ alfabesi üzerinde tanımlı bağlamdan-bağımsız $G_{S.4.19}$ dilbilgisinin yeniden yazma kuralları aşağıdaki gibidir:

$$S \Rightarrow E \mid SaS \mid bba \mid Ge$$

$$A \Rightarrow AbC \mid aSE \mid bCa \mid aGC \mid bc$$

$$B \Rightarrow aBS \mid D \mid bBH \mid cd$$

$$C \Rightarrow aSA \mid bCb \mid A \mid bbH \mid ac$$

$$D \Rightarrow H \mid aAH \mid BaS \mid ab$$

$$E \Rightarrow cE \mid aCA \mid bbS$$

$$G \Rightarrow GAS \mid aaG \mid cHB$$

$$H \Rightarrow AHc \mid AcG \mid aGS$$

Bu dilbilgisini indirgeyerek yararsız simge, değişken ve kural ile birim türetme kuralı içermeyen bir dilbilgisi elde etmeniz isteniyor. Bunun için:

- Nasıl yaptığınızı göstererek, önce, varsa yararsız değişken kural ve simgeleri yok ediniz.
- Sonra da birim türetme kuralını yok ediniz.

S.4.20. $L_{S.4.20}$ dili aşağıdaki gibi tanımlanıyor:

$$L_{S.4.20} = \{a^k (bb)^k (cc)^n d^n \mid k \geq 1, n \geq 1\}$$

- Yalnız 3 sözdizim değişkeni (S, X, ve Y) kullanarak, $L_{S.4.20}$ dilini türeten bağlamdan-bağımsız bir dilbilgisi oluşturunuz.
- Oluşturduğunuz dilbilgisi ile sol öncelikli olarak 9 uzunluğunda bir tümce türetiniz ve bu tümce için türetme ağacını oluşturunuz.

S.4.21. $G_{S.4.21}$ dilbilgisi aşağıdaki gibi tanımlanıyor:

$$G_{S.4.21} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B, C\}$$

$$V_T = \{0, 1, 2\}$$

$$P : S \Rightarrow A \mid C$$

$$A \Rightarrow A0B \mid A0C \mid B \mid 0$$

$$B \Rightarrow B1 \mid C1$$

$$C \Rightarrow 2C \mid 2$$

- $G_{S.4.21}$ dilbilgisindeki birim türetme kurallarını yok ederek, birim türetme kuralı içermeyen eşdeğer bir dilbilgisi ($G'_{S.4.21}$) oluşturunuz.
- $G'_{S.4.21}$ dilbilgisini Chomsky normal biçime dönüştürünüz ($G''_{S.4.21}$). Bunun için dilbilgisinin Chomsky normal biçime uygun olan kurallarını koruyunuz. Chomsky normal biçime uygun olmayan kurallarını ise en az sayıda yeni değişken kullanarak ve yeni değişkenleri D, E, diye (dizinsiz değişkenler) adlandırarak Chomsky normal biçime uygun hale getiriniz. Sonunda her değişkene ilişkin kuralları bir satırda toplayarak tüm yeniden yazma kurallarını ($S \Rightarrow \dots \mid \dots \mid \dots$) biçimde yazınız.

S.4.22. $G_{S.4.22} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, A, B, C\}$$

$$V_T = \{a, b\}$$

$$P : S \Rightarrow bC \mid aBB$$

$$C \Rightarrow bA \mid aB$$

$$B \Rightarrow bS \mid aBBB \mid b$$

$$A \Rightarrow aS \mid bCA \mid a$$

- Yukarıda tanımı verilen dilbilgisinin türü nedir? Dilbilgisi belirli bir normal biçimde midir? Bu dilbilgisi ile, soldan türetmeyi kullanarak, farklı uzunluklarda 3 tümce türetiniz.
- Bu dilbilgisi tarafından türetilen dilin ($L_{S.4.22}$) sözlü tanımını veriniz.

S.4.23. Bağlamdan-bağımsız $G_{S.4.23}$ dilbilgisi aşağıdaki gibi tanımlanıyor.

$$G_{S.4.23} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B, C, D, E, F, G\}$$

$$V_T = \{a, b, c, d, e\}$$

$$P : S \Rightarrow aC \mid aBa \mid bAb \mid eCe \mid aSbb \mid aDbb$$

$$A \Rightarrow aA \mid dB \mid eC$$

$$B \Rightarrow eB \mid bcB \mid aCD$$

$$C \Rightarrow aB \mid bB \mid cC$$

$$D \Rightarrow cA \mid cDc \mid cFc \mid aDA$$

$$E \Rightarrow cF \mid dG \mid a \mid b \mid c$$

$$F \Rightarrow dFd \mid dB \mid d$$

$$G \Rightarrow aE \mid eG$$

- $G_{S.4.23}$ deki yararsız değişken, kural ve uç simgeleri bulunuz ve dilbilgisini yalınlaştırınız..
- Elde ettiğiniz yalın dilbilgisini inceleyerek, bu dilbilgisi tarafından türetilen bağlamdan-bağımsız dilin ($L_{S.4.23}$) biçimsel tanımını (küme tanımı) oluşturunuz.

S.4.24. $G_{S.4.24}$ dilbilgisi aşağıdaki gibi tanımlanıyor:

$$G_{S.4.24} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B, C\}$$

$$V_T = \{a, b, c\}$$

$$P : S \Rightarrow A \mid C$$

$$A \Rightarrow AaB \mid AaC \mid B \mid a$$

$$B \Rightarrow Bb \mid Cb$$

$$C \Rightarrow cC \mid c$$

- a) $G_{S.4.24}$ deki yeniden yazma kurallarından doğrudan özyineli (*direct recursive*) olanları yok ederek eşdeğer ($G'_{S.4.24}$) dilbilgisini bulunuz (yeni değişkenleri D, E, .. diye adlandırınız).
- b) Elde ettiğiniz ($G'_{S.4.24}$) dilbilgisinin birim türetme kurallarını yok ederek eşdeğer ($G''_{S.4.24}$) dilbilgisini bulunuz.

Not: Gerekli ara adımları gösterdikten sonra ($G'_{S.4.24}$) ve ($G''_{S.4.24}$) nün yeniden yazma kurallarını her değişkene ilişkin kuralı bir satırda toplayarak yazınız.

S.4.25. $w_1, w_2, \dots \{0, 1, 2\}$ alfabesinde boş olmayan (en az bir uzunluğunda) dizgiler olmak üzere $L_{S.4.25}$ dili aşağıdaki gibi tanımlanıyor.

$$L_{S.4.25} = (w_1 w_1^R) [(w_2 w_2^R)] [(w_3 w_3^R)] \dots$$

- a) $L_{S.4.25}$ 'i türeten ve yok edilebilir değişkeni bulunmayan bağlamdan bağımsız bir dilbilgisi tanımlayınız.
- b) Tanımladığınız dilbilgisi ile $w = (w_1 w_1^R) (w_2 w_2^R)$ biçiminde, en az 10 uzuluğunda bir tümceyi sol öncelikli olarak türetiniz (*leftmost derivation*) ve bu tümce için türetme ağacını oluşturunuz.

S.4.26. $G_{S.4.26} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b\}$$

$$P : S \Rightarrow 111A \mid 00B$$

$$A \Rightarrow 00A \mid \lambda$$

$$B \Rightarrow 1B \mid \lambda$$

- a) Yukarıda tanımlı verilen sağ-doğrusal dilbilgisi tarafından türetilen dili ($L_{S.4.26}$) tanıyan NFA'nın geçiş çizeneğini oluşturunuz

Not: Çizeneği sistematik yöntemle oluşturabileceğiniz gibi sezgisel olarak da oluşturabilirsiniz. Ancak sezgisel olarak yalın bir çizenek oluştursanız, sorunun devamı çok kolaylaşır.

- b) Oluşturduğunuz geçiş çizeneği λ -geçişleri içeriyorsa, λ -geçiş içermeyen eşdeğer geçiş çizeneğini bulunuz.
- c) Çizenekte yararsız durumlar varsa, bunları yok ederek çizeneği yalınlaştırınız.
- d) Birden çok başlangıç durumu varsa, tek başlangıç durumlu eşdeğer çizeneği bulunuz.
- e) Denklem sistemi yazıp çözmeden, $L_{S.4.26}$ 'yı tanımlayan bir düzgün deyim yazınız.

S.4.27. $G_{S.4.27}$ dilbilgisi aşağıdaki gibi tanımlanıyor:

$$G_{S.4.27} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B, C, D\}$$

$$V_T = \{a, b, c, d, e\}$$

$$P : S \Rightarrow aaS \mid A \mid C \mid BD$$

$$A \Rightarrow bbA \mid bb \mid cB$$

$$B \Rightarrow cB \mid \lambda$$

$$C \Rightarrow ddC \mid dd \mid eD$$

$$D \Rightarrow eD \mid \lambda$$

- a) $G_{S.4.27}$ 'ye denk, başlangıç değişkeni dışında yok edilebilir değişkeni bulunmayan eşdeğer dilbilgisini ($G'_{S.4.27}$) bulunuz.
- b) Elde ettiğiniz ($G'_{S.4.27}$) dilbilgisinin birim türetme kurallarını yok ederek eşdeğer ($G''_{S.4.27}$) dilbilgisini bulunuz.

Not: Gerekli ara adımları gösterdikten sonra ($G'_{S.4.27}$) ve ($G''_{S.4.27}$) dilbilgilerinin yeniden yazma kurallarını her değişkene ilişkin kuralı bir satırda toplayarak yazınız.

S.4.28. $L_{S.4.28} = a(a^*b^* + c^*d^*)$

- Yukarıdaki düzgün deyimle tanımlanan dili türeten NFA'nın geçiş çizeneğini durumları S, A, B, C, ... diye adlandırarak ve olabildiğince az durum kullanarak oluşturunuz. Oluşturacağınız geçiş çizeneğinde tek başlangıç durumu (S) bulunsun.
- Oluşturduğunuz geçiş çizeneği λ -geçişleri içeriyorsa, bu geçişleri uygun sırada yok ederek λ -geçişsiz eşdeğer NFA'nın geçiş çizeneğini bulunuz.
- $L_{S.4.28}$ dilini türeten ve başlangıç değişkeni dışında yok edilebilir değişkeni bulunmayan bir tür-3 dilbilgisi oluşturunuz (gerekliyse başlangıç değişkeni yok edilebilir olabilir).

S.4.29. $G_{S.4.29} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b, c, d\}$$

$$P : S \Rightarrow aSa \mid bSb \mid aABa \mid bABb$$

$$A \Rightarrow cAc \mid cc$$

$$B \Rightarrow dB \mid \lambda$$

- $G_{S.4.29}$ tarafından türetilen dilin biçimsel tanımını $L_{S.4.29} = \{\dots\}$ biçiminde oluşturunuz
- $G_{S.4.29}$ ile sol öncelikli olarak eşit sayıda a, b, c, ve d içeren 8 uzunluğunda bir dizgi türetiniz ve bu dizgiye karşı gelen türetme ağacını oluşturunuz.

S.4.30. $L_{S.4.30}$ dili {a, b, c, d} alfabesinde, içindeki a'lar ile c'lerin toplamı b'ler ile d'lerin toplamına eşit olan dizgiler kümesi olarak tanımlanıyor. $L_{S.4.30}$ içinde λ yer almıyor.

- $L_{S.4.30}$ dilini türeten bir dilbilgisi tanımlayınız.
- Tanımladığınız dilbilgisi ile sol öncelikli olarak $w = acdcbbab$ tümcesini türetiniz ve bu tümceye karşı gelen türetme ağacını oluşturunuz.

Bölüm 5

Yığıtlı Özdevinirler (Pushdown Automata)

Düzdün diller, düzdün ya da tür-3 dilbilgileri tarafından türetilen ve sonlu özdevinirler tarafından tanınan dillerdir. Düzdün dil, düzdün dilbilgisi ve sonlu özdevinirler birbiriyle ilişkili bir dil-dilbilgisi-makine üçlüsü oluşturur. Benzer biçimde, bu bölümün konusu olan yığıtlı özdevinirler ile bağlamdan-bağımsız dilbilgisi ve diller birbiriyle ilişkili bir dil-dilbilgisi-makine üçlüsü oluşturur. Yığıtlı özdevinirler, bağlamdan-bağımsız dilleri (CFL) tanıyan makine modelidir. Bağlamdan-bağımsız diller ise, bağlamdan-bağımsız dilbilgileri (CFG) tarafından türetilen dillerdir. Yığıtlı özdevinirlerin İngilizce karşılığı "Pushdown Automata (PDA)" dır. Modelin Türkçe adlandırılmasında, modelin yığıt içeren bir model olması ve pushdown automata'nın sözcük sözcük karşılığının çok anlamlı olmaması nedeniyle, "yığıtlı özdevinir" terimi tercih edilmiştir. Kısa ad olarak ise PDA kullanılacaktır.

5.1. Yığıtlı Özdevinirin Temel Modeli

Biçimsel olarak, yığıtlı özdevinir (PDA) bir yedili olarak tanımlanabilir.

$$PDA = \langle Q, \Sigma, \Gamma, \delta, q_0, Z_0, F \rangle$$

- Q : Sonlu sayıda durum içeren Durumlar Kümesi
- Σ : Sonlu sayıda giriş simgesinden oluşan Giriş Alfabeti
- Γ : Sonlu sayıda simge içeren Yığıt Alfabeti. Yığıt ve giriş alfabelerindeki simgelerin tümü ya da bir bölümü ortak olabileceği gibi iki alfabede hiç ortak simge bulunmayabilir de.
- q_0 : Başlangıç durumu ($q_0 \in Q$)
Başlangıç durumu durumlar kümesinin bir elemanı olduğuna göre Q boş olmayan bir kümedir.

Z_0 : Yığıt alfabesindeki simgelerden, Yığıt Başlangıç Simgesi olarak adlandırılan özel bir simge ($Z_0 \in \Gamma$). Z_0 , giriş alfabesinde yer almayan bir simgedir. Yığıt alfabesinin diğer tüm simgeleri ise giriş alfabesinde de yer alabilir.

F : Uç durumlar kümesi
Durumlar kümesinin bir altkümesidir: $F \subseteq Q$

δ : Geçiş ya da hareket işlevi (*transition or move function*)
Deterministik PDA modelinde geçiş işlevi,

$$[Q \times (\Sigma \cup \{\lambda\}) \times \Gamma]^* \text{ dan } [Q \times \Gamma]^* \text{ a}$$

bir eşleme oluşturur. Deterministik olmayan modelde ise geçiş işlevi,

$$[Q \times (\Sigma \cup \{\lambda\}) \times \Gamma]^* \text{ dan } [Q \times \Gamma]^* \text{ 'in sonlu altkümelerine}$$

bir eşleme oluşturur.

PDA'nın Soyut Makine Modeli

Tanımlandığı biçimiyle yığıtlı özdevinir (PDA) bir matematiksel modeldir. PDA'yı soyut bir makine olarak düşünmek de mümkündür. PDA'nın nasıl çalıştığını daha iyi anlayabilmek için, bilinen bileşenlerden oluşan bir makine modeli de kullanılmaktadır. Bu model aşağıdaki bileşenlerden oluşmaktadır (Çizim 5.1):

- Hücrelerden oluşan ve her hücresinde bir giriş simgesi bulunan bir miknatıslı şerit. Yalnız okunabilen şeridin sağ ucu sonsuzdur.
- Bir sonlu denetim birimi (SDB)
- Bir okuma kafası
- Bir yığıt

PDA'nın Çizim 5.1'de görülen soyut makine modeli aşağıda açıklanan biçimde çalışır.

- Başlangıçta şerit üzerinde giriş simgelerinden oluşan bir dizgi (w) kayıtlıdır ve okuma kafası ilk (en soldaki) hücre üzerindedir.
- Yığıtın her elemanına bir yığıt simgesi yazılabilmektedir ve başlangıçta yığıtta Z_0 başlangıç simgesi kayıtlıdır.
- Makinenin 2 türlü hareketi vardır. Bu hareketler normal hareket ve λ -hareketi (λ -move) olarak adlandırılır.

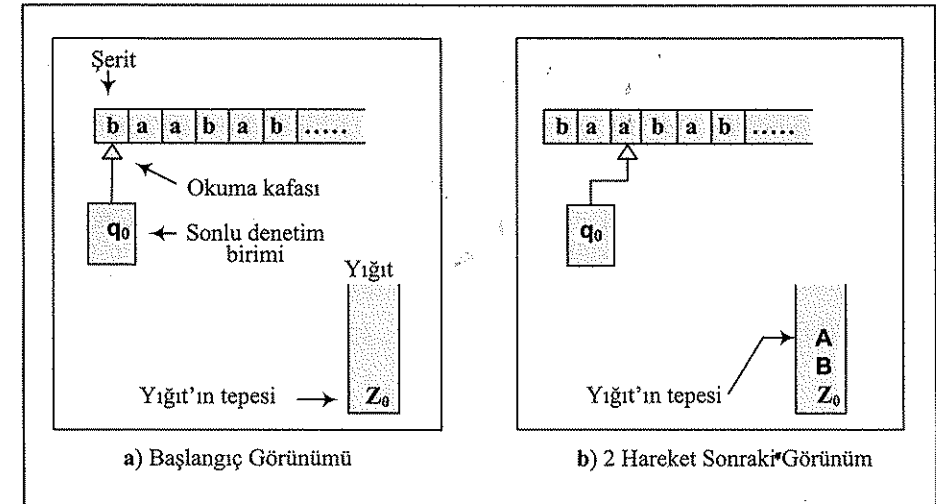
Normal hareketlerde aşağıdaki işlemler yapılır:

- Şeritten bir simge okunur.
- Okuma kafası bir sağdaki hücreye geçer.
- Yığıtın en üstündeki simge silinir (*pop*).

- Yığıtın üstüne, yığıt simgelerinden oluşan bir dizgi ($\alpha \in \Gamma^*$) eklenir (*push*). Yığıta eklenen dizgi boş (λ) da olabilir. Eğer yığıta eklenen dizgi birden çok yığıt simgesinden oluşan bir dizgi ise ($\alpha = X_1 X_2 \dots X_n$, $X_1, X_2, \dots, X_n \in \Gamma$), bu simgelerin, en soldaki (X_1) simge en üste gelecek biçimde yığıta ekleneceği varsayılacaktır.

λ -hareketinde ise aşağıdaki işlemler yapılır:

- Şeritten bir simge okunmaz ve okuma kafası yer değiştirmez. Dolayısıyla λ -hareketi şeritteki simgeden bağımsız bir harekettir.
- Yığıtın en üstündeki simge silinir (*pop*).
- Yığıtın üstüne, yığıt simgelerinden oluşan bir dizgi ($\alpha \in \Gamma^*$) eklenir (*push*). Yığıta eklenen dizgi boş (λ) da olabilir.



Çizim 5.1. PDA'nın Soyut Makine Modeli

Anlık Tanımlar (*Instantaneous Descriptions*)

PDA'nın belirli bir andan sonraki davranışlarını kestirebilmek için o ana ilişkin üç bilginin bilinmesi gerekir:

- PDA'nın (sonlu denetim biriminin) bulunduğu durum
- Giriş dizgisinin henüz işlenmemiş (okuma kafasının üzerinde bulunduğu hücre ile bu hücrenin sağındaki hücrelerde bulunan) kesimi
- Yığıtın içeriği

Bu üç bilginin belirli bir andaki değerlerinden oluşan üçlüye, PDA'nın anlık tanımı denir:

Anlık tanım (ID) = (p, v, X)

p : PDA'nın durumu

v : giriş dizgisinin henüz işlenmemiş kesimi

X : yığıtın içeriği

PDA'nın Tanıdığı Dil

Her PDA, giriş alfabesindeki simgelerden oluşan dizgilerin bir kesimini tanıır (kabul eder), bir kesimini ise tanımaz. PDA'nın tanıdığı dizgiler kümesi, PDA'nın tanıdığı dildir. Dizgileri tanıma açısından PDA'nın iki alt modeli vardır. Bu modeller:

- Uç durumla tanıyan PDA
- Boş yığıtla tanıyan PDA

olarak adlandırılır. Uç durumla tanıyan PDA modelinde, FA modelinde olduğu gibi, eğer bir dizginin tüm simgeleri okunduktan sonra PDA bir uç durumda bulunuyorsa, bu dizgi PDA tarafından tanınır. Uç durumla tanıyan PDA modelinde, dizginin tüm simgeleri okunduktan sonra, yığıtın içeriğine bakılmaz; dizginin PDA tarafından tanınmasının tek koşulu, son durumun bir uç durum olmasıdır. Anlık tanımları (*instantaneous descriptions*) kullanarak, uç durumla tanıyan bir PDA'nın tanıdığı dizgiler kümesi aşağıdaki gibi tanımlanır:

$$T(M) = \{w \mid w \in V_T^*, (q_0, w, Z_0) \xrightarrow{*} (p, \lambda, \gamma), p \in F\}$$

Boş yığıtla tanıyan PDA modelinde ise, dizginin tüm simgeleri okunduktan sonra, eğer yığıt boşalmışsa dizgi PDA tarafından tanınır. Bu modelde, PDA'nın son durumu tanımada etkili olmaz. Anlık tanımları kullanarak, boş yığıtla tanıyan bir PDA'nın tanıdığı dizgiler kümesi aşağıdaki gibi tanımlanır:

$$T(M) = \{w \mid w \in V_T^*, (q_0, w, Z_0) \xrightarrow{*} (p, \lambda, \lambda), p \in Q\}$$

Örnek 5.1. $L_{5.1}$ dili aşağıdaki gibi tanımlanıyor:

$$L_{5.1} = \{w c w^R \mid w \in (0+1)^*\}$$

Bu dili türeten bir dilbilgisi aşağıdaki gibi tanımlanabilir:

$$G_{5.1} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S\}$$

$$V_T = \{0, 1, c\}$$

$$P: S \Rightarrow 0S0 \mid 1S1 \mid c$$

$L_{5.1}$ 'i tanıyan PDA'yı tanımlayalım:

$$M_{5.1} = \langle Q, \Sigma, \Gamma, \delta, q_0, Z_0, \Phi \rangle$$

$$Q = \{q_0, q_1\}$$

$$\Sigma = \{0, 1, c\}$$

$$\Gamma = \{0, 1, Z_0\}$$

PDA'nın çalışma ilkesi. PDA'nın tanıdığı dizgiler, $\{0, 1\}$ alfabesindeki herhangi bir w altdizgisi ile bunun tersinden oluşmaktadır. w ile w^R arasında da "c" simgesi yer almaktadır. Bu dizgileri tanıyan PDA:

- Önce w 'nin simgelerini okur ve okuduğu her simgesyi yığıtı ekler. Ekleme sırasında PDA q_0 durumunda bulunur.
- c 'yi okuduğunda, w 'nin bittiğini anlar, yığıtta bir değişiklik yapmaz ve durum değiştirerek q_1 durumuna geçer.
- Sonra (w^R) 'nin simgelerini okur, ve okuduğu her simge için eğer yığıtın tepesinde aynı simge varsa bunu yığıttan çıkarır (siler).
- Giriş dizgisinin tüm simgeleri okunduktan sonra, eğer yığıtın tepesinde Z_0 varsa, bunu da siler ve yığıtı tamamen boşaltır.

Yukarıda sözlü olarak tanımlanan işlemleri yapan PDA'nın hareket işlevi biçimsel olarak aşağıdaki gibi tanımlanır:

$$\delta : \delta(q_0, 0, Z_0) = (q_0, 0Z_0)$$

$$\delta(q_0, 1, Z_0) = (q_0, 1Z_0)$$

$$\delta(q_0, c, Z_0) = (q_1, Z_0)$$

$$\delta(q_0, 0, 0) = (q_0, 00)$$

$$\delta(q_0, 1, 0) = (q_0, 10)$$

$$\delta(q_0, 0, 1) = (q_0, 01)$$

$$\delta(q_0, 1, 1) = (q_0, 11)$$

$$\delta(q_0, c, 0) = (q_1, 0)$$

$$\delta(q_0, c, 1) = (q_1, 1)$$

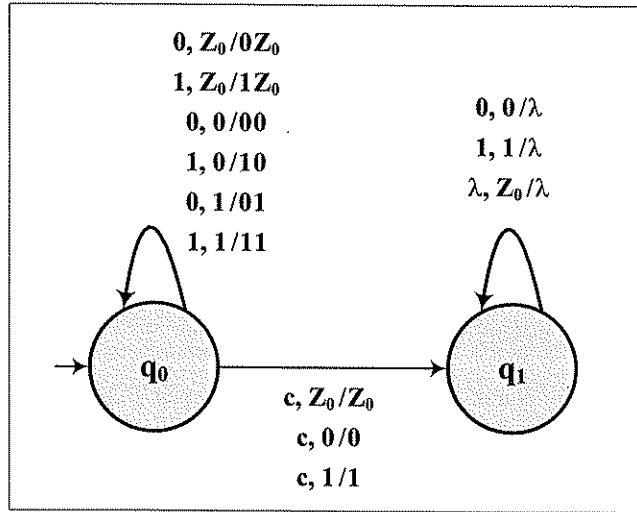
$$\delta(q_1, 0, 0) = (q_1, \lambda)$$

$$\delta(q_1, 1, 1) = (q_1, \lambda)$$

$$\delta(q_1, \lambda, Z_0) = (q_1, \lambda)$$

Örnek olarak verilen $M_{5.1}$ deterministik bir PDA'dır.

PDA'nın hareket işlevi yukarıdaki biçimde tanımlanabileceği gibi, bir çizenek ile de tanımlanabilir (Çizim 5.2). Çizenekte, her hareket iki durum arasındaki bir yay ile gösterilmekte ve yayın üzerine bir üçlü yazılmaktadır. Üçlünün birinci elemanı şeritten okunan giriş simgesini, ikinci elemanı yığıtın tepesinde bulunan ve silinen yığıt simgesini, üçüncü elemanı ise yığıtı eklenen yığıt simgeleri dizgisini göstermektedir. Bu üçlü İngilizce sözcüklerle kısaca "*read, pop, push*" üçlüsü olarak gösterilebilir. PDA'nın temel modeline göre (Bkz. 5.1) bu üçlünün birinci ve sonuncu elemanı λ olabilir.



Çizim 5.2. $L_{5,1}$ Dilini Tanıyan PDA'nın ($M_{5,1}$) Geçiş Çizeneği

Örnek 5.2. $L_{5,2}$ dili aşağıdaki gibi tanımlanıyor:

$$L_{5,2} = \{w w^R \mid w \in (0+1)^*\}$$

Bu dili türeten bir dilbilgisi aşağıdaki gibi tanımlanabilir:

$$G_{5,2} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S\}$$

$$V_T = \{0, 1\}$$

$$P: S \Rightarrow 0S0 \mid 1S1 \mid \lambda$$

$L_{5,2}$ 'yi tanıyan PDA'yı tanımlayalım:

$$M_{5,2} = \langle Q, \Sigma, \Gamma, \delta, q_0, Z_0, \Phi \rangle$$

$$Q = \{q_0, q_1\}$$

$$\Sigma = \{0, 1\}$$

$$\Gamma = \{A, B, Z_0\}$$

PDA'nın çalışma ilkesi. PDA'nın tanıdığı dizgiler, $\{0, 1\}$ alfabesindeki herhangi bir w alt dizgisi ile bunun tersinden oluşmaktadır. w ile w^R arasında herhangi bir simge yer almamaktadır. Bu dizgileri tanıyan PDA:

- Önce w 'nin simgelerini okur ve yığıtı okuduğu her 0 için bir A, her 1 için de bir B ekler. PDA'nın yığıt alfabesi, giriş alfabesindeki simgeleri içerebileceği gibi, bu simgeleri içermeyebilir de. Örnek 5.1'de, giriş alfabesindeki tüm simgeler yığıt alfabesinde yer almaktadır. Örnek 5.2'de ise, yığıt alfabesinde, giriş alfabesindeki simgelere yer verilmemiştir.
- w 'nin simgeleri okunurken PDA q_0 durumundadır. w bitip, w^R başladığında PDA q_1 durumuna geçecektir. Ancak giriş dizgisinde, w 'nin bitip (w^R)'nin başladığını belirten özel bir simge yoktur. Okunan simge eğer bir önceki simgenin aynısı ise, bu simge w 'nin bir sonraki simgesi olabileceği gibi, (w^R)'nin ilk simgesi de olabilir. Bu koşullarda iki hareket tanımlamak gerekir: birinci harekette PDA ekleme durumunda (q_0) kalır ve yığıtı ekleme yapar; ikinci harekette ise PDA silme durumuna (q_1) geçer ve yığıtın tepesindeki simgeyi siler.
- Silme durumuna (q_1) geçen PDA (w^R)'nin simgelerini okur, ve okuduğu her simge için eğer yığıtın tepesinde bu simgenin karşılığı (0 için A, 1 için B) varsa bunu yığıttan çıkarır (siler).
- Giriş dizgisinin tüm simgeleri okunduktan sonra, eğer yığıtın tepesinde Z_0 varsa, PDA bunu da siler ve yığıtı tamamen boşaltır.

Yukarıda sözlü olarak tanımlanan işlemleri yapan PDA'nın hareket işlevi biçimsel olarak aşağıdaki gibi tanımlanır:

$$\delta: \delta(q_0, 0, Z_0) = (q_0, AZ_0)$$

$$\delta(q_0, 1, Z_0) = (q_0, BZ_0)$$

$$\delta(q_0, \lambda, Z_0) = (q_1, \lambda)$$

$$\delta(q_0, 0, A) = \{(q_0, AA), (q_1, \lambda)\}$$

$$\delta(q_0, 1, A) = (q_0, BA)$$

$$\delta(q_0, 0, B) = (q_0, AB)$$

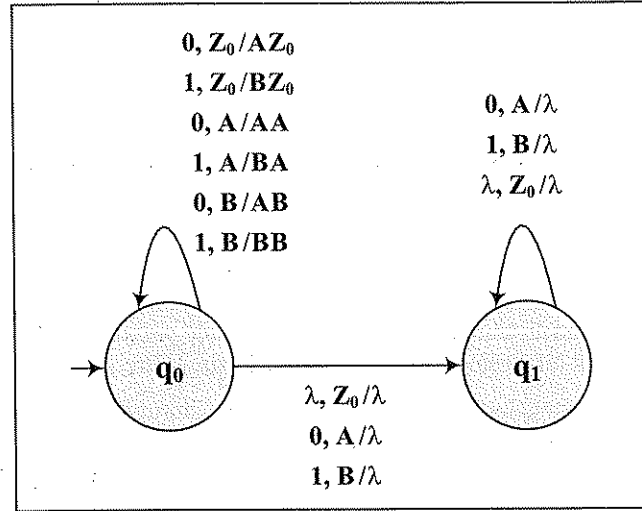
$$\delta(q_0, 1, B) = \{(q_0, BB), (q_1, \lambda)\}$$

$$\delta(q_1, 0, A) = (q_1, \lambda)$$

$$\delta(q_1, 1, B) = (q_1, \lambda)$$

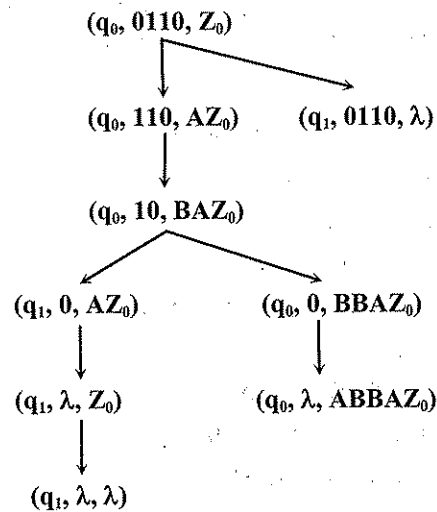
$$\delta(q_1, \lambda, Z_0) = (q_1, \lambda)$$

PDA'nın hareket işlevi yukarıdaki biçimde tanımlanabileceği gibi, Çizim 5.3'deki çizim ile de tanımlanabilir.



Çizim 5.3. $L_{5,2}$ Dilini Tanıyan PDA'nın ($M_{5,2}$) Geçiş Çizeneği

$M_{5,2}$ deterministik olmayan (*non deterministic*) bir PDA'dır. $w = 0110$ dizgisinin bu PDA tarafından tanındığını, anlık tanımları izleyerek aşağıdaki gibi bulabiliriz:



($q_0, 0110, Z_0$) başlangıç ID'sinden (q_1, λ, λ) ID'sine ulaşılabildiği için bu makine 0110 dizgisini tanıtır.

Örnek 5.3. $L_{5,3}$ dili, $\{0, 1\}$ alfabesindeki palindrom'ları içeren dil olarak tanımlanıyor. Palindrom, her iki yönden okunuşu aynı olan dizgiler kümesidir. Türkçedeki palindrom'lara örnek olarak *ebe, aba, kok, talat, kepek, teğet, kelek* .. gibi sözcükler verilebilir. $\{0, 1\}$ alfabesindeki palindromlara örnek olarak ise $\lambda, 0, 1, 00, 11, 000, 010, 100001, 101101, \dots$ gibi dizgiler sayılabilir. $\{0, 1\}$ alfabesindeki palindromlar, biçimsel olarak aşağıdaki gibi tanımlanabilir:

$$L_{5,3} = \{ww^R + w0w^R + w1w^R \mid w \in (0+1)^*\}$$

Bu dili türeten bir dilbilgisi aşağıdaki gibi tanımlanabilir:

$$G_{5,3} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S\}$$

$$V_T = \{0, 1\}$$

$$P: S \Rightarrow 0S0 \mid 1S1 \mid \lambda \mid 0 \mid 1$$

$L_{5,3}$ 'ü tanıyan PDA'yı tanımlayalım:

$$M_{5,3} = \langle Q, \Sigma, \Gamma, \delta, q_0, Z_0, \Phi \rangle$$

$$Q = \{q_0, q_1\}$$

$$\Sigma = \{0, 1\}$$

$$\Gamma = \{0, 1, Z_0\}$$

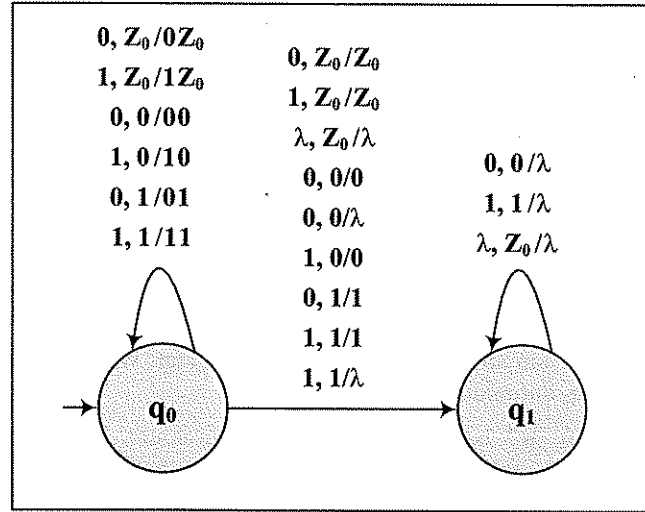
PDA'nın çalışma ilkesi. PDA'nın tanıdığı dizgiler, $\{0, 1\}$ alfabesindeki herhangi bir w alt dizgisi ile bunun tersinden oluşmaktadır. w ile w^R arasında herhangi bir simge yer almayabileceği gibi, 0 ya da 1 simgesi de yer alabilir. Bu dizgileri tanıyan PDA:

- Önce w 'nin simgelerini okur ve yığığa ekler. w 'nin simgeleri okunurken PDA q_0 durumundadır.
- w bitip, w^R başladığında PDA q_1 durumuna geçecektir. Ancak giriş dizgisinde, w 'nin bitip (w^R)'nin başladığını belirten özel bir simge yoktur. Okunan simge w 'nin bir sonraki simgesi olabileceği gibi, w ve w^R arasındaki orta simge ya da (w^R)'nin ilk simgesi de olabilir. Hareketlerin buna göre tanımlanması gerekir. PDA deterministik olmayacaktır.
- Silme durumuna (q_1) geçen PDA (w^R)'nin simgelerini okur, ve okuduğu her simge için eğer yığığın tepesinde aynı simge varsa bunu yığıttan çıkarır (siler).
- Giriş dizgisinin tüm simgeleri okunduktan sonra, eğer yığığın tepesinde Z_0 varsa, PDA bunu da siler ve yığıtı tamamen boşaltır.

Yukarıda sözlü olarak tanımlanan işlemleri yapan PDA'nın hareket işlevi biçimsel olarak aşağıdaki gibi tanımlanır:

$$\begin{aligned}\delta : \delta(q_0, 0, Z_0) &= \{(q_0, 0Z_0), (q_1, Z_0)\} \\ \delta(q_0, 1, Z_0) &= \{(q_0, 1Z_0), (q_1, Z_0)\} \\ \delta(q_0, \lambda, Z_0) &= (q_1, \lambda) \\ \delta(q_0, 0, 0) &= \{(q_0, 00), (q_1, 0), (q_1, \lambda)\} \\ \delta(q_0, 1, 0) &= \{(q_0, 10), (q_1, 0)\} \\ \delta(q_0, 0, 1) &= \{(q_0, 01), (q_1, 1)\} \\ \delta(q_0, 1, 1) &= \{(q_0, 11), (q_1, 1), (q_1, \lambda)\} \\ \delta(q_1, 0, 0) &= (q_1, \lambda) \\ \delta(q_1, 1, 1) &= (q_1, \lambda) \\ \delta(q_1, \lambda, Z_0) &= (q_1, \lambda)\end{aligned}$$

PDA'nın hareket işlevi yukarıdaki biçimde tanımlanabileceği gibi, Çizim 5.4'deki çizenek ile de tanımlanabilir.



Çizim 5.4. L_{5.3} Dilini Tanıyan PDA'nın (M_{5.3}) Geçiş Çizeneği

Örnek 5.4. L_{5.4} dili aşağıdaki gibi tanımlanıyor:

$$L_{5.4} = \{a^i b^n a^j b^n a^k \mid i, j, k, n \geq 1\}$$

Bu dili türeten bir dilbilgisi aşağıdaki gibi tanımlanabilir:

$$G_{5.4} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b\}$$

$$P : S \Rightarrow ABA$$

$$A \Rightarrow aA \mid a$$

$$B \Rightarrow bBb \mid bAb$$

L_{5.4}'ü tanıyan PDA'yı tanımlayalım:

$$M_{5.4} = \langle Q, \Sigma, \Gamma, \delta, q_0, Z_0, \Phi \rangle$$

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$$

$$\Sigma = \{a, b\}$$

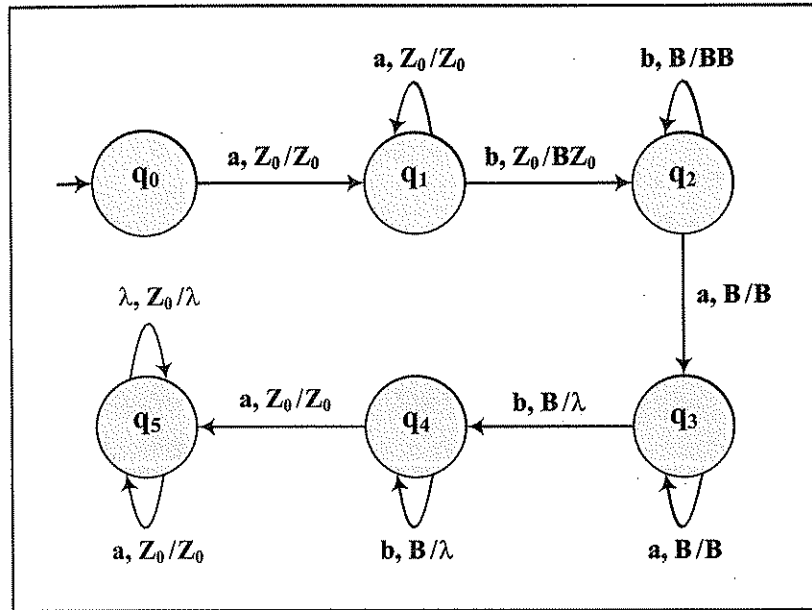
$$\Gamma = \{B, Z_0\}$$

PDA'nın çalışma ilkesi. PDA'nın tanıdığı dizgiler, 5 öbekten oluşmaktadır. Her öbekte en az bir simge yer almaktadır. 1., 3., ve 5. öbeklerdeki simgeler a; 2. ve 4. öbeklerdeki simgeler ise b'dir. 2. ve 4. öbeklerdeki b'lerin sayıları da eşittir. Buna göre PDA'nın 1., 3. ve 5. öbeklerdeki a'ların sayılarının en az bir olduğunu denetlemesi yeterlidir. Yığıt 2. ve 4. öbeklerdeki b'lerin sayılarının eşit olduğunu denetlemek için kullanılır. Bunun için de 2. öbekteki b'ler okunurken yığta eklenir; 4. öbekteki b'ler okunurken de yığıt boşaltılır.

Yukarıda sözlü olarak tanımlanan işlemleri yapan PDA'nın hareket işlevi biçimsel olarak aşağıdaki gibi tanımlanır:

$$\begin{aligned}\delta : \delta(q_0, a, Z_0) &= (q_1, Z_0) & \delta(q_3, b, B) &= (q_4, \lambda) \\ \delta(q_1, a, Z_0) &= (q_1, Z_0) & \delta(q_4, b, B) &= (q_4, \lambda) \\ \delta(q_1, b, Z_0) &= (q_2, BZ_0) & \delta(q_4, a, Z_0) &= (q_5, Z_0) \\ \delta(q_2, b, B) &= (q_2, BB) & \delta(q_5, a, Z_0) &= (q_5, Z_0) \\ (q_2, a, B) &= (q_3, B) & \delta(q_5, \lambda, Z_0) &= (q_5, \lambda) \\ \delta(q_3, a, B) &= (q_3, B)\end{aligned}$$

PDA'nın hareket işlevi yukarıdaki biçimde tanımlanabileceği gibi, Çizim 5.5'deki çizenek ile de tanımlanabilir.

Çizim 5.5. $L_{5,4}$ Dilini Tanıyan PDA'nın ($M_{5,4}$) Geçiş Çizeneği

PDA'nın Deterministik Olma Koşulu

Yukarıdaki örneklerin de gösterdiği gibi, bağlamdan-bağımsız her dil (CFL) için, bu dili tanıyan deterministik bir PDA bulmak mümkün değildir. Başka bir deyişle, deterministik PDA'lar tarafından tanınan diller, bağlamdan-bağımsız dillerin bir altkümesidir. Her CFL için, dili tanıyan bir PDA bulunabilir. Ancak CFL'lerin yalnız bir altkümesi için, dili tanıyan deterministik bir PDA bulmak mümkündür. Diğer CFL'ler için ise, dili tanıyan deterministik olmayan PDA'lar bulunabilir; ancak dili tanıyan deterministik PDA bulmak mümkün değildir.

Bir PDA'nın deterministik olabilmesi için, her anlık tanım için, yapılabilecek hareket sayısının en çok bir olması; başka bir deyişle aynı koşullarda hiçbir zaman birden çok hareket yapılamaması gerekir. Bunun için de:

- Her $\delta(q, a, X)$ için tanımlı en çok bir hareket olması,
- Eğer $\delta(q, \lambda, X)$ için bir hareket tanımlı ise de, hiçbir a giriş simgesi için, $\delta(q, a, X)$ hareketinin tanımlı olmaması

gerekir. Özellikle aynı koşullarda sağlanan bir normal hareket ile bir λ -hareketi tanımlı ise PDA'nın deterministik olmayacağı görülmektedir. Yukarıda verilen örneklerden, $M_{5,1}$ 'in deterministik olduğu; $M_{5,2}$ ile $M_{5,3}$ 'ün de deterministik olmadığı açıktır. $M_{5,4}$ 'e gelince, bu PDA'nın q_0, q_1, q_2, q_3 ve q_4 durumları için tanımlı hareketler deterministik. Ancak q_5 durumu için tanımlı iki hareket nedeniyle $M_{5,4}$ deterministik değildir.

5.2. Bağlamdan-Bağımsız Dilbilgisi (CFG) – Yığıtlı Özdevinir (PDA) İlişkisi

Bağlamdan-bağımsız bir dilbilgisi tarafından türetilen dil bağlamdan-bağımsız bir dildir. Bağlamdan-bağımsız bir dilbilgisi verildiğinde, bu dilbilgisi tarafından türetilen dili tanıyan bir PDA bulunabilir. Diğer taraftan, bir PDA verildiğinde, bu PDA tarafından tanınan bağlamdan-bağımsız dili türeten bağlamdan-bağımsız bir dilbilgisi bulunabilir. Dolayısıyla, PDA'larla CFG'ler arasında bir eşdeğerlik ilişkisi vardır. Bu eşdeğerlik dillerin eşitliği üzerine kuruludur. Eğer bir PDA'nın tanıdığı dil ile bir CFG'nin türettiği dil aynı dil ise, bu PDA ile bu CFG birbirinin eşdeğeridir. Bu bölümde, verilen bir CFG'nin eşdeğeri olan PDA'nın nasıl bulunacağı ile verilen bir PDA'nın eşdeğeri olan CFG'nin nasıl bulunacağı görülecektir.

5.2.1. Verilen bir CFG'nin Eşdeğeri PDA'nın Bulunması

Önerme 5.1. Eğer G bağlamdan-bağımsız bir dilbilgisi ise, bu dilbilgisinin türettiği bağlamdan-bağımsız $L(G)$ dilini tanıyan bir PDA vardır.

Bu önermenin doğruluğunu ispat etmek için, verilen bir CFG'nin eşdeğeri PDA'nın nasıl bulunacağı gösterilecektir. CFG'yi G , PDA'yı da M ile gösterirsek, verilen bir CFG için $T(M) = L(G)$ olan PDA'nın bulunması için iki yöntem sunulacaktır. Yöntemlerden birincisinin kullanılabilmesi için dilbilgisinin Greibach normal biçimde (GNF) olması gerekir. İkinci yöntem için ise dilbilgisinin normal biçimlerden birinde olması gerekmez. Ancak ikinci yöntem için, PDA tanımında küçük bir değişiklik yapmak gerekir.

CFG'nin Eşdeğeri PDA'nın Bulunması İçin 1. Yöntem

Bağlamdan-bağımsız dilbilgisi : $G = \langle V_N, V_T, P, S \rangle$

Bu yöntemin uygulanabilmesi için dilbilgisinin Greibach normal biçiminde olması, bunun içinde yeniden yazma kurallarının:

$$A \Rightarrow a\alpha$$

$$A \Rightarrow \lambda : A \in V_N, a \in V_T, \alpha \in V_N^*$$

biçiminde olması gerekir. Verilen dilbilgisine eşdeğer PDA aşağıdaki gibi bulunur:

$$M = \langle Q, \Sigma, \Gamma, q_0, \delta, Z_0, F \rangle$$

$$Q = \{q_0\}$$

$$\Sigma = V_T$$

$$\Gamma = V_N$$

$$Z_0 = S$$

$$F = \Phi$$

δ : Dilbilgisinin

$$A \Rightarrow a\alpha$$

biçimindeki her yeniden yazma kuralına karşılık, PDA'da

$$\delta(q_0, a, A) = (q_0, \alpha)$$

hareketi tanımlanır.

Eğer $\alpha = \lambda$ ise, $A \Rightarrow a$ kuralına karşılık $\delta(q_0, a, A) = (q_0, \lambda)$ hareketi tanımlanmış olur.

$F = \Phi$ olduğuna göre, boş yığıtla tanıyan bir PDA oluşturulmaktadır. Oluşturulan PDA'nın:

- giriş alfabesi, dilbilgisinin uç simgeler kümesine eşittir.
- yığıt alfabesi, dilbilgisinin sözdizim değişkenleri kümesine eşittir.
- yığıt başlangıç simgesi, dilbilgisinin başlangıç değişkeni olan S'dir.
- durumlar kümesi tek durumlu $\{q_0\}$ kümesidir.
- durum geçiş işlevi, dilbilgisinin yeniden yazma kurallarından türetilmektedir.

Örnek 5.5. Prefix notasyonundaki (işleç öncelikli) aritmetik deyimleri türeten $G_{5.5}$ dilbilgisi aşağıdaki gibi tanımlanıyor.

$$G_{5.5} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S\}$$

$$V_T = \{+, -, *, /, v, c\}$$

$$P: S \Rightarrow +SS \mid -SS \mid *SS \mid /SS \mid v \mid c$$

Bu dilbilgisinin türettiği bağlamdan-bağımsız dili tanıyan PDA aşağıdaki gibi bulunur.

$$M_{5.5} = \langle Q, \Sigma, \Gamma, q_0, \delta, Z_0, \Phi \rangle$$

$$Q = \{q_0\}$$

$$\Sigma = \{+, -, *, /, v, c\}$$

$$\Gamma = \{S\}$$

$$Z_0 = S$$

$$F = \Phi$$

$$\delta: \delta(q_0, +, S) = (q_0, SS)$$

$$\delta(q_0, -, S) = (q_0, SS)$$

$$\delta(q_0, *, S) = (q_0, SS)$$

$$\delta(q_0, /, S) = (q_0, SS)$$

$$\delta(q_0, v, S) = (q_0, \lambda)$$

$$\delta(q_0, c, S) = (q_0, \lambda)$$

Dilbilgisinin türettiği örnek bir dizgi bulalım ve bu dizginin PDA tarafından tanındığını gösterelim.

$$S \Rightarrow *SS \Rightarrow *+SSS \Rightarrow *+v.SS \Rightarrow *+v/SSS$$

$$\Rightarrow *+v/v.SS \Rightarrow *+v/v.c.S \Rightarrow *+v/v.c.-SS$$

$$\Rightarrow *+v/v.c.-v.S \Rightarrow *+v/v.c.-v.*SS$$

$$\Rightarrow *+v/v.c.-v.*c.S \Rightarrow *+v/v.c.-v.*cv$$

$w = *+v/v.c.-v.*cv$ dizgisi $G_{5.5}$ tarafından türetilen bir dizgidir. Prefix notasyonundaki bu dizginin karşılığı $w = (v+v/c)*(v-\xi v)$ aritmetik deyimidir. Anlık tanımlar dizisi ile w 'nin $M_{5.5}$ tarafından tanınıp tanınmadığını araştıralım:

$$(q_0, *+v/v.c.-v.*cv, S) \vdash (q_0, +v/v.c.-v.*cv, SS)$$

$$\vdash (q_0, v/v.c.-v.*cv, SSS) \vdash (q_0, /v.c.-v.*cv, SS)$$

$$\vdash (q_0, v.c.-v.*cv, SSS) \vdash (q_0, c.-v.*cv, SS)$$

$$\vdash (q_0, -v.*cv, S) \vdash (q_0, v.*cv, SS)$$

$$\vdash (q_0, *.cv, S) \vdash (q_0, c.v, SS)$$

$$\vdash (q_0, v, S) \vdash (q_0, \lambda, \lambda)$$

Sonuçta tüm dizgi okunduğu ve yığıt boşaldığı için w PDA tarafından tanınır.

Örnek 5.6. $\{a, b\}$ alfabelinde, eşit sayıda a ve b içeren dizgileri türeten aşağıdaki CFG veriliyor.

$$G_{5.6} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b\}$$

$$P: S \Rightarrow bA \mid aB$$

$$A \Rightarrow bAA \mid aS \mid a$$

$$B \Rightarrow aBB \mid bS \mid b$$

Bu dilbilgisinin türettiği bağlamdan-bağımsız dili tanıyan PDA aşağıdaki gibi bulunur.

$$M_{5.6} = \langle Q, \Sigma, \Gamma, q_0, \delta, Z_0, \Phi \rangle$$

$$Q = \{q_0\}$$

$$\Sigma = \{a, b\}$$

$$\Gamma = \{S, A, B\}$$

$$Z_0 = S$$

$$F = \Phi$$

$$\delta: \delta(q_0, b, S) = (q_0, A)$$

$$\delta(q_0, a, S) = (q_0, B)$$

$$\delta(q_0, b, A) = (q_0, AA)$$

$$\delta(q_0, a, A) = \{(q_0, S), (q_0, \lambda)\}$$

$$\delta(q_0, a, B) = (q_0, BB)$$

$$\delta(q_0, b, B) = \{(q_0, S), (q_0, \lambda)\}$$

Bulunan PDA, deterministik olmayan bir PDA'dır.

CFG'nin Eşdeğeri PDA'nın Bulunması İçin 2. Yöntem

Bu yöntemin kullanılabilmesi için, PDA tanımında küçük bir değişiklik yapmak gerekir. PDA için daha önce tanımlanan temel modelde, hareketler aşağıdaki gibi gösteriliyordu:



Bu gösterimde a giriş şeridinden okunan simgeyi, Z yığıtın tepesindeki simgeyi, α ise yığıtta eklenen yığıt simgeleri dizgisini göstermektedir. Temel modeldeki bu tanımda a ve α yerine λ gelebilmekte; ancak Z yerine λ gelememektedir. Şimdi Z 'nin yerine de λ gelebileceğini belirterek PDA'nın değişik bir tanımını yapıyoruz. İlk yapılan tanım ile bu ikinci tanım arasında, model açısından bir fark yoktur. Çünkü Z yerine λ konulduğunda, yığıtın tepesindeki simge ne olursa olsun hareketin yapılabileceği, ve hareket sırasında yığıtın tepesindeki simgenin silinmeyeceği anlaşılmaktadır. Bu durum modelin gücünde bir değişiklik yapmaz. Çünkü,

$$\delta(q_i, a, \lambda) = (q_j, \alpha)$$

yerine

$$\forall Z_k \in \Gamma \quad \delta(q_i, a, Z_k) = (q_j, \alpha Z_k)$$

hareketleri tanımlandığında, aynı sonuç elde edilir.

PDA'nın tanımında bu değişikliği yaptıktan sonra, Greibach normal biçiminde olmayan bağlamdan-bağımsız bir dilbilgisinin türettiği dili tanıyan PDA'yı aşağıdaki gibi bulabiliriz:

$$\text{Bağlamdan-bağımsız dilbilgisi : } G = \langle V_N, V_T, P, S \rangle$$

Verilen dilbilgisine eşdeğer PDA:

$$M = \langle Q, \Sigma, \Gamma, q_0, \delta, Z_0, F \rangle$$

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = V_T$$

$$\Gamma = V_N \cup V_T \cup \{Z_0\}$$

$$F = \{q_2\}$$

δ :

- Dilbilgisi tanımından bağımsız olarak PDA'da

$$\delta(q_0, \lambda, \lambda) = (q_1, S) \quad \text{ve} \quad \delta(q_1, \lambda, Z_0) = (q_2, \lambda)$$

hareketleri tanımlanır.

- Dilbilgisindeki her a giriş simgesi için PDA'da

$$\delta(q_1, a, a) = (q_1, \lambda)$$

hareketi tanımlanır.

- Dilbilgisindeki her $A \Rightarrow \alpha$ yeniden yazma kuralı için PDA'da

$$\delta(q_1, \lambda, A) = (q_1, \alpha)$$

hareketi tanımlanır.

$F = \{q_2\}$ olduğuna göre, uç durumla tanıyan bir PDA oluşturulmaktadır. Ancak oluşturulan PDA tanıma anında yığıtı da boşaltmaktadır.

Örnek 5.7. $\{a, b\}$ alfabesinde aşağıdaki CFG veriliyor.

$$G_{5.7} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S\}$$

$$V_T = \{a, b\}$$

$$P: S \Rightarrow aSb \mid aSbb \mid ab \mid abb$$

Bu dilbilgisi tarafından türetilen bağlamdan-bağımsız dil aşağıdaki gibi tanımlanabilir:

$$L_{5.7} = \{a^n b^m \mid 1 \leq n, n \leq m \leq 2n\}$$

Verilen dilbilgisine eşdeğer PDA:

$$M_{5.7} = \langle Q, \Sigma, \Gamma, q_0, \delta, Z_0, \Phi \rangle$$

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{a, b\}$$

$$\Gamma = \{a, b, S, Z_0\}$$

$$F = \{q_2\}$$

$$\delta: \delta(q_0, \lambda, \lambda) = (q_1, S)$$

$$\delta(q_1, a, a) = (q_1, \lambda)$$

$$\delta(q_1, b, b) = (q_1, \lambda)$$

$$\delta(q_1, \lambda, S) = (q_1, aSb)$$

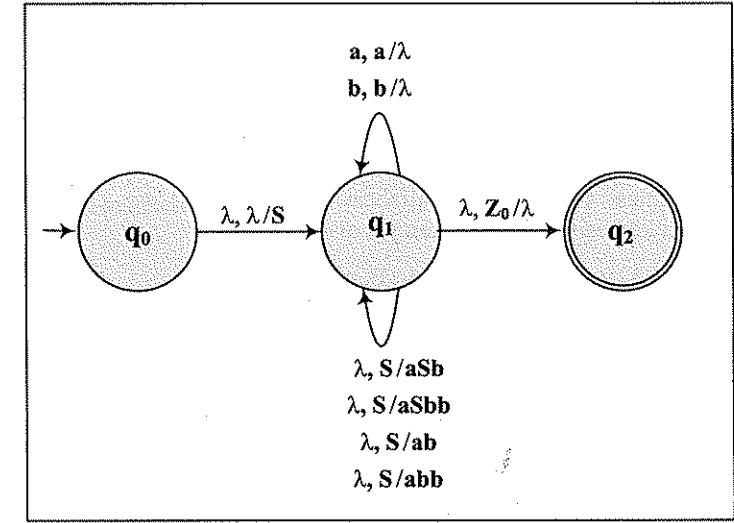
$$\delta(q_1, \lambda, S) = (q_1, aSbb)$$

$$\delta(q_1, \lambda, S) = (q_1, ab)$$

$$\delta(q_1, \lambda, S) = (q_1, abb)$$

$$\delta(q_1, \lambda, Z_0) = (q_2, \lambda)$$

Bulunan PDA, deterministik olmayan bir PDA'dır. PDA'nın hareketleri Çizim 5.6'daki çizimle de gösterilebilir.



Çizim 5.6. $L_{5.7}$ Dilini Tanıyan PDA'nın ($M_{5.7}$) Geçiş Çizeneği

5.2.2. Verilen bir PDA'nın Eşdeğeri CFG'nin Bulunması

Önerme 5.2. Eğer M bir PDA ise, türettiği dil bu PDA'nın tanıdığı dile eşit olan ($G(L) = T(M)$) bağlamdan-bağımsız bir dilbilgisi (CFG) vardır.

$$PDA: M = \langle Q, \Sigma, \Gamma, q_0, \delta, Z_0, F \rangle$$

Verilen PDA'ya eşdeğer, Greibach normal biçimindeki bağlamdan-bağımsız dilbilgisi aşağıdaki gibi bulunur:

$$G = \langle V_N, V_T, P, S \rangle$$

$$V_T = \Sigma$$

$$V_N = \{S, [q, A, p] \mid A \in \Gamma, p, q \in Q\}$$

$P:$

- PDA'nın her durumuna karşılık CFG'de bir yeniden yazma kuralı:
 $\forall q \in Q : S \Rightarrow [q_0, Z_0, q]$
- PDA'nın her hareketine karşılık CFG'de bir ya da birçok yeniden yazma kuralı:

$$\forall \delta(q_i, a, A) = (q_j, B_1 B_2 B_3 \dots B_k) \in P :$$

her $p_1, p_2, \dots, p_k \in Q$ birleşimi için bir yeniden yazma kuralı

$$[q_i, A, p_k] \Rightarrow a[q_j, B_1, p_1] [p_1, B_2, p_2] \dots [p_{k-1}, B_k, p_k]$$

Bu yönteme göre, verilen bir PDA'ya eşdeğer CFG aşağıdaki gibi bulunuyor:

1. Dilbilgisinin uç simgeler kümesi, PDA'nın giriş alfabesine eşit olur : $V_T = \Sigma$
2. Dilbilgisinin sözdizim değişkenleri, S ile [durum, yığıt simgesi, durum] üçlülerinden oluşur. Buna göre eğer PDA'da n adet durum ile k adet yığıt simgesi varsa, CFG'de $(kn^2 + 1)$ adet sözdizim değişkeni bulunacaktır. Örneğin PDA'nın durumlar kümesi $\{q_0, q_1\}$, yığıt alfabesi ise $\{A, B, Z_0\}$ ise, CFG'nin sözdizim değişkenleri kümesinde 13 değişken yer alacaktır.

$$V_N = \{S, [q_0, A, q_0], [q_0, A, q_1], [q_0, B, q_0], [q_0, B, q_1], [q_0, Z_0, q_0], [q_0, Z_0, q_1], [q_1, A, q_0], [q_1, A, q_1], [q_1, B, q_0], [q_1, B, q_1], [q_1, Z_0, q_0], [q_1, Z_0, q_1]\}$$

3. CGG'nin yeniden yazma kuralları aşağıdaki gibi oluşturulur:

- 3.1. PDA'nın her durumuna karşılık bir yeniden yazma kuralı oluşturulur:

$$\forall q \in Q : S \Rightarrow [q_0, Z_0, q]$$

- 3.2. PDA'nın

$$\delta(q_0, \lambda, A) = (q_1, \lambda)$$

biçimindeki her hareketine karşılık,

$$[q_0, A, q_1] \Rightarrow \lambda$$

biçiminde bir yeniden yazma kuralı oluşturulur.

- 3.3. PDA'nın

$$\delta(q_0, a, A) = (q_1, \lambda)$$

biçimindeki her hareketine karşılık

$$[q_0, A, q_1] \Rightarrow a$$

biçiminde bir yeniden yazma kuralı oluşturulur.

- 3.4. PDA'nın

$$\delta(q_0, a, A) = (q_1, B_1)$$

biçimindeki her hareketine karşılık, her durum için ($\forall p_1 \in Q$)

$$[q_0, A, p_1] \Rightarrow a[q_1, B_1, p_1]$$

biçiminde bir yeniden yazma kuralı oluşturulur.

- 3.5. PDA'nın

$$\delta(q_0, a, A) = (q_1, B_1 B_2)$$

biçimindeki her hareketine karşılık, her p_1, p_2 durum çifti için ($\forall p_1, p_2 \in Q$)

$$[q_0, A, p_2] \Rightarrow a[q_1, B_1, p_1] [p_1, B_2, p_2]$$

yeniden yazma kuralları.

- 3.6. PDA'nın

$$\delta(q_0, a, A) = (q_1, B_1 B_2 B_3)$$

biçimindeki her hareketine karşılık, her p_1, p_2, p_3 durum birleşimi için ($\forall p_1, p_2, p_3 \in Q$)

$$[q_0, A, p_3] \Rightarrow a[q_1, B_1, p_1] [p_1, B_2, p_2] [p_2, B_3, p_3]$$

biçiminde bir yeniden yazma kuralı oluşturulur.

.....

Örnek 5.8. Aşağıdaki PDA veriliyor.

$$M_{5.8} = \langle Q, \Sigma, \Gamma, q_0, \delta, Z_0, \Phi \rangle$$

$$Q = \{q_0, q_1\}$$

$$\Sigma = \{0, 1\}$$

$$\Gamma = \{Z_0, X\}$$

$$\delta: \delta(q_0, 0, Z_0) = (q_0, X Z_0)$$

$$\delta(q_0, 0, X) = (q_0, X X)$$

$$\delta(q_0, 1, X) = (q_1, \lambda)$$

$$\delta(q_1, 1, X) = (q_1, \lambda)$$

$$\delta(q_1, \lambda, Z_0) = (q_1, \lambda)$$

Bu PDA'nın tanıdığı dil aşağıdaki gibi tanımlanabilir:

$$L_{5.8} = \{a^n b^n \mid n \geq 1\}$$

Bu dili türeten bir CFG aşağıdaki gibi bulunur:

$$G_{5.8} = \langle V_N, V_T, P, S \rangle$$

$$V_T = \{0, 1\}$$

$$V_N = \{S, A_1, A_2, A_3, A_4, A_5, A_6, A_7, A_8\}$$

$$A_1 = [q_0, X, q_0] \quad A_5 = [q_1, X, q_0]$$

$$A_2 = [q_0, X, q_1] \quad A_6 = [q_1, X, q_1]$$

$$A_3 = [q_0, Z_0, q_0] \quad A_7 = [q_1, Z_0, q_0]$$

$$A_4 = [q_0, Z_0, q_1] \quad A_8 = [q_1, Z_0, q_1]$$

Yeniden yazma kuralları:

- PDA'nın durumlarına karşılık oluşturulan kurallar:

$$S \Rightarrow [q_0, Z_0, q_0]$$

$$S \Rightarrow [q_0, Z_0, q_1]$$

- $\delta(q_0, 0, Z_0) = (q_0, XZ_0)$ hareketine karşılık oluşturulan kurallar:

$$[q_0, Z_0, q_0] \Rightarrow 0 [q_0, X, q_0] [q_0, Z_0, q_0]$$

$$[q_0, Z_0, q_1] \Rightarrow 0 [q_0, X, q_0] [q_0, Z_0, q_1]$$

$$[q_0, Z_0, q_0] \Rightarrow 0 [q_0, X, q_1] [q_1, Z_0, q_0]$$

$$[q_0, Z_0, q_1] \Rightarrow 0 [q_0, X, q_1] [q_1, Z_0, q_1]$$

- $\delta(q_0, 0, X) = (q_0, XX)$ hareketine karşılık oluşturulan kurallar:

$$[q_0, X, q_0] \Rightarrow 0 [q_0, X, q_0] [q_0, X, q_0]$$

$$[q_0, X, q_1] \Rightarrow 0 [q_0, X, q_0] [q_0, X, q_1]$$

$$[q_0, X, q_0] \Rightarrow 0 [q_0, X, q_1] [q_1, X, q_0]$$

$$[q_0, X, q_1] \Rightarrow 0 [q_0, X, q_1] [q_1, X, q_1]$$

- $\delta(q_0, 1, X) = (q_1, \lambda)$ hareketine karşılık oluşturulan kural:

$$[q_0, X, q_1] \Rightarrow 1$$

- $\delta(q_1, 1, X) = (q_1, \lambda)$ hareketine karşılık oluşturulan kural:

$$[q_1, X, q_1] \Rightarrow 1$$

- $\delta(q_1, \lambda, Z_0) = (q_1, \lambda)$ hareketine karşılık oluşturulan kural:

$$[q_1, Z_0, q_1] \Rightarrow \lambda$$

Değişkenler için üçlü gösterimler yerine sembolik adları ($A_1, A_2, \dots$) kullanarak yeniden yazma kurallarını toplu olarak yazalım:

$$P: S \Rightarrow A_3 \mid A_4$$

$$A_3 \Rightarrow 0 A_1 A_3 \mid 0 A_2 A_7$$

$$A_4 \Rightarrow 0 A_1 A_4 \mid 0 A_2 A_8$$

$$A_1 \Rightarrow 0 A_1 A_1 \mid 0 A_2 A_5$$

$$A_2 \Rightarrow 0 A_1 A_2 \mid 0 A_2 A_6 \mid 1$$

$$A_6 \Rightarrow 1$$

$$A_8 \Rightarrow \lambda$$

Yeniden yazma kuralları incelendiğinde:

A_1, A_3, A_5 ve A_7 değişkenlerinin yararsız (*useless*) değişkenler, olduğu görülür.

Bu değişkenlerin yer aldığı kurallar çıkarıldığında, aşağıdaki yeniden yazma kuralları kalır.

$$P: S \Rightarrow A_4$$

$$A_4 \Rightarrow 0 A_2 A_8$$

$$A_2 \Rightarrow 0 A_2 A_6 \mid 1$$

$$A_6 \Rightarrow 1$$

$$A_8 \Rightarrow \lambda$$

Bu kurallar da sadeleştirildiğinde aşağıdaki yeniden yazma kuralları elde edilir:

$$P: S \Rightarrow 0 A_2$$

$$A_2 \Rightarrow 0 A_2 1 \mid 1$$

Sonuç olarak, dilbilgisi iki sözdizim değişkeni ve üç yeniden yazma kuralı ile aşağıdaki gibi tanımlanabilir.

$$G_{5.8} = \langle V_N, V_T, P, S \rangle$$

$$V_T = \{0, 1\}$$

$$V_N = \{S, A\}$$

$$P: S \Rightarrow 0 A$$

$$A \Rightarrow 0 A 1 \mid 1$$

Sorular

S.5.1. $L_{S.5.1} = \{a^n b^m c^k \mid n \geq 1, m \geq 2, k \geq 1, m = n + k\}$

- a) $L_{S.5.1}$ dilini türeten bağlamdan-bağımsız bir dilbilgisi tanımlayınız.
- b) $L_{S.5.1}$ dilini boş yığıtla tanıyan bir PDA tanımlamanız isteniyor. Bunun için önce PDA'nın nasıl çalışacağını belirleyip sözel olarak açıklayınız. Sonra da PDA'nın biçimsel tanımını veriniz. Bu bağlamda PDA'nın hareketlerinin önce işlev tanımlarını yazınız, sonra da hareketleri bir geçiş çizeneği ile gösteriniz.

S.5.2. $L_{S.5.2} = \{a^n b^{n-k} c^k \mid n \geq 1, k \geq 1, n \geq k\}$

dilini boş yığıtla tanıyan bir PDA tasarlamamız isteniyor. Bunun için:

- a) PDA'nın nasıl çalışacağını belirleyip sözel olarak açıklayınız.
- c) PDA'nın biçimsel tanımını veriniz. Bu bağlamda PDA'nın hareketlerinin önce işlev tanımlarını yazınız sonra da hareketleri bir geçiş çizeneği ile gösteriniz.

S.5.3. $\{a, b, c\}$ alfabesindeki bağlamdan-bağımsız (CF) $L_{S.5.3}$ dili "içindeki c'lerin sayısı b'lerin sayısının iki katı olan dizgiler kümesi" olarak tanımlanıyor. $L_{S.5.3}$ dili λ 'yı içermiyor. Dilin tümcelerinden bazı örnekler aşağıda görülmektedir.

$L_{S.5.3} = \{a, aaa, bcac, cbc, baacccacb, ccaaacbcabbcaaaac, \dots\}$

- a) $L_{S.5.3}$ dilini boş yığıtla tanıyan bir PDA oluşturmanız isteniyor. Bunun için önce PDA'nın nasıl çalışacağını kısaca açıklayınız. Daha sonra PDA'nın biçimsel tanımını veriniz ve durumların anlamlarını belirtiniz.
- b) PDA'nın hareketlerini bir çizenekle gösteriniz. Oluşturduğunuz PDA deterministik midir? Niçin?
- c) $L_{S.5.3}$ dilini türeten bağlamdan-bağımsız bir dilbilgisi (CFG) tanımlayınız ve $w = abcaabccac$ tümcesi için türetme ağacını oluşturunuz.

S.5.4. Bağlamdan-bağımsız (CF) $L_{S.5.4}$ dili $\{a, b, c\}$ alfabesinde, içindeki b ve c'lerin sayılarının toplamı, a'ların sayısına eşit olan dizgiler (strings) kümesi olarak tanımlanıyor. $L_{S.5.4}$ dili λ 'yı içermiyor.

$L_{S.5.4} = \{ab, ca, baab, acab, caaababc, \dots\}$

$L_{S.5.4}$ dilini boş yığıtla tanıyan deterministik (λ -hareketleri dışında) bir PDA tasarlamamız isteniyor. Bunun için:

- a) PDA'nın çalışma ilkesini (nasıl çalışacağını) belirleyip sözel olarak açıklayınız.
- b) PDA'nın durumlarını belirleyip tanımlayınız (durum sayısının gerektiği kadar ve olabildiğince az olması isteniyor).
- c) PDA'nın biçimsel tanımını veriniz.
- d) $w = aabcca$ tümcesinin PDA tarafından nasıl tanındığını ID'ler dizisi ile gösteriniz.

S.5.5. $G_{S.5.5}$ dilbilgisi aşağıdaki gibi tanımlanıyor.

$G_{S.5.5} = \langle V_N, V_T, P, S \rangle$

$V_N = \{S, A, B\}$

$V_T = \{a, b, c, d\}$

$P: S \Rightarrow aAb \mid cBd \mid ab \mid cd$

$A \Rightarrow aAb \mid ab$

$B \Rightarrow cBd \mid cd$

- a) Bu dilbilgisinin türettiği dili tanımlayan matematiksel bir gösterim oluşturunuz.
- b) $L(G_{S.5.5})$ dilini boş yığıtla tanıyan deterministik bir PDA tasarlamamız isteniyor. PDA'nın çalışma ilkesini (nasıl çalışacağını) belirleyip sözel olarak açıklayınız.
- c) PDA'nın durumlarını belirleyip tanımlayınız (durum sayısının gerektiği kadar ve olabildiğince az olması isteniyor).
- d) PDA'nın biçimsel tanımını veriniz.

S.5.6. Bağlamdan-bağımsız (CF) $L_{S.5.6}$ dili $\{a, b\}$ alfabesinde, eşit sayıda a ve b içeren dizgiler (strings) kümesi olarak tanımlanıyor ($\lambda \in L_{S.5.6}$). $L_{S.5.6}$ dilini boş yığıtla tanıyan deterministik bir PDA tasarlamamız isteniyor. Bunun için:

- a) PDA'nın çalışma ilkesini (nasıl çalışacağını) belirleyip sözel olarak açıklayınız.
- b) PDA'nın durumlarını belirleyip tanımlayınız (durum sayısının gerektiği kadar ve olabildiğince az olması isteniyor).
- c) PDA'nın biçimsel tanımını veriniz.

- d) $w_1 = aabbba$ ve $w_2 = baabba$ tümcelerinin PDA tarafından nasıl tanındığını ID dizileri ile gösteriniz.

S.5.7. Bağlamdan-bağımsız $L_{S.5.7}$ dili aşağıdaki gibi tanımlanıyor:

$$L_{S.5.7} = \{a^n(bc)^n \mid n \geq 1\}$$

$L_{S.5.7}$ dilini boş yığıtla tanıyan deterministik bir PDA tasarlamamız isteniyor. Bunun için:

- PDA'nın çalışma ilkesini belirleyip sözel olarak açıklayınız.
- Durum sayısının gerektiği kadar ve olabildiğince az olmasını gözeterek PDA'nın durumlarını belirleyip tanımlayınız.
- PDA'nın biçimsel tanımını veriniz.
- $w = abcabc$ tümcesinin PDA tarafından nasıl tanındığını ID dizisi ile gösteriniz.

S.5.8. Bağlamdan-bağımsız $L_{S.5.8}$ dili aşağıdaki gibi tanımlanıyor:

$$L_{S.5.8} = \{(ab)^n(bc)^n \mid n \geq 1\}$$

$L_{S.5.8}$ dilini boş yığıtla tanıyan deterministik bir PDA tasarlamamız isteniyor. Bunun için:

- PDA'nın çalışma ilkesini belirleyip sözel olarak açıklayınız.
- Durum sayısının gerektiği kadar ve olabildiğince az olmasını gözeterek PDA'nın durumlarını belirleyip tanımlayınız.
- PDA'nın biçimsel tanımını veriniz.

S.5.9. $L_{S.5.9} = \{a^n b^m c^k d^s \mid 1 \leq n \leq m \leq 2n, 1 \leq s \leq k \leq 2s\}$

- Yukarıda küme tanımı verilen bağlamdan-bağımsız $L_{S.5.9}$ dilini türeten bir dilbilgisi tanımlayınız.
- $L_{S.5.9}$ dilini boş yığıtla tanıyan bir PDA oluşturunuz.

S.5.10. Bağlamdan-bağımsız $L_{S.5.10}$ dili aşağıdaki gibi tanımlanıyor:

$$L_{S.5.10} = \{a^n b^m c^k d^n \mid n > 0, m > 0, k \geq m\}$$

- $L_{S.5.10}$ dilini türeten bir dilbilgisi oluşturunuz.
- $L_{S.5.10}$ dilini boş yığıtla tanıyan deterministik bir PDA tasarlamamız isteniyor. Bunun için PDA'nın çalışma ilkesini (nasıl çalışacağını) belirleyip sözel olarak açıklayınız, durumları belirleyip tanımlayınız ve PDA'nın biçimsel tanımını oluşturunuz.

S.5.11. $L_{S.5.11} = \{1^n 0^m 1^k 0^p \mid n \geq 2, m, k, p \geq 1, n+k = m+p\}$

Yukarıda küme tanımı verilen dili tanıyan bir PDA tasarlamamız isteniyor. Tasarlayacağınız PDA'nın mutlaka deterministik olması gerekmiyor. Boş yığıtla ya da uç durumla tanıyan bir PDA tasarlayabilirsiniz.

- PDA'nın çalışma ilkesini sözel olarak açıklayınız.
- PDA'nın biçimsel tanımını veriniz.
- $w = 110100$ için, anlık tanımlar dizisini oluşturarak, PDA'nın bu dizgiyi tanıyıp tanımadığını gösteriniz.

S.5.12. $L_{S.5.12} = \{1^n 0^k \mid n \geq 1, n \leq k < 2n\}$

Yukarıda küme tanımı verilen dili tanıyan bir PDA tasarlamamız isteniyor. Tasarlayacağınız PDA'nın mutlaka deterministik olması gerekmiyor. Boş yığıtla ya da uç durumla tanıyan bir PDA tasarlayabilirsiniz.

- PDA'nın çalışma ilkesini sözel olarak açıklayınız.
- PDA'nın biçimsel tanımını veriniz.
- $w = 1110000$ için, anlık tanımlar dizisini oluşturarak, PDA'nın bu dizgiyi tanıyıp tanımadığını gösteriniz.

S.5.13. Bağlamdan-bağımsız $L_{S.5.13}$ dili aşağıdaki gibi tanımlanıyor.

$$L_{S.5.13} = \{a^n b c^n \cup c^n b a^n \mid n > 0\}$$

- $L_{S.5.13}$ dilini türeten bir dilbilgisi oluşturunuz.
- $L_{S.5.13}$ dilini boş yığıtla tanıyan deterministik bir PDA tasarlamamız isteniyor. Bunun için PDA'nın çalışma ilkesini (nasıl çalışacağını) belirleyip sözel olarak açıklayınız, durumları belirleyip tanımlayınız ve PDA'nın biçimsel tanımını oluşturunuz.

S.5.14. $G_{S.5.14}$ dilbilgisi aşağıdaki gibi tanımlanıyor.

$$G_{S.5.14} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S\}$$

$$V_T = \{a, b\}$$

$$P: S \Rightarrow aSb \mid \lambda$$

- $G_{S.5.14}$ dilbilgisini GNF biçimine dönüştürünüz. Dönüştürme işlemi sezgisel olarak kolaylıkla yapabilirsiniz.
- $L(G_{S.5.14})$ dilini tanıyan PDA'nın tanımını, 5.2.1'deki 1. yöntemi kullanarak, sistematik biçimde oluşturunuz.

S.5.15. Aşağıda tanımı verilen PDA boş yığıtla tanıyan bir PDA'dır.

$$M_{S.5.15} = \langle Q, \Sigma, \Gamma, q_0, \delta, Z_0, \Phi \rangle$$

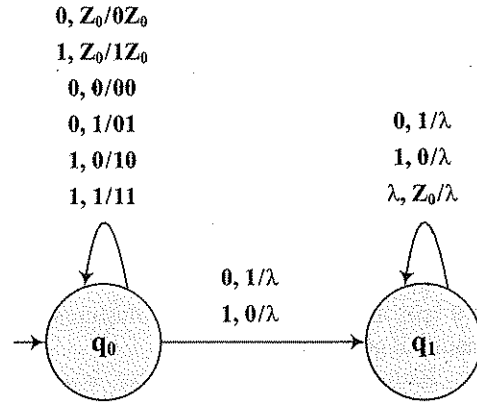
$$Q = \{q_0, q_1\}$$

$$\Sigma = \{0, 1\}$$

$$\Gamma = \{0, 1, Z_0\}$$

$$F = \Phi$$

δ :



- Sistematik yöntemle, $M_{S.5.15}$ 'in tanıdığı dili türeten bağlamdan-bağımsız bir dilbilgisi ($G_{S.5.15}$) bulunuz.
- Bulduğunuz dilbilgisi tarafından türetilen, dolayısıyla PDA tarafından tanınan dilin ($L_{S.5.15}$) küme tanımını oluşturunuz.

S.5.16. $L_{S.5.16} = \{a^n b^{2n} c^{2k} a^k \mid n, k \geq 1\}$

- $L_{S.5.16}$ dilini türeten bağlamdan bağımsız bir dilbilgisi tanımlayınız.
- $L_{S.5.16}$ dilini boş yığıtla tanıyan temel model bir PDA tasarlamamız isteniyor. Bunun için önce PDA'nın nasıl çalışacağını belirleyip sözel olarak açıklayınız. Sonra da PDA'nın biçimsel tanımını veriniz. Tanımda PDA'nın hareketlerini bir geçiş çizeneği ile gösteriniz.

S.5.17. $L_{S.5.17}$ dili aşağıdaki gibi tanımlanıyor:

$$L_{S.5.17} = \{a^n b^m \mid n > 0, n \leq m \leq 2n\} \cup \{b^k a^p \mid p > 0, p \leq k \leq 2p\}$$

- $L_{S.5.17}$ dilini türeten bağlamdan-bağımsız bir dilbilgisi oluşturunuz.
- $L_{S.5.17}$ dilini boş yığıtla tanıyan bir PDA'nın tanımını oluşturunuz. Bunun için PDA'nın nasıl çalışacağını belirleyip sözel olarak açıklayınız. Sonra PDA'nın biçimsel tanımını veriniz. Biçimsel tanımda PDA'nın hareketlerini bir geçiş çizeneği ile gösteriniz.

S.5.18. $\{a, b, c\}$ alfabesinde $L_{S.5.18}$ dili aşağıdaki gibi tanımlanıyor:

$$L_{S.5.18} = \{a^n b a^k c a^m \mid n, m \geq 1, k = n+m\}$$

- $L_{S.5.18}$ dilini türeten bağlamdan-bağımsız bir dilbilgisi oluşturunuz.
- $L_{S.5.18}$ dilini boş yığıtla tanıyan bir PDA'nın tanımını oluşturunuz. Bunun için PDA'nın nasıl çalışacağını belirleyip sözel olarak açıklayınız. Sonra PDA'nın biçimsel tanımını veriniz. Biçimsel tanımda PDA'nın hareketlerini bir geçiş çizeneği ile gösteriniz.

S.5.19. $L_{S.5.19}$ dili $\{a, b\}$ alfabesinde içindeki a 'ların sayısı ile b 'lerin sayısı arasındaki fark en çok iki (± 2) olan dizgiler kümesi olarak tanımlanıyor. Buna göre örneğin $abaaa$ ve $bbbababb$ dizgileri $L_{S.5.19}$ 'da yer almıyor. Buna karşılık $ababaa$, $bbbabaa$ ve $abaababb$ dizgileri $L_{S.5.19}$ 'da yer alıyor. $L_{S.5.19}$ dili λ 'yı içermiyor.

$L_{S.5.19}$ 'u boş yığıtla tanıyan bir temel model PDA oluşturmamız isteniyor. Bunun için önce PDA'nın nasıl çalışacağını sözel olarak açıklayınız. Sonra da PDA'nın biçimsel tanımını oluşturunuz. Biçimsel tanımda PDA'nın hareketlerini bir çizenekle gösteriniz.

Not: Oluşturacağınız PDA *deterministic* ya da *non-deterministic* olabilir.

S.5.20. $L_{S.5.20}$ dili $\{a, b\}$ alfabesinde, ilk simgesi a olan ve tüm örneklerinde $N(a) \geq N(b)$ eşitsizliği sağlanan dizgiler kümesi olarak tanımlanıyor.

$L_{S.5.20}$ 'yi hem uç durumla hem de boş yığıtla tanıyan ve durum sayısı olabildiğince az olan bir PDA tasarlamamız isteniyor.

$$PDA = \langle Q, \Sigma, \Gamma, \delta, q_0, Z_0, F \rangle$$

$$Q = \{q_0, \dots\}$$

$$\Sigma = \{a, b\}$$

$$\Gamma = \{a, b, Z_0\}$$

$$F = ?$$

Q ve F'yi belirleyip δ hareket işlevini bir çizenekle göstererek tasarımı tamamlayınız.

Notlar : $N(a)$: a'ların sayısını göstermektedir.
Dizginin tamamı da kendisinin bir önekidir.

S.5.21. Bağlamdan-bağımsız $L_{S.5.21}$ dili, $\{a, b\}$ alfabesinde içindeki a'ların sayısı b'lerin sayısından büyük olan dizgiler kümesi olarak tanımlanıyor. $L_{S.5.21}$ dili λ 'yı içermiyor:

$$N(a) > N(b)$$

$$N(a) + N(b) > 0$$

$$N(a) = a'ların sayısı.$$

$L_{S.5.21}$ dilini hem uç durum hem de boş yığıtla tanıyan bir PDA tanımlamanız isteniyor.

$$PDA = \langle Q, \Sigma, \Gamma, \delta, q_0, Z_0, F \rangle$$

$$Q = \{q_0, \dots\}$$

$$\Sigma = \{a, b\}$$

$$\Gamma = \{Z_0, A, B\}$$

$$q_0 : \text{Başlangıç durumu.}$$

$$F = \{q_f\}$$

PDA'nın çalışma ilkesini sözle açıklayınız.

Durumlar kümesini tamamlayıp geçiş çizeneğini anlaşılır biçimde oluşturunuz.

Not : Dizgi tanındığında hem yığıtın boşalması hem de PDA'nın uç durumda (q_f) bulunması isteniyor.

S.5.22. $L_{S.5.22}$ dili $\{a, b\}$ alfabesinde, uzunluğu en az 2 olan ve içindeki a'ların sayısı b'lerin sayısından 2 fazla olan dizgiler kümesi olarak tanımlanıyor.

$L_{S.5.22}$ 'yi hem uç durumla hem de boş yığıtla tanıyan temel model bir PDA tasarlamamız isteniyor.

$$PDA = \langle Q, \Sigma, \Gamma, \delta, q_0, Z_0, F \rangle$$

$$Q = \{q_0, \dots\}$$

$$\Sigma = \{a, b\}$$

$$\Gamma = \{a, b, Z_0\}$$

$$F = ?$$

Q ve F'yi belirleyip δ hareket işlevini bir çizenekle göstererek tasarımı tamamlayınız.

Not : Dizgi tanındığında hem yığıtın boşalması hem de PDA'nın bir uç durumda (q_f) bulunması isteniyor.

S.5.23. Bağlamdan-bağımsız $L_{S.5.23}$ dili, $\{a, b, c\}$ alfabesinde aşağıdaki gibi tanımlanıyor:

$$L_{S.5.23} = \{a^n (bc)^n a^k c^k \mid n \geq 1, k \geq 1\}$$

$L_{S.5.23}$ dilini hem uç durum hem de boş yığıtla tanıyan bir PDA tanımlamanız isteniyor.

$$PDA = \langle Q, \Sigma, \Gamma, \delta, q_0, Z_0, F \rangle$$

$$Q = \{q_0, \dots\}$$

$$\Sigma = \{a, b, c\}$$

$$\Gamma = \{Z_0, X\}$$

$$q_0 : \text{Başlangıç durumu.}$$

$$F = \{q_f\}$$

PDA'nın çalışma ilkesini sözle açıklayınız.

Durumlar kümesini tamamlayıp geçiş çizeneğini anlaşılır biçimde oluşturunuz.

Not : Dizgi tanındığında hem yığıtın boşalması hem de PDA'nın uç durumda (q_f) bulunması isteniyor.

S.5.24. $L_{S.5.24}$ dili $\{a, b\}$ alfabesinde aşağıdaki gibi tanımlanıyor:

$$L_{S.5.24} = \{a^n b^{2k} a^k b^{2n} \mid n \geq 1, k \geq 1\}$$

$L_{S.5.24}$ dilini tanıyan PDA'nın aşağıdaki tanımını tamamlamanız isteniyor.

$$PDA = \langle Q, \Sigma, \Gamma, \delta, q_0, Z_0, F \rangle$$

$$Q = \{q_0, \dots\}$$

$$\Sigma = \{a, b\}$$

$$\Gamma = \{X, Y, Z_0\}$$

$$F = ?$$

Q ve F'yi belirleyip δ hareket işlevini bir çizenekle göstererek tasarımı tamamlayınız.

Not : Dizgi tanındığında hem yığıtın boşalması hem de PDA'nın bir uç durumda (q_f) bulunması isteniyor.

S.5.25. Bağlamdan-bağımsız $L_{S.5.25}$ dili, $\{1, 2, 3, 4\}$ alfabesinde aşağıdaki gibi tanımlanıyor:

$$L_{S.5.25} = \{(1^n 2 3^n)^k 4^k \mid n \geq 1, k \geq 1\}$$

Örnek dizgi: $w = 112331112333123444$

- a) $L_{S.5.25}$ dilini türeten bir dilbilgisi ($G_{S.5.25}$) tanımlayınız. Tanımladığınız dilbilgisinde sözdizim değişkenlerini $S, A, B, \dots$ vb. olarak adlandırınız.
- b) $L_{S.5.25}$ dilini hem uç durum hem de boş yığıtla tanıyan bir PDA tanımlamanız isteniyor.

$$PDA = \langle Q, \Sigma, \Gamma, \delta, q_0, Z_0, F \rangle$$

$$Q = \{q_0, \dots\}$$

$$\Sigma = \{1, 2, 3, 4\}$$

$$\Gamma = \{Z_0, X, Y\}$$

q_0 : Başlangıç durumu

$$F = \{q_f\}$$

PDA'nın çalışma ilkesini sözle açıklayınız.

Durumlar kümesini tamamlayıp geçiş çizeneğini anlaşılır biçimde oluşturunuz.

Not : Dizgi tanındığında hem yığıtın boşalması hem de PDA'nın bir uç Durumda (q_f) bulunması isteniyor.

Bölüm 6

Turing Makineleri

Turing makineleri, genel amaçlı bilgisayarlar için matematiksel bir modeldir. Herhangi bir makineyle yapılabilecek işlemleri Turing makinesi ile modellemek mümkündür. Başka bir deyişle, bir işlemin herhangi bir makineyle yapılabilmesi için, bu işlemin Turing makineleriyle yapılabilir (modellenebilir) bir işlem olması gerekir. Bu özellikleri dolayısıyla Turing makinesi modeli, genel amaçlı sayısal bilgisayarlar için önemli bir modeldir.

Turing makinelerinin kullanım alanları aşağıdaki gibi sınıflandırılabilir:

1. **Dil tanıyıcı.** Turing makineleri kısıtlamasız (*unrestricted*) ya da özyineli sayılabilir (*r.e. : recursively enumerable*) dilleri tanımak için kullanılabilir. Buna göre Turing makinesi, verilen bir tümcenin dilin bir tümcesi olup olmadığını bulabilir.
2. **Hesaplayıcı.** Turing makineleri, kısmi özyineli tamsayı fonksiyonların (*partially recursive integer functions*) hesaplanmasında kullanılabilir.
3. **Dil üreticisi.** Turing makineleri ile, *r.e.* dillerin tümcelerini ardarda üretmek mümkündür. Eğer biçimsel bir dil bir Turing makinesi tarafından üretilebiliyorsa bu dil *r.e.* bir dildir. Diğer taraftan her *r.e.* dil için, dilin tüm tümcelerini ardarda üreten bir Turing makinesi vardır.

Bu kitapta Turing makinelerinin dil tanıyıcı ve hesaplayıcı olarak kullanımı üzerinde durulacak ve *r.e.* dilleri tanıyan ve basit hesaplamaları gerçekleştiren Turing makinesi örnekleri verilecektir.

6.1. Turing Makinelerinin Temel Modeli

Biçimsel olarak, Turing makinelerinin temel modeli bir yedili olarak tanımlanır.

$$M = \langle Q, \Sigma, \Gamma, \delta, q_0, B, F \rangle$$

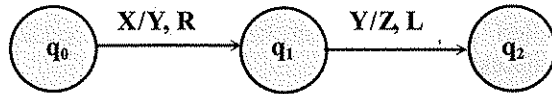
Q : Sonlu sayıda durum içeren Durumlar Kümesi

- Σ : Sonlu sayıda giriş simgesinden oluşan Giriş Alfabeti
- Γ : Sonlu sayıda simge içeren Şerit Alfabeti. Şerit alfabeti, giriş alfabetinin tüm simgelerini içeren bir kümedir: $\Gamma \supseteq \Sigma$
- q_0 : Başlangıç durumu ($q_0 \in Q$)
Başlangıç durumu durumlar kümesinin bir elemanı olduğuna göre Q boş olmayan bir kümedir.
- B : Şerit alfabetindeki simgelerden *blank* olarak adlandırılan özel bir simge. B şerit alfabetinde yer alan ancak giriş alfabetinde yer almayan bir simgedir: $B \in \Gamma$, $B \notin \Sigma$
- F : Uç durumlar kümesi
Durumlar kümesinin bir altkümesidir: $F \subseteq Q$
- δ : Geçiş ya da hareket işlevi (*transition or move function*)
Turing makinelerinin temel modeli deterministik bir modeldir. Bu modelde, hareket işlevi

$[Q \times \Gamma]$ 'dan $[Q \times \Gamma \times \{L, R\}]$ 'ye

bir eşleme oluşturur. Bu tanımda yer alan L ve R simgeleri, hareketin sonunda okuma kafasının bir sağdaki (R) ya da bir soldaki (L) hücreye geçeceğini gösterir. Okuma kafasının hareketiyle ilgili seçenekler arasına "sağa/sola hareket etmeyip aynı hücre üzerinde kalma (S)" seçeneğini eklemek ve iki elemanlı $\{L, R\}$ kümesi yerine üç seçenekli $\{L, R, S\}$ kümesini kullanmak da mümkündür. Ancak bu kitapta çoğunlukla $\{L, R\}$ kümesi kullanılacak ve Turing makineleri oluşturulurken, okuma kafasının her harekette bir sağ ya da bir sol hücreye geçmesi sağlanacaktır. Buna göre Turing makinesinin hareketleri aşağıdaki örneklerde görüldüğü gibi tanımlanacak, istenirse hareketler bir çizimle de gösterilecektir.

$$\begin{aligned} \delta(q_0, X) &= (q_1, Y, R) & q_0, q_1, q_2 &\in Q \\ \delta(q_1, Y) &= (q_2, Z, L) & X, Y, Z &\in \Gamma \end{aligned}$$



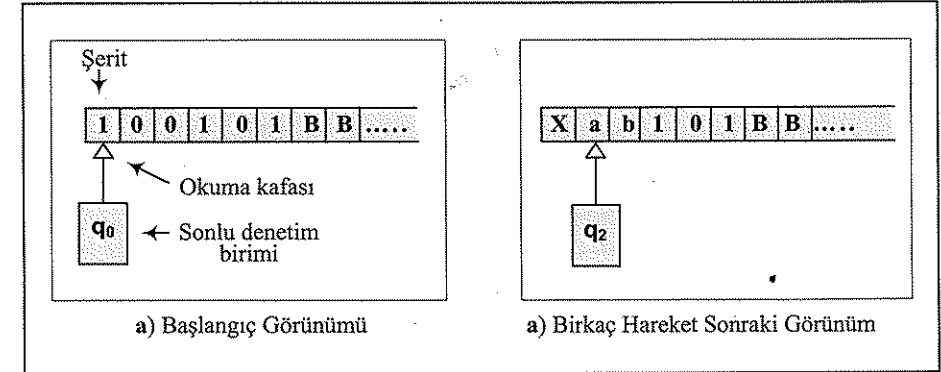
6.1.1. Soyut Makine Görünümü

Tanımlandığı biçimiyle Turing makinesi matematiksel bir modeldir. Ancak bundan önceki modellerde olduğu gibi, Turing makinesini de soyut bir makine olarak düşünmek mümkündür. Turing makinesinin nasıl çalıştığını daha iyi anlayabilmek için, aşağıdaki bileşenlerden oluşan soyut bir makine modeli kullanılmaktadır (Çizim 6.1):

- Hücrelerden oluşan ve her hücresinde bir giriş simgesi bulunan bir miknatıslı şerit. Hem okuma hem de yazma işlemleri yapılabilen şeridin sağ ucu sonsuzdur.
- Bir sonlu denetim birimi (SDB)
- Bir okuma kafası

Turing makinesinin Çizim 6.1'de görülen soyut makine modeli aşağıda açıklanan biçimde çalışır:

- Başlangıçta şerit üzerinde giriş simgelerinden oluşan bir dizgi (w) kayıtlıdır ve okuma kafası bu dizginin ilk (en soldaki) simgesi üzerindedir.
- Makinenin her hareketinde aşağıdaki işlemler yapılır:
 - Şeritten bir simge (okuma kafasının üzerinde bulunduğu simge) okunur.
 - Okunan simgenin yerine yeni bir simge yazılır.
 - Okuma kafası bir sağdaki (R) ya da bir soldaki (L) hücreye geçer.
 - Sonlu denetim birimi yeni bir duruma geçer.



Çizim 6.1. Turing Makinesinin Soyut Makine Görünümü

6.1.2. Anlık Tanımlar (*Instantaneous Descriptions*)

Turing makinesi, şerit üzerinde her iki yönde hareket ederek hem okuma hem de yazma yapabilen bir model olduğundan, makinenin sonraki davranışlarını kestirebilmek için belirli bir andaki üç bilginin bilinmesi gerekir:

- Şerit üzerinde, okuma kafasının solunda bulunan dizgi (α_1)
- Sonlu denetim biriminin durumu (q)

- Şerit üzerinde okuma kafasının sağındaki dizgi (α_2). Okuma kafasının üzerinde bulunduğu simge α_2 'ye dahildir.

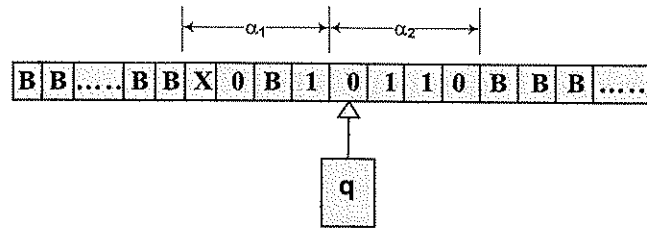
Bu üç bilginin belirli bir andaki değerlerinden oluşan üçlüye, Turing makinesinin anlık tanımı denir:

Anlık tanım (ID) = (α_1 , q , α_2)

q : makinenin durumu

α_1 : okuma kafasının solundaki dizgi

α_2 : okuma kafasının sağındaki dizgi (okuma kafası α_2 'nin en solundaki simge üzerinde bulunur)



B (blank) simgesi boşlukları gösterdiği için, şeridin sağ ve sol tarafındaki, salt **B** simgelerinden oluşan kesimler α_1 ve α_2 'ye dahil edilmez. Buna göre α_1 'in en solundaki, α_2 'nin de en sağındaki simge **B** olamaz; ancak α_1 ve α_2 'nin içinde **B** simgesi bulunabilir.

6.1.3. Turing Makinesinin Tanıdığı Dil

Dil tanıyıcı olarak kullanılan bir Turing makinesinin tanıdığı dil biçimsel olarak aşağıdaki gibi tanımlanabilir:

$$T(M) = \{w \mid w \in V_T^*, (q_0, w) \vdash^* (\alpha_1, p, \alpha_2), \alpha_1, \alpha_2 \in \Gamma^*, p \in F\}$$

Yukarıdaki tanıma göre, giriş simgelerinden oluşan w dizgisinin Turing makinesi tarafından tanınabilmesi için, makinenin bir uç durumda durması gerekir. Başka bir deyişle, bitiş konfigürasyonunu gösteren anlık tanımda (α_1 , p , α_2), önemli olan p 'nin bir uç durum olmasıdır. Şeridin görünümü (α_1 ve α_2) tanımada etkili değildir.

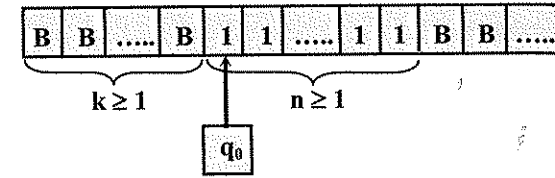
6.2. Turing Makinesi Örnekleri

Örnek 6.1. n sıfırdan büyük (pozitif) bir tamsayı olmak üzere

$$f(n) = 2n$$

değerini hesaplayan Turing makinesini tasarlayalım.

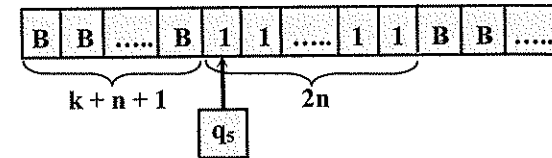
Başlangıç konfigürasyonu:



Hesaplama yöntemi:

- 1'ler öbeğinin sağdaki ilk hücreye bir işaretleyici (örneğin ϕ) yazılır.
- 1'ler öbeğinin en solundaki 1, yerine **B** yazılarak silinir. Silinen 1'e karşılık, sağdaki ilk iki **B**'nin yerine 1 yazılır.
- 2'deki işlemler, ϕ 'nin solundaki 1'ler bitinceye kadar tekrarlanır. ϕ 'nin solundaki 1'ler bittiğinde, ϕ 'nin yerine **B** yazılır ve hesaplama biter.

Bitiş konfigürasyonu:



Makinenin biçimsel tanımı:

$$M_{6.1} = \langle Q, \Sigma, \Gamma, \delta, q_0, B, F \rangle$$

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$$

$$\Sigma = \{1\}$$

$$\Gamma = \{B, 1, \phi\}$$

q_0 : Başlangıç durumu.

$F = \emptyset$ (makine bir tanıyıcı değil, bir hesaplayıcı olduğundan, uç durum anlamsızdır.)

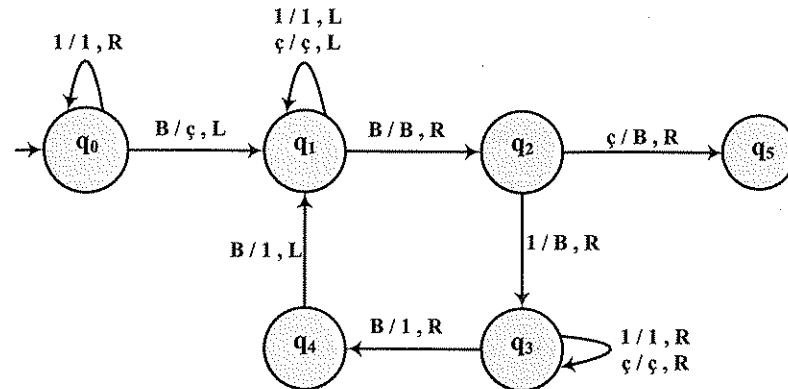
Durumların anlamları:

- q_0 : Başlangıç durumu. 1'ler öbeğinin sağına işaretleyicinin {ç} konulduğu durum bu durumdur.
- q_1 : Solu ilerleyip ilk B'nin bulunduğu durum.
- q_2 : 1'ler öbeğinin en solundaki 1'in silindiği durum.
- q_3 : Sağa ilerleyip ilk B'nin bulunduğu ve yerine 1 yazıldığı durum.
- q_4 : İkinci B'nin yerine 1 yazıldığı durum.
- q_5 : Son durum. Hesaplama tamamlandığında makine bu duruma gelir ve durur.

Hareket çizelgesi:

	1	ç	B
q_0	$(q_0, 1, R)$	-	$(q_1, ç, L)$
q_1	$(q_1, 1, L)$	$(q_1, ç, L)$	(q_2, B, R)
q_2	(q_3, B, R)	(q_5, B, R)	-
q_3	$(q_3, 1, R)$	$(q_3, ç, R)$	$(q_4, 1, R)$
q_4	-	-	$(q_1, 1, L)$
q_5	-	-	-

Hareket çizeneği:

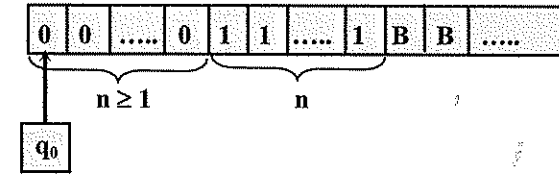


Örnek 6.2. $L_{6.2}$ dili aşağıdaki gibi tanımlanıyor.

$$L_{6.2} = \{0^n 1^n \mid n \geq 1\}$$

Bu dil bağlamdan-bağımsız (CF) bir dildir. Ancak Turing makineleri tarafından tanınan diller sınıfı, bağlamdan-bağımsız dilleri de içerdiği için, $L_{6.2}$ dilini tanıyan bir Turing makinesi tanımlamak mümkündür.

Başlangıç konfigürasyonu:



Çalışma yöntemi:

- En soldaki 0'ın yerine X yazılır.
- En soldaki 1'in yerine Y yazılır. Eğer yerine X yazılan bir 0'a karşılık 1 bulunamazsa, makine uç olmayan bir durumda durur ve dizgi tanınmaz.
- a ve b'deki işlemler 0'lar bitinceye kadar tekrarlanır.
- 0'lar bittiğinde, 1 kalmadığı denetlenir. Eğer 1 kalmamışsa makine uç durumda durur ve dizgi tanınır; 1 kalmışsa makine uç olmayan bir durumda durur ve dizgi tanınmaz.

Makinenin biçimsel tanımı:

$$M_{6.2} = \langle Q, \Sigma, \Gamma, \delta, q_0, B, F \rangle$$

$$Q = \{q_0, q_1, q_2, q_3, q_4\}$$

$$\Sigma = \{0, 1\}$$

$$\Gamma = \{B, 0, 1, X, Y\}$$

q_0 : Başlangıç durumu

$$F = \{q_4\}$$

Durumların anlamları:

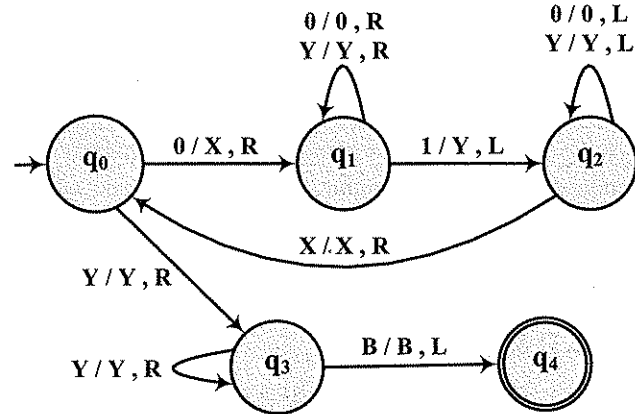
- q_0 : En soldaki 0'ın X yapıldığı durum; aynı zamanda başlangıç durumu.
- q_1 : Sağa ilerleyip en soldaki 1'in Y yapıldığı durum.

- q_2 : Sola ilerleyip ilk **X**'de sağa dönülen durum.
 q_3 : Sağa ilerleyip başka **1** kalmadığının denetlendiği durum.
 q_4 : Uç durum. Makine q_4 'de durduğunda dizgi tanınır.

Hareket çizelgesi:

	0	1	X	Y	B
q_0	(q_1, X, R)	-	-	(q_3, Y, R)	-
q_1	($q_1, 0, R$)	(q_2, Y, L)	-	(q_1, Y, R)	-
q_2	($q_2, 0, L$)	-	(q_0, X, R)	(q_2, Y, L)	-
q_3	-	-	-	(q_3, Y, R)	(q_4, B, L)
q_4	-	-	-	-	-

Hareket çizeneği:

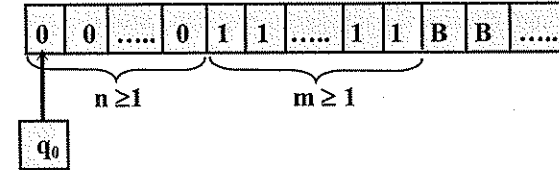


Örnek 6.3. n ve m sıfırdan büyük (pozitif) birer tamsayı olmak üzere

$$f(n, m) = n \times m$$

değerini hesaplayan Turing makinesini tasarlayalım.

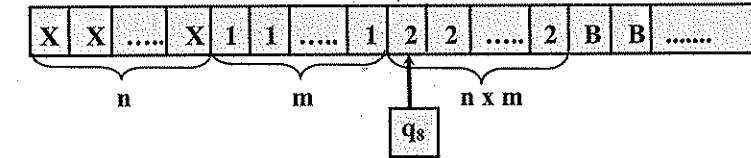
Başlangıç konfigürasyonu:



Hesaplama yöntemi:

- Çarpma işlemi, m 'nin n kere toplanması ile gerçekleştirilecektir.
- Bunun için, n 'yi oluşturan her **0** için, **1**'ler öbeği kopyalanacaktır.
- 1**'ler öbeği n kere kopyalandığında $n \times m$ değeri elde edilmiş olacaktır.

Bitiş konfigürasyonu:



Makinenin biçimsel tanımı:

$$M_{6.3} = \langle Q, \Sigma, \Gamma, \delta, q_0, B, F \rangle$$

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_8\}$$

$$\Sigma = \{0, 1\}$$

$$\Gamma = \{B, 0, 1, 2, X, Y\}$$

q_0 : Başlangıç durumu

$F = \emptyset$ (Makine bir tanıyıcı değil, bir hesaplayıcı olduğundan, uç durum anlamsızdır).

Durumların anlamları:

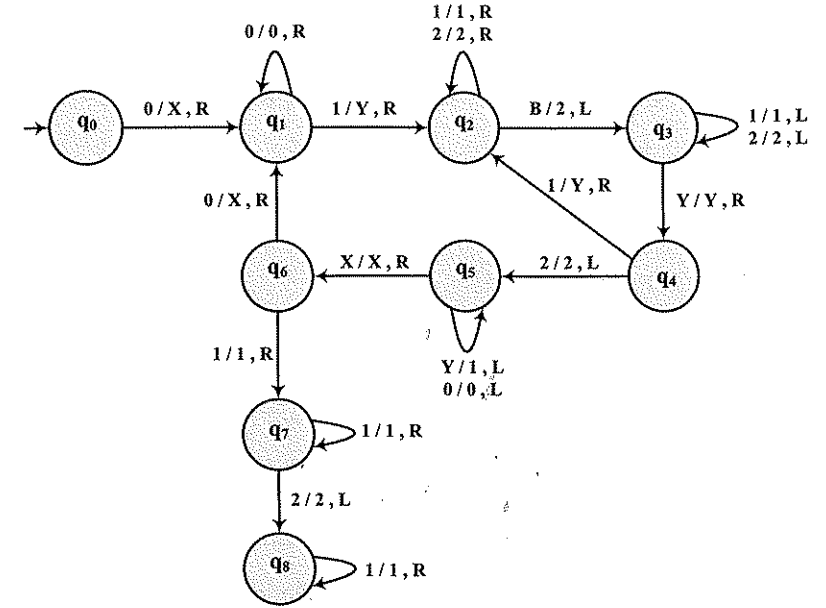
- q_0 : Başlangıç durumu. Dış döngünün birinci adımını başlatır.
 q_1 : İç döngünün birinci adımını başlatır.
 q_2 : Sağa ilerleyerek ilk **B**'nin **2** yapıldığı durum.

- q_3 : İç döngüde geriye dönüşü sağlayan durum.
 q_4 : İç döngünün sonraki adımlarını başlatan; iç döngü bitti ise dış döngünün geriye dönüşünü başlatan durum.
 q_5 : Dış döngünün geriye dönüşünü sağlayan, bu arada 1'ler öbeğine eski görünümünü kazandıran durum.
 q_6 : Dış döngünün sonraki adımlarını başlatan durum.
 q_7, q_8 : Bitiş konfigürasyonunda, okuma kafasının 2'ler öbeğinin başında olmasını sağlayan durumlar.

Hareket çizelgesi:

	0	1	2	X	Y	B
q_0	(q_1, X, R)	-	-	-	-	-
q_1	$(q_1, 0, R)$	(q_2, Y, R)	-	-	-	-
q_2	-	$(q_2, 1, R)$	$(q_2, 2, R)$	-	-	$(q_3, 2, L)$
q_3	-	$(q_3, 1, L)$	$(q_3, 2, L)$	-	(q_4, Y, R)	-
q_4	-	(q_2, Y, R)	$(q_5, 2, L)$	-	-	-
q_5	$(q_5, 0, L)$	-	-	(q_6, X, R)	$(q_5, 1, L)$	-
q_6	(q_1, X, R)	$(q_7, 1, R)$	-	-	-	-
q_7	-	$(q_7, 1, R)$	$(q_8, 2, L)$	-	-	-
q_8	-	$(q_8, 1, R)$	-	-	-	-

Hareket çizeneği:

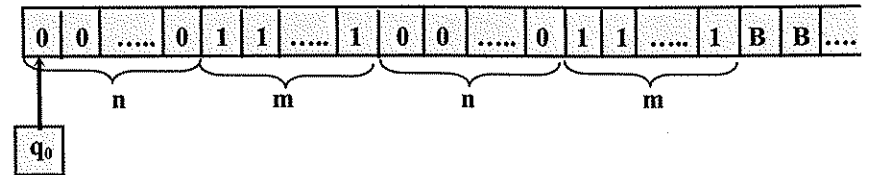


Örnek 6.4. $L_{6,4}$ dili aşağıdaki gibi tanımlanıyor.

$$L_{6,4} = \{0^n 1^m 0^n 1^m \mid n \geq 1, m \geq 1\}$$

Bu dil bağlamdan-bağımsız (CF) bir dil değildir. Bu dili ancak bir Turing makinesiyle tanımak mümkündür. $L_{6,4}$ dilini tanıyan bir Turing makinesi aşağıdaki gibi tanımlanabilir.

Başlangıç konfigürasyonu:



Çalışma yöntemi:

- Önce 0 öbeklerindeki simgeler karşılıklı olarak yerlerine X ve Z yazılarak silinir.
- Sonra da 1 öbeklerindeki simgeler karşılıklı olarak yerlerine Y ve W yazılarak silinir.

c) Sonunda hiç 0 ve 1 kalmadıysa dizgi tanınır.

Makinenin biçimsel tanımı:

$$M_{6.4} = \langle Q, \Sigma, \Gamma, \delta, q_0, B, F \rangle$$

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_8, q_9, q_{10}\}$$

$$\Sigma = \{0, 1\}$$

$$\Gamma = \{B, 0, 1, X, Y, Z, W\}$$

q_0 : Başlangıç durumu

$$F = \{q_{10}\}$$

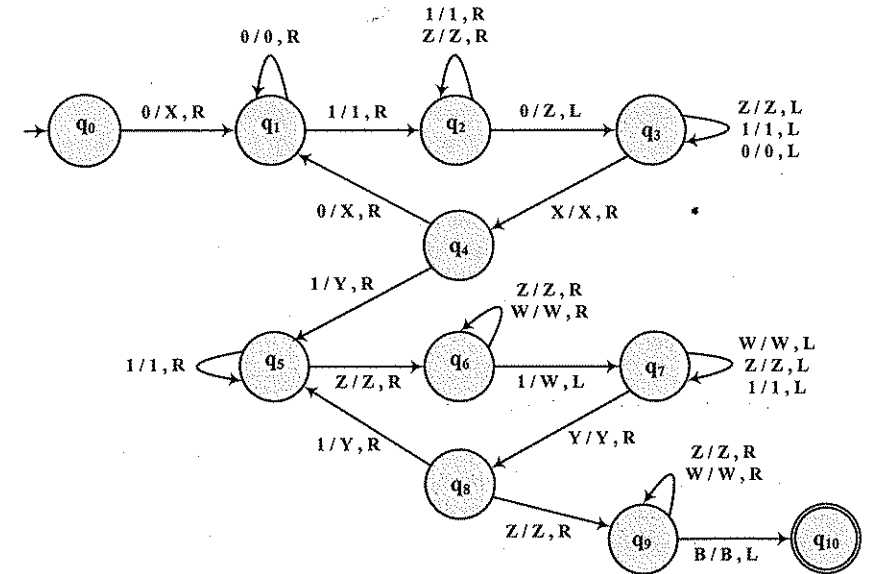
Durumların anlamları:

- q_0 : En soldaki 0'ın X yapıldığı durum; aynı zamanda başlangıç durumu.
- q_1 : Sağa ilerleyerek soldaki 1'ler öbeğinin bulunduğu durum.
- q_2 : Sağa ilerleyerek ikinci 0'lar öbeğindeki ilk 0'ın bulunup Z yapıldığı durum.
- q_3 : X'i buluncaya kadar sola ilerlenen durum.
- q_4 : Soldaki ilk 0'ın X yapıldığı; 0'lar bittiğinde ise ilk 1'in Y yapıldığı durum.
- q_5 : Sağa ilerleyerek soldaki Z'ler öbeğinin bulunduğu durum.
- q_6 : Sağa ilerleyerek ikinci 1'ler öbeğindeki ilk 1'in bulunup W yapıldığı durum.
- q_7 : Y'i buluncaya kadar sola ilerlenen durum.
- q_8 : Soldaki ilk 1'in Y yapıldığı durum.
- q_9 : Sağa ilerlenerek başka 0 ve 1 kalmadığının denetlendiği durum.
- q_{10} : Uç durum

Hareket çizelgesi:

	0	1	X	Y	Z	W	B
q_0	(q_1, X, R)	-	-	-	-	-	-
q_1	($q_1, 0, R$)	($q_2, 1, R$)	-	-	-	-	-
q_2	(q_3, Z, L)	($q_2, 1, R$)	-	-	(q_2, Z, R)	-	-
q_3	($q_3, 0, L$)	($q_3, 1, L$)	(q_4, X, R)	-	(q_3, Z, L)	-	-
q_4	(q_1, X, R)	(q_5, Y, R)	-	-	-	-	-
q_5	-	($q_5, 1, R$)	-	-	(q_6, Z, R)	-	-
q_6	-	(q_7, W, L)	-	-	(q_6, Z, R)	(q_6, W, R)	-
q_7	-	($q_7, 1, L$)	-	(q_8, Y, R)	(q_7, Z, L)	(q_7, W, L)	-
q_8	-	(q_5, Y, R)	-	-	(q_9, Z, R)	-	-
q_9	-	-	-	-	(q_9, Z, R)	(q_9, W, R)	(q_{10}, B, L)
q_{10}	-	-	-	-	-	-	-

Hareket çizeneği:



6.3. Turing Makinesi Modelinde Değişiklikler

Turing makinelerinin temel modelinde yalnız bir ucu sonsuz, tek izli bir şerit ile tek okuma kafası kullanılmakta ve her harekette yalnız bir simge okunup-yazılabilmektedir (bkz 6.1). Şeridin iki yönde sonsuz olması, şerit sayısı, iz sayısı ve okuma kafası sayısı ile ilgili olarak Turing makinesi modelinde bir dizi değişiklik yapılabilir. Bu değişiklikler modelin kullanımında esneklik ve kolaylıklar sağlar. Ancak modelin gücünde hiçbir değişiklik yapmaz. Başka bir deyişle, hangi değişiklik model kullanılırsa kullanılsın, yapılan hesaplamayı temel model bir Turing makinesi ile yapmak ya da tanınan dili temel model bir Turing makinesi ile tanımak mümkündür. Turing makinesi modelinde yapılabilecek değişikliklerden başlıcaları aşağıdaki gibi sıralanabilir.

1. İki yönlü sonsuz şerit kullanan Turing makinesi

Temel modelde sol ucu sınırlı, sağ ucu sınırsız olan şeridin bu modelde her iki ucu da sonsuzdur. Başlangıçta şerit üzerinde belirli bir giriş dizgisi kayıtlıdır. Şerit üzerinde giriş dizgisinin sol ve sağ taraflarının sonsuz sayıda B ile dolu olduğu varsayılır.

2. Çok şeritli Turing makinesi

Bu modelde makinenin bir sonlu denetim birimi, ancak n adet şeridi ve n adet okuma kafası vardır. Başlangıçta giriş dizgisi şeritlerden birinde kayıtlıdır. Diğer şeritler ise boştur. Her harekette, Turing makinesi her şeritten bir simge okur, her şeride bir simge yazar ve okuma kafalarının her biri bir sağdaki ya da bir soldaki hücreye geçer. Şeritler üzerindeki yazma ve sağa/sola geçme hareketleri birbirinden bağımsızdır. Ancak sonlu denetim birimi tek olduğu için durum geçişleri ortaktır.

3. Çok izli Turing makinesi

Bu modelde makinenin bir sonlu denetim birimi, n izli bir şeridi, n izli şerit üzerinde okuma ve yazma yapabilen bir de okuma kafası vardır. Her harekette makine her izden bir tane olmak üzere n simge okur, her izde bir tane olmak üzere n simge yazar; okuma kafası bir sağa ya da bir sola, sonlu denetim birimi de yeni bir duruma geçer.

4. Birden çok okuma kafası bulunan Turing makinesi

Bu modelde makinenin bir sonlu denetim birimi, tek izli bir şeridi, birden çok da okuma kafası bulunur. Belirli bir anda okuma kafalarının her biri şeridin belli bir hücresi üzerindedir. Her harekette okuma kafalarının her biri bulunduğu hücredeki simgeyi okur, yerine bir simge yazar ve bir sağdaki/soldaki hücreye geçer.

5. Çok boyutlu Turing makinesi

Temel modelde tek boyutlu olan okuma şeridi bu modelde çok boyutludur. Örneğin k boyutlu olan şerit 2k yönde sonsuzdur. Her harekette okuma kafası

üzerinde bulunduğu hücredeki simgeyi okuyup, yerine yeni bir simge yazdıktan sonra 2k yönden birinde ilerler.

6. Off-line Turing makinesi

Çok şeritli modelde olduğu gibi bu modelde de makinenin birden çok şeridi vardır. Ancak şeritlerden biri giriş şerididir ve bu şerit üzerinde yalnız okuma yapılabilir. Başlangıçta giriş şeridi üzerinde bir dizgi kayıtlıdır, diğer şeritler ise boştur. Makine her hareketinde her şeritten bir simge okur, giriş şeridi dışındaki şeritlere birer simge yazar ve her şerit üzerindeki okuma kafası bir sağ ya da sol hücreye geçer.

7. Deterministik olmayan Turing makinesi

Turing makinesinin temel modeli deterministik bir modeldir. Deterministik modelde her anlık tanıma en çok bir hareket eşlenir. Deterministik olmayan modelde ise her anlık tanıma sıfır, bir ya da birden çok hareket eşlenebilir.

6.4. Chomsky Sıradüzeni

Sonlu özdevinir, yığıtlı özdevinir (PDA) ve Turing makinesi adlarıyla sunulan üç modelden birincisi sonlu bellekli, diğer ikisi ise sonsuz bellekli modellerdir. Sonlu özdevinir modelinde bilgi saklamak için kullanılan açık bir birim yoktur. Modelin saklama kapasitesi durumlarla sınırlıdır. Durum sayısı sınırlı olduğu için de model sonlu bellekli bir modeldir. Yığıtlı özdevinir ve Turing makinesi modellerinde ise bilgi saklamak için kullanılan sonsuz kapasiteli birer birim (yığıt ve şerit) vardır. Sonsuz bellekli modellerin işlem gücünün sonlu bellekli modelden daha büyük olduğu açıktır. Sonsuz bellekli modellerde ise, Turing makinesinin gücü yığıtlı özdevinirden daha büyüktür. Yığıtlı özdevinir ve Turing makinesi modellerinin işlem gücü açısından farkı bellek kapasitesinden değil, modelin yapısından kaynaklanmaktadır. Turing makinesi modeli, yığıtlı özdevinir modeline göre daha güçlü bir modeldir.

Biçimsel diller ve biçimsel dillere karşı gelen makine modellerini aşağıdaki gibi sıralayabiliriz :

Biçimsel Dil Sınıfı	Makine Modeli
Tür-3 ya da düzgün dil	Sonlu Özdevinir (FA)
Tür-2 ya da bağlamdan-bağımsız dil	Yığıtlı Özdevinir (PDA)
Tür-1 ya da bağlama-bağımlı dil	Doğrusal-Sınırlı Özdevinir (LBA)
Tür-0 ya da kısıtlamasız dil	Turing makinesi (TM)

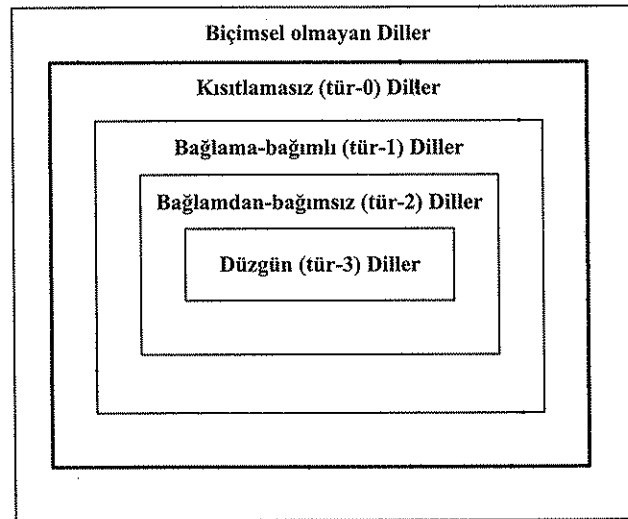
Yukarıdaki çizelgede yer alan biçimsel dil sınıflarının tümü ile makine sınıflarının üçü bu kitapta incelenmektedir. Doğrusal-sınırlı özdevinirler (*linear-bounded automata* – LBA) olarak adlandırılan makine modeli ise, diğer üç model kadar yaygın kullanılan bir model olmadığı için bu kitap kapsamına alınmamıştır. Çok kısa olarak LBA modeli, Turing makinesi modelinin şerit kapasitesi sınırlı bir biçimdir ve şerit kapasitesindeki sınırlama giriş dizgisi tarafından belirlenir.

Biçimsel dil ve makine sınıfları arasındaki ilişki ise aşağıdaki gibi özetlenebilir :

1. Sonlu özdevinirler düzgün dilleri tanıyan makinelerdir.
2. Yığıtlı özdevinirler bağlamdan-bağımsız dilleri tanıyan makinelerdir.
3. Doğrusal-sınırlı özdevinirler bağlama-bağımlı dilleri tanıyan makinelerdir.
4. Turing makineleri ise kısıtlamasız dilleri tanıyan makinelerdir.

Yukarıda sıralanan dil sınıfları birbirinden bağımsız değildir. Biçimsel dil sınıfları arasındaki sıradüzen Chomsky sıradüzeni (*Chomsky hierarchy*) olarak bilinir ve kısaca aşağıdaki gibi ifade edilebilir (Çizim 6.2):

1. Her bağlama-bağımlı dil aynı zamanda bir kısıtlamasız dildir. Başka bir deyişle bağlama-bağımlı dil sınıfı kısıtlamasız dil sınıfının bir alt sınıfıdır.
2. Her bağlamdan-bağımsız dil aynı zamanda bir bağlama-bağımlı dildir. Başka bir deyişle bağlamdan-bağımsız dil sınıfı, bağlama-bağımlı dil sınıfının bir alt sınıfıdır.
3. Her Düzgün dil aynı zamanda bir bağlamdan-bağımsız dildir. Başka bir deyişle düzgün diller sınıfı, bağlamdan-bağımsız dil sınıfının bir alt sınıfıdır.



Çizim 6.2. Biçimsel Dil Sınıfları Sıradüzeni : Chomsky Sıradüzeni

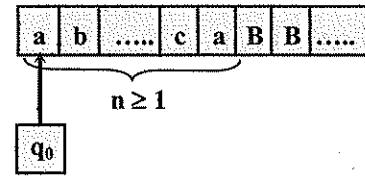
Sorular

S.6.1. $L_{S.6.1}$ dili, $\{a, b, c\}$ alfabesinde, içindeki b ve c 'lerin sayılarının toplamı, a 'ların sayısının iki katı olan dil olarak tanımlanıyor. Bu dil λ içermiyor. Dilin tümcelerinden birkaç örnek aşağıda görülmektedir.

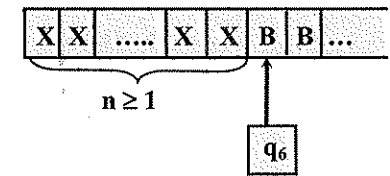
$$L_{S.6.1} = \{abb, cab, aacbbb, cbabca, baacacbbc, \dots\}$$

Aşağıdaki Turing makinesinin tanımını, $L_{S.6.1}$ dilini tanıyan bir Turing makinesi oluşturacak biçimde tamamlamanız isteniyor.

Başlangıç konfigürasyonu:



Bitiş konfigürasyonu:



$$M_{S.6.1} = \langle Q, \Sigma, \Gamma, \delta, q_0, B, F \rangle$$

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6\}$$

$$\Sigma = \{a, b, c\}$$

$$\Gamma = \{a, b, c, X, Y, B\}$$

$$q_0 : \text{Başlangıç durumu}$$

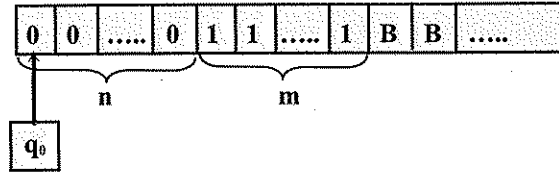
$$F = \{q_6\}$$

- $L_{S.6.1}$ dilinin tümceleri $3n$ uzunluğundadır ($n \geq 1$). Bir tümce verildiğinde, eğer tümce geçerli bir tümce ise, $M_{S.6.1}$ makinesi tümceyi soldan sağa en çok $(n+1)$ kez, sağdan sola da en çok n kez tarayarak bitiş konfigürasyonuna ulaşacaktır. Bu bağlamda, $M_{S.6.1}$ 'in çalışma ilkesini açıklayınız.
- Makinenin biçimsel tanımını tamamlayınız. Bu bağlamda makinenin hareketlerini bir geçiş çizeneği ile gösteriniz.

$$S.6.2. \quad f(n, m) = |n - m| \quad n \geq 1, \quad m \geq 1$$

Yukarıdaki işlevin değerini hesaplayan bir Turing makinesini tasarlamamız isteniyor.

Başlangıç konfigürasyonu:

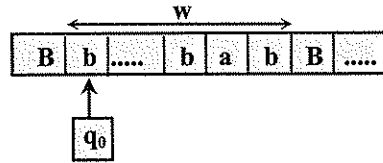


- Makinenin nasıl çalışacağını sözel olarak açıklayınız.
- Makinenin biçimsel tanımını veriniz. Bu bağlamda makinenin hareketlerini hem bir çizelge, hem de bir çizenek ile gösteriniz. Bitiş konfigürasyonunu gösteren bir/birkaç çizim oluşturunuz.

S.6.3. $L_{S.6.3}$ dili, $\{a, b\}$ alfabesinde içindeki b 'lerin sayısı a 'ların sayısının en az 2 katı olan dizgiler kümesi olarak tanımlanıyor. Bu dil λ içermiyor.

$$L_{S.6.3} = \{b, bb, bbb, \dots, abb, bab, bba, babb, \dots\}$$

$L_{S.6.3}$ dilinin tümcelerini tanıyan deterministik bir Turing makinesi (TM) tasarlamamız isteniyor. TM'nin başlangıç konfigürasyonu aşağıdaki gibidir.



- TM'nin nasıl çalışacağını sözel olarak açıklayınız.
- Durumları belirleyip anlamlarını yazınız. TM'nin işlem verimliliğini önemsemeyiniz. TM'nin doğru çalışması ve durum sayısının olabildiğince az (en çok 7) olması önemlidir. TM'nin tanıdığı dizgiler için bitiş konfigürasyonunun nasıl olacağını bir çizimle gösteriniz.
- TM'nin biçimsel tanımını, hareketleri hem bir çizenek ile, hem de bir çizelge ile göstererek veriniz.
- Tasarladığınız TM'yi,

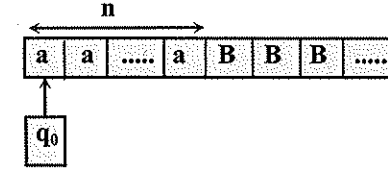
$$w_1 = bab$$

$$w_2 = abbbbab$$

tümceleri için deneyiniz ve TM'nin doğru çalıştığını gösteriniz.

S.6.4. $f(n) = \lceil n/2 \rceil \times 3$ $n \geq 1$

olarak tanımlanan fonksiyonun değerini hesaplayan deterministik bir Turing makinesi (TM) tasarlamamız isteniyor. TM'nin başlangıç konfigürasyonu aşağıdaki gibidir.

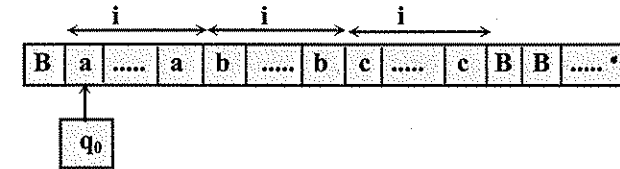


- TM'nin nasıl çalışacağını sözel olarak açıklayınız.
- Durumları belirleyip anlamlarını yazınız (durum sayısının olabildiğince az olması isteniyor). Bitiş konfigürasyonunu bir çizimle gösteriniz.
- TM'nin biçimsel tanımını, hareketleri hem bir çizenek hem de bir çizelge ile göstererek veriniz.
- Tasarladığınız TM'yi $n=1$, $n=2$ ve $n=3$ için deneyiniz ve doğru çalıştığını gösteriniz.

S.6.5. $L_{S.6.5}$ dili aşağıdaki gibi tanımlanıyor:

$$L_{S.6.5} = \{a^i b^i c^i \mid i \geq 1\}$$

$L_{S.6.5}$ dilini tanıyan deterministik bir Turing makinesi (TM) oluşturmanız isteniyor. TM'nin başlangıç konfigürasyonu ile giriş ve şerit alfabelerinin aşağıdaki gibi olması isteniyor.



TM'nin giriş alfabesi $\Sigma = \{a, b, c\}$, şerit alfabesinin de $\Gamma = \{a, b, c, X, B\}$ olması istenmektedir.

- TM'nin nasıl çalışacağını sözel olarak açıklayınız.
- Durumları belirleyip anlamlarını yazınız (durum sayısının olabildiğince az olması isteniyor).
- TM'nin biçimsel tanımını, hareketleri hem bir çizelge hem de bir çizenek ile göstererek veriniz.
- İstedığınız gösterimi kullanarak, birkaç tümce için tasarladığınız TM'nin doğru çalıştığını gösteriniz. Bu tümceler arasında en az bir doğru (örneğin

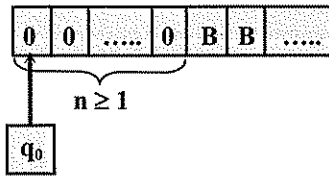
$w_1 = aabbcc$ tümce ile en az iki yanlış (örneğin $w_2 = abbc$ ve $w_3 = aabbc$) tümce yer alsın.

S.6.6. $n \geq 1$ olmak üzere $f(n)$ işlevi aşağıdaki gibi tanımlanıyor:

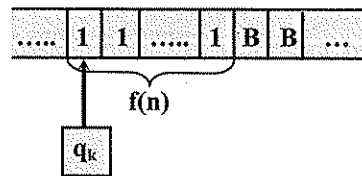
$$f(n) = \lfloor n/3 \rfloor$$

Verilen n değerine göre $f(n)$ işlevinin değerini hesaplayan deterministik bir Turing makinesi tasarlamamız isteniyor. Makinenin başlangıç ve bitiş konfigürasyonlarının aşağıdaki gibi olması isteniyor:

Başlangıç konfigürasyonu:



Bitiş konfigürasyonu:

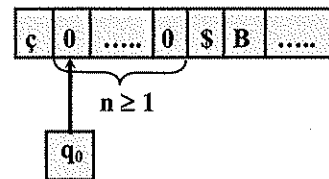


- Makinenin nasıl çalışacağını sözel olarak açıklayınız.
- Durum sayısının olabildiğince az olmasını gözeterek makinenin durumlarını belirleyip anlamlarını yazınız.
- Hareketleri bir çizenelekle göstererek makinenin biçimsel tanımını oluşturunuz.

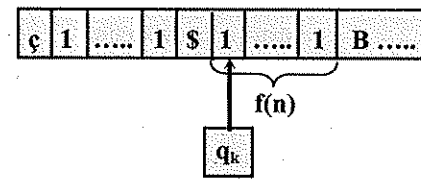
S.6.7. $n \geq 1$ olmak üzere $f(n) = n^2$ işlevi tanımlanıyor.

Verilen n değerine göre $f(n)$ işlevinin değerini hesaplayan deterministik bir Turing makinesi tasarlamamız isteniyor. Makinenin başlangıç ve bitiş konfigürasyonlarının aşağıdaki gibi olması isteniyor:

Başlangıç konfigürasyonu:



Bitiş konfigürasyonu:

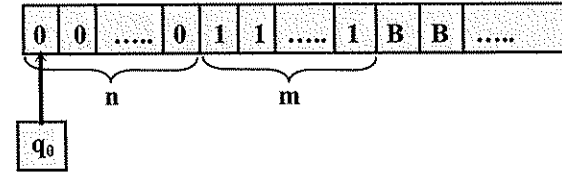


- Makinenin nasıl çalışacağını sözel olarak açıklayınız.
- Hareketleri bir çizenelekle göstererek makinenin biçimsel tanımını oluşturunuz.

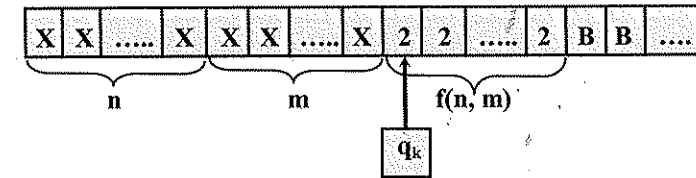
S.6.8. $f(n, m) = \lceil (n+m)/2 \rceil$ $n \geq 1, m \geq 1$

Yukarıdaki işlevin değerini hesaplayan bir Turing makinesini tasarlamamız isteniyor.

Başlangıç konfigürasyonu:



Bitiş konfigürasyonu:



- Makinenin nasıl çalışacağını sözel olarak açıklayınız.
- Makinenin biçimsel tanımını veriniz. Bu bağlamda makinenin hareketlerini bir çizenelekle gösteriniz.

S.6.9. $L_{S.6.9} = \{a^n b^m c^n \mid n > 0, m > 0, n \geq m\}$

Yukarıda küme tanımı verilen $L_{S.6.9}$ dilini tanıyan bir Turing makinesi (TM) oluşturmamız isteniyor. Bunun için:

- TM'nin nasıl çalışacağını sözel olarak açıklayınız.
- Durumları belirleyip anlamlarını yazınız (durum sayısının olabildiğince az olması isteniyor).
- TM'nin biçimsel tanımını, hareketleri hem bir çizelge, hem de bir çizenelekle göstererek, veriniz.

S.6.10. $M_{S.6.10} = \langle Q, \Sigma, \Gamma, \delta, q_0, B, F \rangle$

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6\}$$

$$\Sigma = \{0, 1\}$$

$$\Gamma = \{0, 1, B, X, ç, \$\}$$

$$F = \{q_6\}$$

δ :

	ϵ	0	1	\$	X	B
q_0	-	-	(q_1, X, R)	-	-	-
q_1	-	$(q_1, 0, R)$	$(q_1, 1, R)$	$(q_2, \$, R)$	-	-
q_2	-	-	$(q_2, 1, R)$	-	-	$(q_3, 1, L)$
q_3	-	$(q_3, 0, L)$	$(q_3, 1, L)$	$(q_3, \$, L)$	(q_4, X, R)	-
q_4	-	$(q_5, 0, L)$	(q_1, X, R)	$(q_5, \$, L)$	-	-
q_5	(q_6, ϵ, R)	-	-	-	$(q_5, 1, L)$	-
q_6	-	-	-	-	-	-

a) Aşağıdaki başlangıç konfigürasyonlarından her biri için makinenin çıkış konfigürasyonunu bulunuz.

1) $\epsilon 1 0 1 1 0 0 1 \$ B B \dots$
 $\uparrow$

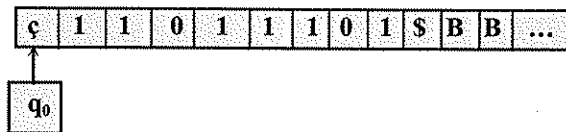
2) $\epsilon 1 1 1 1 \$ B B \dots$
 $\uparrow$

3) $\epsilon 0 1 1 1 0 1 \$ B B \dots$
 $\uparrow$

4) $\epsilon 1 1 1 0 1 1 0 1 1 1 0 \$ B B \dots$
 $\uparrow$

b) Makinenin çalışmasını sözel olarak açıklayınız.

S.6.11. $\{0, 1\}$ alfabesinde, uzunluğu en az iki olan ve içinde çift sayıda 0 ile çift sayıda 1 bulunan dizgiler kümesini tanıyan bir Turing makinesi tasarlamamız isteniyor. Örnek bir giriş dizgisi için makinenin başlangıç konfigürasyonu aşağıda görülmektedir



a) Turing makinesinin nasıl çalışacağını açıklayınız.

b) TM'nin biçimsel tanımını, hareketleri hem bir çizelge, hem de bir çizenek ile göstererek, veriniz.

S.6.12. $M_{S.6.12} = \langle Q, \Sigma, \Gamma, \delta, q_0, B, F \rangle$

$Q = \{q_0, q_1, q_2, q_3, q_4\}$

$\Sigma = \{0, 1\}$

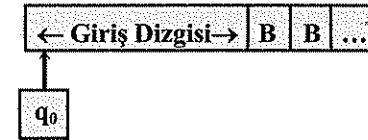
$\Gamma = \{0, 1, B\}$

$F = \{q_4\}$

δ :

	0	1	B
q_0	(q_1, B, R)	-	(q_4, B, R)
q_1	$(q_1, 0, R)$	$(q_1, 1, R)$	(q_2, B, L)
q_2	-	(q_3, B, L)	-
q_3	$(q_3, 0, L)$	$(q_3, 1, L)$	(q_0, B, R)
q_4	-	-	-

Başlangıç konfigürasyonu:



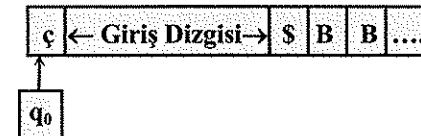
a) Yukarıda biçimsel tanımı verilen Turing makinesinin çalışmasını sözel olarak açıklayınız ve tanıdığı dilin küme tanımını veriniz.

b) Turing makinesinin tanıdığı dilin türü nedir? Bu dili türeten bir dilbilgisi tanımlayınız.

c) $w = 011$ için anlık tanımlar (ID) dizisini oluşturarak makinenin dizgiyi tanıyıp tanımadığını bulunuz.

S.6.13. $\{0, 1\}$ alfabesinde, eşit sayıda 0 ve 1 içeren dizgiler kümesini tanıyan bir Turing makinesi tasarlayınız.

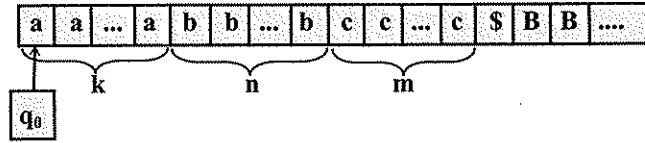
Başlangıç konfigürasyonu:



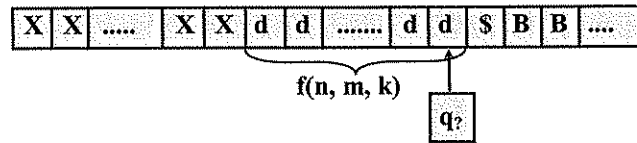
S.6.14. $f(n, m, k) = n + m - k$ $n, m, k \geq 1$, $n + m \geq k$

İşlevinin değerini hesaplayan bir Turing Makinesi ($M_{S.6.14}$) tasarlamamız isteniyor.

Başlangıç konfigürasyonu:



Bitiş konfigürasyonu:



- a) Makinenin nasıl çalışacağını sözel olarak açıklayınız.
- b) Makinenin biçimsel tanımını veriniz. Biçimsel tanımda TM'nin hareketlerini bir çizenek ile gösteriniz.

Not : 1. Şerit üzerinde \$ simgesinin sağındaki hücrelerde hiçbir değişiklik yapılamayacak; işlevin değeri \$ simgesinin solunda oluşturulacaktır (başlangıç ve bitiş konfigürasyonlarını dikkatlice inceleyiniz).

2. k'nın değeri n ve m'nin herbirinin değerinden küçük, eşit, ya da büyük olabilir. Ancak k'nın değerinin (n+m)'den büyük olmadığı varsayılacaktır.

S.6.15. $M_{S.6.15} = \langle Q, \Sigma, \Gamma, \delta, q_0, B, F \rangle$

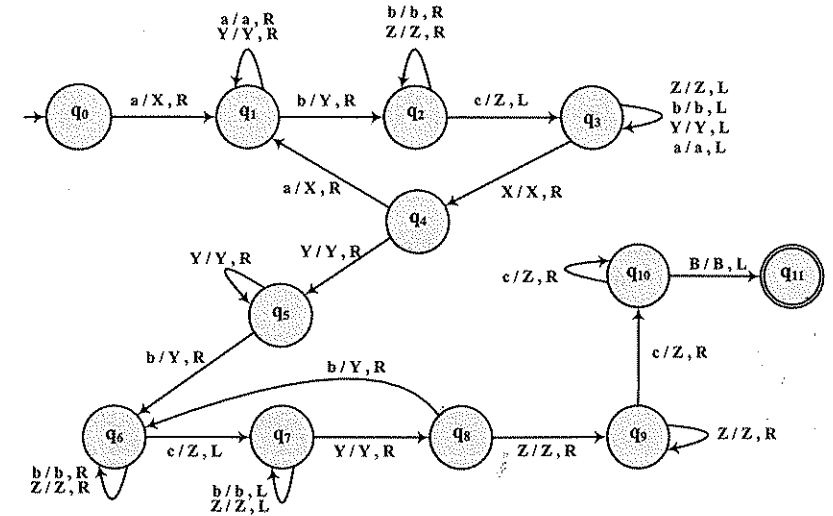
$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_8, q_9, q_{10}, q_{11}\}$$

$$\Sigma = \{a, b, c\}$$

$$\Gamma = \{a, b, c, X, Y, Z, B\}$$

q_0 : Başlangıç durumu

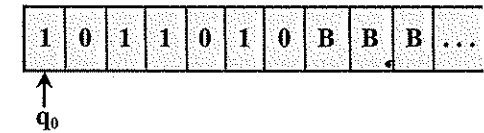
$$F = \{q_{11}\}$$



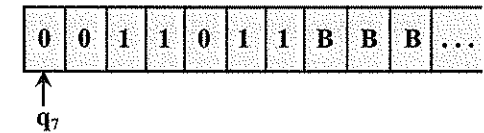
- a) Yukarıda tanımlanan temel model Turing makinesinin tanıdığı dilin ($L_{S.6.15}$) küme tanımını, tam doğru ve eksiksiz olarak veriniz.
- b) $M_{S.6.15}$ tarafından tanınan en az 7 uzunluğunda bir tümce yazınız ve bu tümce için makinenin başlangıç ve bitiş konfigürasyonlarını gösteriniz.

S.6.16. $M_{S.6.16}$ Turing makinesinin şeridinde $\{0, 1\}$ alfabesinde bir dizgi (w) yer almaktadır. Makinenin, dizginin en solundaki simge ile en sağdaki simgenin yerlerini değiştirmesi istenmektedir.

Örnek başlangıç konfigürasyonu :



Örnek bitiş konfigürasyonu :



$M_{S.6.16} = \langle Q, \Sigma, \Gamma, \delta, q_0, B, F \rangle$

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7\}$$

$$\Sigma = \{0, 1\}$$

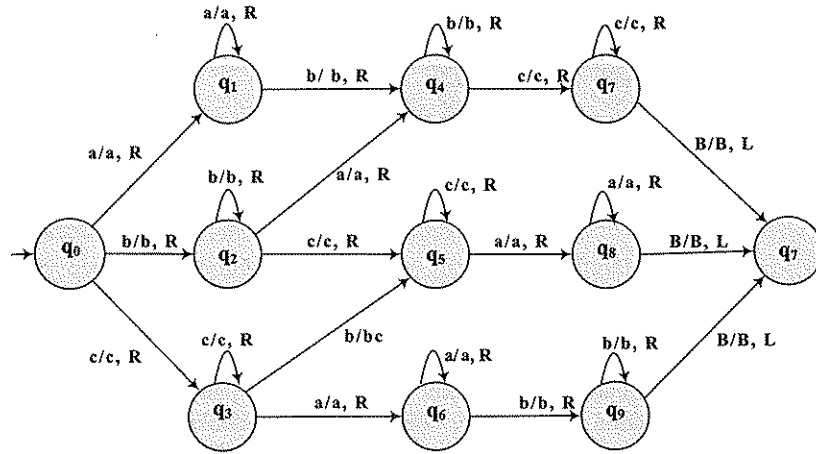
$$\Gamma = \{0, 1, B\}$$

$$F = \{q_7\}$$

Hareketleri (δ) bir çizenekle göstererek $M_{S.6.16}$ 'nın tanımını tamamlayınız.

Not: Makine sadece uçlardaki iki simgeyi değiştirecek, dizginin diğer simgelerine dokunmayacaktır. Durum sayısının da olabildiğince az olması istenmektedir. Makinenin hareketlerinde **L** ve **R** yanında **S** seçeneğini de kullanabilirsiniz.

S.6.17. Aşağıda hareket çizeneği görülen Turing makinesi tarafından tanınan $L_{S.6.17}$ dilini tanımlayan bir düzgün deyim yazınız ($L_{S.6.17}$ dili $\{a, b, c\}$ alfabesinde tanımlı bir dildir).



S.6.18. Bağlamdan-bağımsız $L_{S.6.18}$ dili aşağıdaki gibi tanımlanıyor.

$$L_{S.6.18} = (ab^*a)^n (cd^*c)^n \quad n \geq 1$$

- $L_{S.6.18}$ dilini türeten bağlamdan bağımsız bir dilbilgisi tanımlayınız. Tanımlayacağınız dilbilgisinde olabildiğince az değişken bulunsun.
- $L_{S.6.18}$ dilini hem uç durum hem de boş yığıla tanıyan bir PDA tanımlamanız isteniyor.

$$PDA = \langle Q, \Sigma, \Gamma, \delta, q_0, Z_0, F \rangle$$

$$Q = \{q_0, \dots\}$$

$$\Sigma = \{a, b, c, d\}$$

$$\Gamma = \{Z_0, A\}$$

q_0 : Başlangıç durumu.

$$F = \{q_f\}$$

PDA'nın çalışma ilkesini sözlü olarak açıklayınız.

Durumlar kümesini tamamlayıp Geçiş Çizeneğini (δ) oluşturunuz.

Dizgi tanındığında hem yığıtın boşalması hem de PDA'nın uç durumda (q_f) bulunması isteniyor.

c) $L_{S.6.18}$ dilini tanıyan bir Turing makinesi tanımlamanız isteniyor.

$$M_{S.6.18} = \langle Q, \Sigma, \Gamma, \delta, q_0, B, F \rangle$$

$$Q = \{q_0, \dots\}$$

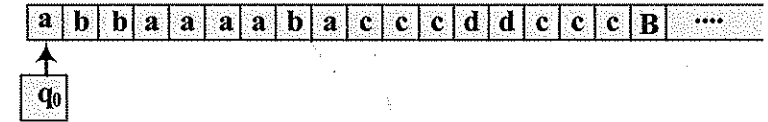
$$\Sigma = \{a, b, c, d\}$$

$$\Gamma = \{a, b, c, d, X, Y, B\}$$

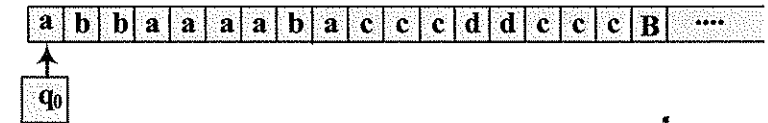
q_0 : Başlangıç durumu.

$$F = \{q_f\}$$

Başlangıç kofigürasyonu (örnek dizgi $w = abbaaaabacccddccccc$ için)



Bitiş kofigürasyonu (örnek dizgi $w = abbaaaabacccddccccc$ için)



Durumlar kümesini tamamlayıp uç durumu (bir tane) belirleyiniz ve makinenin hareket çizeneğini (δ) oluşturunuz.

Tasarlayacağınız makinenin yukarıda verilen Γ ile başlangıç ve bitiş kofigürasyonlarına tam uygun olması istenmektedir.